

universidad
de león

Departamento de Ingenierías
Mecánica, Informática y Aeroespacial

MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Máster

Alternativas a la simulación de robots de exteriores
basada en Gazebo para ROS 2

Alternatives to outdoor robot simulation based in
Gazebo for ROS 2

Autor: Jorge Casatejada Santamarta

Tutor: Francisco Javier Rodríguez Lera

Co-Tutor: Miguel Ángel González Santamarta

(Septiembre, 2025)

UNIVERSIDAD DE LEÓN Departamento de Ingenierías Mecánica, Informática y Aeroespacial MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL Trabajo de Fin de Máster
ALUMNO: Jorge Casatejada Santamarta
TUTOR: Francisco Javier Rodríguez Lera
TÍTULO: Alternativas a la simulación de robots de exteriores basada en Gazebo para ROS 2
CONVOCATORIA: Septiembre, 2025
RESUMEN: La simulación es una herramienta esencial en la robótica para reducir costes de hardware, mitigar riesgos y acelerar los ciclos de desarrollo. Su importancia es aún mayor en la robótica de exteriores, donde la replicación de entornos complejos y dinámicos es costosa y complicada. Aunque Gazebo es el simulador de referencia en el ecosistema ROS 2, su elevado consumo de recursos computacionales presenta limitaciones significativas, especialmente en hardware limitado o en simulaciones exigentes. El objetivo de este TFM es realizar un análisis cuantitativo y comparativo del rendimiento de simuladores de robótica alternativos a Gazebo compatibles con ROS 2, con un enfoque en exteriores. Para ello, se diseñó y ejecutó una serie de experimentos automatizados en un entorno de hardware controlado, evaluando estos tres simuladores: Ignition Gazebo, Webots e Isaac Sim. Se midieron métricas clave de rendimiento bajo distintas configuraciones, variando el uso de la interfaz gráfica (GUI), la aceleración por GPU y el estado de la batería. Las métricas recopiladas incluyeron el consumo de CPU, GPU, memoria RAM, consumo energético y el Factor de Tiempo Real (RTF).

Los resultados muestran que, si bien los tres simuladores permiten obtener escenarios funcionalmente similares, presentan diferencias notables en su rendimiento sobre el hardware disponible. Webots presenta un consumo de recursos significativamente menor en todas las configuraciones evaluadas, manteniendo un RTF cercano a 1, con un uso de CPU y memoria notablemente inferior al de Ignition Gazebo. Por el contrario, Ignition Gazebo muestra una mayor demanda computacional y una degradación del RTF bajo diferentes configuraciones. Isaac Sim demostró ser el más exigente en recursos, necesitando una GPU de alta gama que lo hace inviable en muchos casos.

Se concluye que Webots se posiciona como una alternativa viable a Gazebo para la simulación de robótica de exteriores en ROS 2, especialmente en sistemas con recursos computacionales limitados. Además del análisis, el trabajo aporta un conjunto de tutoriales de instalación de los simuladores, scripts para la automatización de pruebas y un ejemplo práctico completo de navegación en exteriores con Webots y Nav2, contribuyendo así a la comunidad con herramientas para la reproducibilidad y la "ciencia abierta".

Palabras clave: ROS 2, Gazebo, Isaac Sim, Webots, GUI, GPU, CPU

Firma del alumno:

VºBº Tutor:

(September, 2025)

UNIVERSITY OF LEÓN Department of Mechanical, Computer and Aerospace Engineering MASTER'S DEGREE IN ROBOTICS AND ARTIFICIAL INTELLIGENCE Master's Thesis
STUDENT: Jorge Casatejada Santamarta
SUPERVISOR: Francisco Javier Rodríguez Lera
TITLE: Alternatives to outdoor robot simulation based on Gazebo for ROS 2
CALL: September, 2025
ABSTRACT: Simulation is an essential tool in robotics to reduce hardware costs, mitigate risks, and accelerate development cycles. Its importance is even greater in outdoor robotics, where replicating complex and dynamic environments is costly and challenging. Although Gazebo is the reference simulator in the ROS 2 ecosystem, its high computational resource consumption presents significant limitations, especially on limited hardware or demanding simulations. The objective of this Master's Thesis is to conduct a quantitative and comparative performance analysis of robotic simulators alternative to Gazebo that are compatible with ROS 2, focusing on outdoor environments. For this purpose, a series of automated experiments were designed and executed in a controlled hardware environment, evaluating these three simulators: Ignition Gazebo, Webots, and Isaac Sim. Key performance metrics were measured under different configurations, varying the use of the graphical interface (GUI), GPU acceleration, and battery state. The collected metrics included CPU, GPU, and RAM usage, energy consumption, and the Real-Time Factor (RTF). The results show that although all three simulators allow the creation of functionally similar scenarios, they present notable differences in performance on the available hardware. Webots shows significantly lower resource consumption across all evaluated configurations, maintaining an RTF close to 1 with notably lower CPU and memory usage compared to Ignition Gazebo. In contrast, Ignition Gazebo requires more computational resources and exhibits RTF degradation under different configurations. Isaac Sim proved to be the most resource-demanding, requiring a high-end GPU, which makes it unfeasible in many cases.

It is concluded that Webots positions itself as a viable alternative to Gazebo for outdoor robotic simulation in ROS 2, especially in systems with limited computational resources. In addition to the analysis, this work provides a set of installation tutorials for the simulators, scripts for test automation, and a complete practical example of outdoor navigation with Webots and Nav2, thus contributing to the community with tools for reproducibility and open science.

Keywords: ROS 2, Gazebo, Isaac Sim, Webots, GUI, GPU, CPU

Student's signature:

Supervisor's approval:

Índice general

Índice de figuras	V
Índice de tablas	VII
Glosario de términos	X
Introducción	1
1. Estudio del problema	6
1.1. El contexto del problema	6
1.2. El estado de la cuestión	7
1.2.1. Principales simuladores de robótica	7
1.2.2. Posibles simuladores sustituyentes de Gazebo	10
1.3. La definición del problema: El simulador como herramienta software .	11
1.3.1. Gazebo	11
1.3.2. Webots	12
1.3.3. NVIDIA Isaac Sim	13
1.4. La definición del problema: Coste computacional asociado a la simu- lación	14
1.4.1. Coste computacional por simulador	14
1.4.2. Lanzar con/sin GUI	15
1.4.3. Lanzar con/sin GPU	18
1.4.4. Tiempo Real	21
1.5. Tabla comparativa resumen	27
2. Gestión de proyecto software	29
2.1. Alcance del proyecto	29
2.2. Plan de trabajo	30
2.2.1. Identificación de tareas	30

2.2.2.	Estimación de tareas	32
2.2.3.	Planificación de tareas	33
2.2.4.	Iteraciones de Scrum	33
2.3.	Gestión de recursos	39
2.3.1.	Especificación de recursos	39
2.3.2.	Presupuesto	40
2.4.	Gestión de riesgos	43
2.4.1.	Identificación de riesgos	43
2.4.2.	Mitigación de riesgos	44
2.5.	Legislación y normativa	45
3.	Solución	47
3.1.	Metodologías de evaluación	47
3.2.	El proceso de desarrollo	48
3.2.1.	Análisis	48
3.2.2.	Diseño	54
3.2.3.	Implementación	60
3.3.	El producto del desarrollo	65
4.	Evaluación	67
4.1.	Proceso de evaluación (Análisis Individual de Simuladores)	67
4.1.1.	Ignition Gazebo	67
4.1.2.	Webots	70
4.1.3.	Isaac Sim	72
4.2.	Proceso de evaluación (Comparativa entre Simuladores)	74
4.2.1.	Cargando - GUI - No GPU	74
4.2.2.	Cargando - GUI - GPU	75
4.2.3.	Cargando - No GUI - No GPU	75
4.2.4.	Cargando - No GUI - GPU	76
4.2.5.	Sin Cargar - GUI - No GPU	77
4.2.6.	Sin Cargar - GUI - GPU	77
4.2.7.	Sin Cargar - No GUI - No GPU	78
4.2.8.	Sin Cargar - No GUI - GPU	79
4.3.	Análisis de resultados	80
4.3.1.	Análisis Individual por Simulador	80
4.3.2.	Análisis Comparativo entre Simuladores	85
4.4.	Explicación detallada de Webots	89
4.4.1.	Porqué Webots	89

4.4.2. Webots: Descripción detallada	89
4.4.3. Funcionalidad básica de Webots	90
4.4.4. Robots y mundos exteriores disponibles	91
4.4.5. Compatibilidad con ROS 2	94
4.4.6. Webots con ROS 2 y NAV2	96
4.5. Ejemplo en Webots ROS 2 y Nav2	99
5. Conclusión	101
Lista de referencias	109
A. Control de versiones	119
B. Datos Experimentales Completos	120
B.1. Análisis Individual de Simuladores	120
B.1.1. Ignition Gazebo	120
B.1.2. Webots	129
B.1.3. Isaac Sim	137
B.2. Comparativa entre Simuladores	143
B.2.1. Cargando - GUI - No GPU	143
B.2.2. Cargando - GUI - GPU	145
B.2.3. Cargando - No GUI - No GPU	148
B.2.4. Cargando - No GUI - GPU	150
B.2.5. Sin Cargar - GUI - No GPU	153
B.2.6. Sin Cargar - GUI - GPU	156
B.2.7. Sin Cargar - No GUI - No GPU	160
B.2.8. Sin Cargar - No GUI - GPU	163
C. Tecnologías utilizadas	167
D. Tutoriales	176
E. Inventario Webots ROS 2	177
E.1. Versiones OS	177
E.2. Versiones ROS 2	177
E.3. Sensores	178
E.4. Actuadores	178
E.5. Robots	179
E.5.1. Con ruedas	179
E.5.2. Con piernas	181

E.5.3. Brazos robóticos	181
E.5.4. Voladores	182
E.5.5. Otros	182
E.6. Vehículos	182
E.7. Mundos	183
E.8. Nodos	184
E.8.1. WebotsLauncher	185
E.8.2. WebotsController	186
E.8.3. WaitForControllerConnection	187

Índice de figuras

1.1. Ejecución de Ignition Gazebo sin GUI	16
1.2. Ejecución de Webots sin GUI	17
1.3. Ejecución de Isaac Sim sin GUI	18
1.4. Simuladores sin GPU	19
1.5. Simuladores con GPU	20
1.6. Gazebo Classic con $RTF \approx 2$	21
1.7. Gazebo Classic con $RTF \approx 0.5$	22
1.8. Ajuste de RTF en Gazebo Classic	22
1.9. Ignition Gazebo con $RTF \approx 2$	23
1.10. Ignition Gazebo con $RTF \approx 0.5$	24
1.11. Ajuste de RTF en Ignition Gazebo	24
1.12. Webots en modo realtime ($RTF \approx 1$)	25
1.13. Webots en modo fast ($RTF \approx 80$)	25
1.14. Configuración de basicTimeStep en Webots	26
1.15. Webots con RTF reducido	26
1.16. Isaac Sim con $RTF \approx 0.8$	27
2.1. Diagrama de Gantt de la planificación inicial de las tareas	33
4.1. Ignition Gazebo - Comparación del RTF dependiendo batería	80
4.2. Ignition Gazebo - Comparación del uso de CPU con/sin GUI	81
4.3. Ignition Gazebo - Comparación del gasto de batería con/sin GPU	82
4.4. Webots - Comparación de la frecuencia dependiendo batería	82
4.5. Webots - Comparación del uso de CPU con/sin GUI	83
4.6. Isaac Sim - Comparación de la temperatura de GPU dependiendo de la batería	84
4.7. Comparación de simuladores - CPU Percent	86
4.8. Comparación de simuladores - GPU Power	86
4.9. Comparación de simuladores - RAM Active MB	87

4.10. Comparación de simuladores - RTF	87
4.11. Comparación de simuladores - Batería	88
4.12. Comparación de simuladores - Con/Sin GUI - CPU Percent	88
4.13. Diagrama del pipeline entre Webots y ROS 2	98
4.14. Captura del ejemplo en Webots mostrando el uso de CPU, GPU y el RTF durante la navegación en un entorno exterior.	99
A.1. Repositorio Github	119

Índice de tablas

3.	Resumen de tecnologías utilizadas	4
1.1.	Alternativas a Gazebo	11
1.2.	Tabla comparativa resumen de simuladores robóticos	27
2.1.	Estimación de tareas	32
2.2.	Iteraciones de Scrum	39
2.3.	Tasas de la Seguridad Social Española	40
2.4.	Coste del personal	41
2.5.	Costes indirectos	42
2.6.	Beneficio industrial	42
2.7.	Coste total	43
3.1.	Caso de uso 1: Automatización de lanzamiento y navegación	50
3.2.	Caso de uso 2: Medición y registro del consumo energético	50
3.3.	Caso de uso 3: Medición y registro de métricas de CPU, RAM y GPU	51
3.4.	Caso de uso 4: Medición y registro del factor de tiempo real (RTF)	52
3.5.	Caso de uso 5: Integración y automatización completa del proceso	53
3.6.	Caso de uso 6: Desarrollo y documentación de ejemplo y tutorial	54
3.7.	Caso de uso 7: Análisis estadístico de los datos	54
3.8.	Escenarios considerados por simulador	55
3.9.	Escenarios considerados Isaac Sim	56
3.10.	Matriz de escenarios y configuraciones evaluadas en los simuladores	65
4.1.	Comparación de métricas clave entre simuladores.	85
B.1.	Ignition Gazebo - Estado batería - CPU	120
B.2.	Ignition Gazebo - Estado batería - GPU	121
B.3.	Ignition Gazebo - Estado batería - RAM	122
B.4.	Ignition Gazebo - Estado batería - RTF	122

B.5. Ignition Gazebo - Uso GUI - CPU	123
B.6. Ignition Gazebo - Uso GUI - GPU	124
B.7. Ignition Gazebo - Uso GUI - RAM	124
B.8. Ignition Gazebo - Uso GUI - RTF	125
B.9. Ignition Gazebo - Uso GUI - Batería	125
B.10. Ignition Gazebo - Uso GPU - CPU	126
B.11. Ignition Gazebo - Uso GPU - GPU	127
B.12. Ignition Gazebo - Uso GPU - RAM	127
B.13. Ignition Gazebo - Uso GPU - RTF	128
B.14. Ignition Gazebo - Uso GPU - Batería	128
B.15. Webots - Estado batería - CPU	129
B.16. Webots - Estado batería - GPU	130
B.17. Webots - Estado batería - RAM	130
B.18. Webots - Estado batería - RTF	131
B.19. Webots - Uso GUI - CPU	131
B.20. Webots - Uso GUI - GPU	132
B.21. Webots - Uso GUI - RAM	133
B.22. Webots - Uso GUI - RTF	133
B.23. Webots - Uso GUI - Batería	134
B.24. Webots - Uso GPU - CPU	134
B.25. Webots - Uso GPU - GPU	135
B.26. Webots - Uso GPU - RAM	136
B.27. Webots - Uso GPU - RTF	136
B.28. Webots - Uso GPU - Batería	137
B.29. Isaac Sim - Estado Batería - CPU	137
B.30. Isaac Sim - Estado Batería - GPU	138
B.31. Isaac Sim - Estado Batería - RAM	139
B.32. Isaac Sim - Estado Batería - RTF	139
B.33. Isaac Sim - Uso GUI - CPU	140
B.34. Isaac Sim - Uso GUI - GPU	141
B.35. Isaac Sim - Uso GUI - RAM	141
B.36. Isaac Sim - Uso GUI - RTF	142
B.37. Isaac Sim - Uso GUI - Batería	142
B.38. Comparación Simuladores Cargando - GUI - No GPU - CPU	143
B.39. Comparación Simuladores Cargando - GUI - No GPU - RAM	144
B.40. Comparación Simuladores Cargando - GUI - No GPU - RTF	144
B.41. Comparación Simuladores Cargando - GUI - GPU - CPU	145

B.42.Comparación Simuladores Cargando - GUI - GPU - GPU	146
B.43.Comparación Simuladores Cargando - GUI - GPU - RAM	146
B.44.Comparación Simuladores Cargando - GUI - GPU - RTF	147
B.45.Comparación Simuladores Cargando - No GUI - No GPU - CPU . . .	148
B.46.Comparación Simuladores Cargando - No GUI - No GPU - RAM . .	149
B.47.Comparación Simuladores Cargando - No GUI - No GPU - RTF . . .	149
B.48.Comparación Simuladores Cargando - No GUI - GPU - CPU	150
B.49.Comparación Simuladores Cargando - No GUI - GPU - GPU	151
B.50.Comparación Simuladores Cargando - No GUI - GPU - RAM	152
B.51.Comparación Simuladores Cargando - No GUI - GPU - RTF	153
B.52.Comparación Simuladores Sin Cargar - GUI - No GPU - CPU	154
B.53.Comparación Simuladores Sin Cargar - GUI - No GPU - RAM	154
B.54.Comparación Simuladores Sin Cargar - GUI - No GPU - RTF	155
B.55.Comparación Simuladores Sin Cargar - GUI - No GPU - Batería . . .	156
B.56.Comparación Simuladores Sin Cargar - GUI - GPU - CPU	156
B.57.Comparación Simuladores Sin Cargar - GUI - GPU - GPU	157
B.58.Comparación Simuladores Sin Cargar - GUI - GPU - RAM	158
B.59.Comparación Simuladores Sin Cargar - GUI - GPU - RTF	159
B.60.Comparación Simuladores Sin Cargar - GUI - GPU - Batería	159
B.61.Comparación Simuladores Sin Cargar - No GUI - No GPU - CPU . .	160
B.62.Comparación Simuladores Sin Cargar - No GUI - No GPU - RAM . .	161
B.63.Comparación Simuladores Sin Cargar - No GUI - No GPU - RTF . .	162
B.64.Comparación Simuladores Sin Cargar - No GUI - No GPU - Batería .	162
B.65.Comparación Simuladores Sin Cargar - No GUI - GPU - CPU	163
B.66.Comparación Simuladores Sin Cargar - No GUI - GPU - GPU	164
B.67.Comparación Simuladores Sin Cargar - No GUI - GPU - RAM	164
B.68.Comparación Simuladores Sin Cargar - No GUI - GPU - RTF	165
B.69.Comparación Simuladores Sin Cargar - No GUI - GPU - Batería . . .	166

Glosario de términos

CPU (Unidad Central de Procesamiento): Componente hardware que ejecuta instrucciones y procesa datos. Se monitoriza mediante métricas como *cpu_percent*, *cpu_user_percent*, *cpu_freq_MHz*, entre otras.

GPU (Unidad de Procesamiento Gráfico): Componente hardware especializado en el procesamiento de gráficos y operaciones paralelas. Se miden métricas como *gpu_util_percent*, *gpu_mem_used_MB*, *gpu_power_W*, entre otras.

GUI (Interfaz Gráfica de Usuario): Componente visual que permite interactuar con el simulador de forma gráfica.

Headless (Sin GUI): Modo de ejecución de un simulador sin interfaz gráfica, destinado a entornos automatizados o servidores.

Ignition Gazebo / Gazebo Sim: Simulador de robótica de código abierto, ampliamente utilizado en la comunidad ROS.

Isaac Sim: Simulador de NVIDIA basado en Omniverse, orientado a aplicaciones de IA y robótica avanzada.

Nav2 (Navigation2): Stack de navegación para ROS 2 que proporciona planificación de rutas, evasión de obstáculos y localización.

ODE (Open Dynamics Engine): Motor de física utilizado en Webots y Gazebo para simular dinámicas de cuerpos rígidos.

PowerStat: Herramienta utilizada para medir el consumo energético del sistema durante las simulaciones.

RAM (Memoria de Acceso Aleatorio): Memoria volátil utilizada para almacenar datos y programas en ejecución. Se analizan valores como *active_MB*, *cached_MB*, *used_MB*, entre otros.

ROS 2 (Robot Operating System 2): Framework de middleware para el desarrollo de aplicaciones robóticas.

ROS 2 Node: Unidad de procesamiento en ROS 2 que realiza una función específica y se comunica mediante topics, servicios o acciones.

RTF (Real Time Factor): Relación entre el tiempo simulado y el tiempo real transcurrido. Un valor de 1 indica simulación en tiempo real.

Scrum: Metodología ágil de gestión de proyectos utilizada para organizar el desarrollo del TFM.

SDF (Simulation Description Format): Formato utilizado por Gazebo para describir entornos y robots en simulación.

Topic: Canal de comunicación asíncrono en ROS 2 utilizado para intercambiar mensajes entre nodos.

URDF (Unified Robot Description Format): Formato XML para describir la estructura y características de un robot.

Webots: Simulador de robótica de código abierto desarrollado por Cyberbotics, con integración con ROS 2.

Introducción

Planteamiento del problema

La simulación se ha establecido como una herramienta fundamental en el mundo de la robótica, permitiendo probar y validar diversos sistemas robóticos en ámbitos como la navegación o el control en entornos virtuales antes de su implementación en sistemas físicos. Esto hace que se reduzcan significativamente los costes asociados al hardware y a posibles daños durante las pruebas. Además, contribuye a acelerar el proceso de desarrollo, ya que facilita la detección temprana de errores y permite realizar experimentos reproducibles en condiciones controladas y seguras.

El uso de entornos de simulación es especialmente crítico en el ámbito de la robótica de exteriores, donde factores como terrenos irregulares, condiciones meteorológicas variables o la interacción con elementos dinámicos resultan extremadamente difíciles y costosos de replicar en el mundo real. Un simulador proporciona un banco de pruebas indispensable para el avance de la investigación y el desarrollo de aplicaciones robóticas autónomas en escenarios abiertos y no estructurados.

En este contexto, Gazebo se ha establecido como uno de los simuladores más populares, siendo una herramienta ampliamente utilizada para la simulación en robótica en todo tipo de entornos. Sin embargo, presenta limitaciones en cuanto a los recursos computacionales necesarios para lanzar simulaciones complejas y cambiantes propios de la robótica de exteriores, como terrenos irregulares, vegetación dinámica o condiciones meteorológicas. Este TFM tiene como objetivo explorar y comparar alternativas a Gazebo para la simulación de robots en exteriores bajo ROS 2. El trabajo incluirá una revisión del estado del arte, la integración de al menos una alternativa con ROS 2, Webots, y la evaluación de su rendimiento, realismo y facilidad de uso en escenarios típicos de campo.

Objetivos

En este Trabajo de Fin de Máster, se plantean una serie de objetivos interconectados que tienen como finalidad principal:

Identificar entornos de simulación eficientes en consumo de recursos para escenarios de simulación de exteriores, priorizando compatibilidad con ROS 2.

De este modo, se pretenden abordar los siguientes objetivos:

- O_1 Comparar el consumo de recursos computacionales entre varios simuladores de robótica de libre acceso y disponibilidad inmediata (off-the-shelf) (Ignition Gazebo, Webots e Isaac Sim).
- O_2 Analizar una de las alternativas a Ignition Gazebo más utilizadas (Webots).
- O_3 Desarrollar un ejemplo de robótica de exteriores en ROS 2 y Webots.
- O_4 Preparar un conjunto de tutoriales de instalación de varios simuladores de robótica de libre acceso y disponibilidad inmediata para favorecer la "ciencia abierta" (Gazebo Classic, Ignition Gazebo, Webots e Isaac Sim).

En conjunto, estos objetivos proporcionarán una base sólida para seleccionar el simulador más eficiente según los recursos disponibles y contribuirán a la validación y justificación del objetivo principal planteado en este trabajo.

Metodología

El desarrollo del Trabajo de Fin de Máster se estructuró siguiendo los principios de la metodología ágil Scrum, adaptado a un contexto académico. Se organizaron los objetivos en sprints iterativos, cada uno centrado en una fase concreta del trabajo: exploración de herramientas, pruebas, análisis de resultados y documentación.

Los roles de Scrum se adaptaron de la siguiente manera:

- El **Product Owner** fue el propio autor del trabajo, en colaboración con el tutor, definiendo los objetivos y prioridades.
- El **Scrum Master** también fue asumido por el autor, velando por el cumplimiento del enfoque iterativo y la adaptación continua.
- El **Equipo de desarrollo** estuvo compuesto por el autor, encargado de la implementación, experimentación y análisis.

Cada sprint se centró en tareas específicas como la exploración de simuladores (Webots, Gazebo, Isaac Sim), ejecución de ejemplos con ROS 2 en los simuladores, recopilación de métricas de uso de recursos y análisis comparativo.

Además, la documentación se integró como parte del proceso de cada sprint: al finalizar cada fase, se documentaron los objetivos alcanzados, los resultados obtenidos y las conclusiones parciales. Esto permitió generar una base sólida de contenidos conforme avanzaba el desarrollo, y facilitó la elaboración de la memoria final.

Las reuniones con el tutor cumplieron la función de revisiones de sprint, permitiendo ajustar la planificación, validar avances y redefinir prioridades cuando fue necesario.

Este enfoque Scrum adaptado permitió mantener una evolución ordenada, flexible y orientada a la mejora continua, clave en un trabajo con fuerte componente exploratorio y experimental [93] [94].

Tecnologías utilizadas

Durante el desarrollo de este Trabajo Fin de Máster se ha hecho uso de una amplia variedad de tecnologías tanto de software como de hardware, esenciales para la simulación, el análisis de datos, el control de robots móviles y la documentación. A continuación, se resumen en una tabla, pudiendo encontrarlas detalladas en el anexo C (Tecnologías utilizadas).

Tabla 3: Resumen de tecnologías utilizadas

Tipo	Nombre	Versión	Referencia
Simulador	Ignition Gazebo (Fortress)	6.17.0	[78]
Simulador	Gazebo Classic	11.10.2	[85]
Simulador	Webots	R2025a	[28, 29]
Simulador	Isaac Sim	4.5.0	[58]
Robot	TurtleBot	3 y 4	[76, 77, 87]
Robot	Nova Carter	–	[95]
Desarrollo	Visual Studio Code	1.101.2	[55]
Desarrollo	Python	3.10.12	[80]
Desarrollo	ROS 2	Humble	[74]
Desarrollo	Nav2	1.1.18	[70]
Desarrollo	RViz	11.2.17	[75]
Desarrollo	JASP	0.19.3	[49]
Desarrollo	Git	2.34.1	[44, 105]
Desarrollo	GitHub	Online	[45, 106]
Documentación	Overleaf	Online	[79]
Documentación	LibreOffice Writer & Calc	7.3.7.2	[100]
Documentación	Google Drive	Online	[47]
Documentación	Gmail	Online	[46]
Documentación	Google Meet	Online	[48]
Documentación	Google Chrome	138.0.7204.92	[107]
Documentación	Discord	0.0.100	[32]
Documentación	Draw.io	Online	[51, 104]
Documentación	Boxy SVG	4.2.1	[37, 102]
Documentación	CompressPNG	Online	[97]
Monitorización	psutil	5.9.0	[43]
Monitorización	Powerstat	0.02.27	[4]

Estructura del trabajo

Este trabajo se estructura en varios capítulos y anexos que presentan de forma ordenada el desarrollo de la investigación, desde la definición del problema hasta la obtención de conclusiones y trabajos futuros:

- **Capítulo 1: Estudio del problema.** Se introduce el contexto y la motivación del trabajo, se revisa el estado del arte sobre simuladores robóticos y se definen los aspectos clave del problema a resolver. Incluye la descripción detallada de los simuladores Ignition Gazebo, Webots e Isaac Sim, así como su comportamiento al ejecutarse con/sin GUI, con/sin GPU y bajo distintas configuraciones de tiempo real.
- **Capítulo 2: Gestión de proyecto software.** Describe la metodología de trabajo adoptada (Scrum), el alcance del proyecto, la planificación y estimación de tareas, la gestión de recursos y presupuesto, así como la identificación de riesgos y la normativa aplicable.
- **Capítulo 3: Solución.** Presenta la propuesta de solución, abarcando el análisis, diseño e implementación de las herramientas y procedimientos utilizados para la evaluación de los simuladores. Incluye la descripción del producto desarrollado y sus funcionalidades.
- **Capítulo 4: Evaluación.** Expone el proceso experimental seguido para medir el rendimiento de los simuladores, primero de manera individual y después de forma comparativa, bajo diferentes configuraciones. Se analizan los resultados obtenidos y se proporciona un ejemplo práctico en Webots con ROS 2 y Nav2.
- **Conclusiones.** Resume los hallazgos principales, las contribuciones del trabajo y las líneas de trabajo futuro.

Finalmente, se incluyen varios **anexos** con información de apoyo, como el control de versiones, los datos experimentales completos, el seguimiento del proyecto, el inventario detallado de compatibilidad de Webots con ROS 2, los tutoriales elaborados y la descripción técnica de las tecnologías utilizadas.

Capítulo 1

Estudio del problema

En este capítulo, se establece el contexto del estudio explorando las tecnologías, plataformas, herramientas y trabajos previos relacionados que sirven como base para la investigación.

1.1. El contexto del problema

La robótica ha experimentado una gran evolución gracias a los avances en computación, inteligencia artificial y software de simulación. Debido a esto, el uso de simuladores ha cobrado una importancia creciente, permitiendo a investigadores y desarrolladores prototipar, probar y validar algoritmos de navegación, percepción y control sin necesidad de hardware físico. Esta práctica no solo reduce costes, sino que también facilita la detección temprana de errores y acelera el ciclo de desarrollo [52].

Uno de los simuladores más empleados en el ámbito académico e industrial es Gazebo, actualmente integrado en el ecosistema de ROS 2 a través de su versión más actualizada, Ignition Gazebo (recientemente renombrado como Gazebo Sim). Aunque Gazebo proporciona una simulación física realista y una integración sólida con ROS 2, ha sido reportado que su consumo de CPU y GPU es elevado, así como la complejidad en su configuración, representan una barrera significativa, especialmente en entornos con recursos limitados.

Este problema se intensifica cuando se utilizan robots móviles con navegación autónoma en entornos urbanos o domésticos virtuales complejos. Las acciones típicas de navegación, como la planificación de trayectorias, la evasión de obstáculos o la localización y mapeado simultáneo (SLAM), requieren una simulación fluida en tiempo real. Sin embargo, el factor de tiempo real (Real Time Factor, RTF) en

Gazebo puede degradarse significativamente cuando se activan múltiples sensores, se ejecutan nodos de ROS 2 complejos o se simulan varios robots a la vez [99].

Por esta razón, surge la necesidad de explorar simuladores alternativos a Gazebo que, sin sacrificar la compatibilidad con ROS 2 ni la fidelidad física, ofrezcan un menor consumo de recursos y mejor rendimiento.

Este trabajo se enmarca dentro de esa necesidad de evaluación y comparación rigurosa, con el objetivo de identificar si existen simuladores que puedan ofrecer un rendimiento más eficiente en tareas robóticas típicas en ROS 2, manteniendo al mismo tiempo la precisión y realismo requeridos para experimentos científicos reproducibles.

1.2. El estado de la cuestión

Una vez visto el contexto del problema, se expondrá ahora el estado de los simuladores en robótica. Primero se expondrán los principales que actualmente existen en el mundo de la robótica y, posteriormente, de estos simuladores cuales podrían ser posibles sustitutos a Gazebo.

1.2.1. Principales simuladores de robótica

En robótica, los simuladores son fundamentales ya que permiten el desarrollo, prueba y validación de distintos comportamientos en un robot sin necesidad de disponer de hardware físico. Estos tienen como objetivo replicar el comportamiento del robot y su interacción con el entorno, cada uno de ellos con distintos niveles de realismo o rendimiento. A continuación, se presentan algunos de los simuladores más populares:

RoboDK

RoboDK es un simulador de robótica industrial que permite programar y validar trayectorias para robots de múltiples fabricantes sin necesidad de disponer del hardware físico. Fue lanzado en 2015 como un proyecto derivado del laboratorio CoRo de la École de technologie supérieure (ETS) de Montreal, se ha convertido en una herramienta ampliamente utilizada en más de 50 países, tanto en entornos industriales como académicos [84].

Entre sus principales ventajas destacan su compatibilidad con más de 1.000 robots de más de 50 marcas distintas, la posibilidad de programar mediante una interfaz gráfica intuitiva, y la integración de scripts en Python para personalizar tareas com-

plejas. Asimismo, RoboDK facilita la integración con herramientas CAD/CAM, lo que favorece su adopción en procesos como el mecanizado, la soldadura robotizada, la impresión 3D o el control offline de robots [84].

En 2025, con motivo de su décimo aniversario, la compañía reafirmó su compromiso con el desarrollo de soluciones accesibles y abiertas para reducir la barrera de entrada a la robótica avanzada [36].

Está centrado en interiores y ofrece una versión de prueba gratuita con ciertas restricciones, así como licencias de pago para uso educativo y profesional [83]. No dispone de compatibilidad directa con ROS o ROS 2, lo que limita su integración en entornos de desarrollo robótico.

Cyberbotics Webots

Webots es un simulador de código abierto desarrollado por la empresa Cyberbotics, orientado principalmente a la simulación de robots móviles, manipuladores y entornos complejos en 3D [28]. Se trata de un simulador ampliamente utilizado tanto en entornos académicos como industriales ya que ofrece una gran facilidad de uso y soporte para múltiples plataformas.

Una de sus principales ventajas es su compatibilidad directa con ROS y ROS 2, incluyendo una integración oficial mantenida activamente por la propia comunidad y por Cyberbotics [26, 90]. Esta integración permite utilizar controladores ROS 2, sensores y nodos directamente sobre los robots simulados, facilitando el desarrollo y prueba de robots en entornos realistas antes de ser desplegados en hardware real. Además, Webots proporciona una gran variedad de robots preconfigurados, así como la posibilidad de diseñar robots personalizados mediante su sistema de nodos y estructuras jerárquicas [25].

Webots permite realizar simulaciones físicas avanzadas con soporte para sensores complejos como cámaras, LIDAR, IMU y motores articulados. Su naturaleza open-source permite modificar tanto el simulador como los controladores, lo que resulta especialmente útil para tareas de investigación, desarrollo e integración con otros frameworks robóticos [25, 28].

CoppeliaSim

CoppeliaSim, anteriormente conocido como V-REP, es un simulador de robótica desarrollado por Coppelia Robotics que destaca por su flexibilidad, modularidad y capacidad de simulación multi-robot y multiproceso [6]. Es una herramienta muy utilizada en investigación y educación, ya que permite crear entornos de simulación

complejos con múltiples robots, sensores, mecanismos y sistemas de control, todo desde una interfaz gráfica intuitiva.

Una de las características más importantes de CoppeliaSim es su arquitectura basada en scripts embebidos, que permite controlar el comportamiento de los robots mediante su propio lenguaje o mediante integración con lenguajes externos como Python, Lua, C/C++, MATLAB o Java. Además, ofrece conectividad con múltiples entornos de desarrollo a través de interfaces remotas y plugins [54].

CoppeliaSim ofrece compatibilidad con ROS y ROS 2 a través de los paquetes simROS y simROS2, disponibles como open-source. Estas interfaces permiten la integración directa de nodos ROS y ROS 2, suscripción y publicación de tópicos, servicios y acciones dentro del entorno simulado [7–9].

Está centrado en interiores y tiene 3 opciones de descarga, lite, pro y educativa [6].

Gazebo

Gazebo es uno de los simuladores de robótica más conocidos y utilizados en investigación, desarrollo y educación. Su arquitectura modular permite simular entornos tridimensionales complejos con gran realismo físico, incluyendo dinámicas de cuerpos rígidos, fricción, colisiones, sensores y actuadores avanzados [39]. Inicialmente desarrollado por Willow Garage y mantenido actualmente por Open Robotics y la comunidad, Gazebo ha sido la plataforma de simulación por excelencia en combinación con ROS.

Con la evolución del sistema, la versión clásica (ahora denominada Gazebo Classic) ha dado paso a una nueva versión, el proyecto Ignition Gazebo, actualmente renombrado simplemente como Gazebo a partir de su sexta versión [40, 41]. Esta nueva arquitectura ofrece mejoras sustanciales en modularidad, rendimiento gráfico, soporte para simulaciones distribuidas y herramientas de desarrollo modernas.

Gazebo se integra de forma nativa con ROS y ROS 2, lo que permite simular robots y entornos directamente vinculados con nodos, tópicos, servicios y acciones ROS [42, 103]. Esta compatibilidad es ampliamente utilizada para probar robots antes de su implementación en el mundo real.

Gazebo además permite la simulación de una gran variedad de sensores como cámaras, LIDAR, IMU o GPS entre otros y es compatible con múltiples motores de renderizado. También es open-source, lo que permite su personalización, extensión y uso gratuito en cualquier proyecto [91].

Isaac Sim

Isaac Sim es un simulador robótico desarrollado por NVIDIA como parte de su plataforma Omniverse, orientado principalmente al desarrollo de inteligencia artificial, robótica avanzada y gemelos digitales [60]. Ofrece simulaciones realistas y físicamente precisas gracias al motor NVIDIA PhysX y la tecnología RTX, lo que permite simular entornos complejos, sensores de alta fidelidad y comportamiento dinámico en tiempo real.

Uno de sus aspectos más potentes es su estrecha integración con flujos de trabajo de entrenamiento y validación de modelos de IA, tanto para visión por computador como para planificación y control. Isaac Sim proporciona herramientas para generar datasets sintéticos, probar redes neuronales y validar arquitecturas de control de robots de manera virtual antes de desplegarlas en hardware real [58, 60].

Isaac Sim cuenta con soporte oficial para ROS 2, incluyendo integración con nodos, tópicos, servicios y acciones a través de un conjunto de herramientas desarrollado por NVIDIA llamado Isaac ROS. Esta integración facilita la interoperabilidad entre el simulador y entornos de desarrollo robótico estándar [56, 68]. El ecosistema se complementa con ejemplos y workspaces disponibles públicamente para facilitar la configuración y uso del entorno [61].

Su uso está especialmente enfocado a aplicaciones industriales, investigación avanzada y entrenamiento de modelos de aprendizaje automático en escenarios complejos. Gracias a sus capacidades gráficas y físicas avanzadas, Isaac Sim se ha posicionado como uno de los simuladores más innovadores en el ámbito de la robótica con IA.

1.2.2. Posibles simuladores sustituyentes de Gazebo

Vistos los principales simuladores actualmente en robótica, se presentan 2 alternativas a Gazebo en una tabla resumen:

Tabla 1.1: Alternativas a Gazebo

Aspecto	Webots	Isaac Sim
Compatibilidad	Compatible con ROS 2 y Nav2	Compatible con ROS 2 y Nav2
Robots	Robots similares a los disponibles en Gazebo	Soporte para múltiples robots avanzados
Entornos y sensores	Mundos exteriores, interiores y sensores disponibles	Entornos realistas con físicas avanzadas y soporte para sensores
Licencia y comunidad	Gratuito y con comunidad amplia	Gratuito (uso personal y educativo), pero requiere GPU NVIDIA. Comunidad y soporte activo

1.3. La definición del problema: El simulador como herramienta software

Vistas las 2 opciones de simuladores que pueden sustituir a Gazebo, a continuación se realizará una explicación detallada de cada una de ellas además de una de Gazebo. Para estas explicaciones inicialmente se realizó una revisión de la literatura y web, capturando los pros y contras que han presentado los desarrolladores y usuarios a lo largo de los últimos años.

1.3.1. Gazebo

Gazebo es uno de los simuladores de robótica más utilizados en la comunidad ROS, desarrollado inicialmente en 2002 por Open Robotics y actualmente mantenido a través de Open Source Robotics Foundation (OSRF) [103]. Desde sus inicios, ha sido un pilar en la simulación robótica, especialmente en entornos que requieren alta precisión física y una integración estrecha con ROS 2 [39, 41, 91].

Soporta múltiples motores físicos como ODE, Bullet y DART, e incluye una amplia gama de sensores (LIDAR, cámaras RGB y de profundidad, GPS, IMU) y actuadores complejos, lo que lo hace especialmente adecuado tanto para interiores como exteriores [103]. La transición reciente de Gazebo Classic hacia Ignition Gazebo, ahora denominado oficialmente como Gazebo Sim, introduce una arquitectura modular basada en componentes como gz-sim, gz-rendering y gz-physics [39–41].

Pros

- Física precisa y realismo en colisiones: Permite simulaciones detalladas de entornos irregulares, ideal para pruebas con movilidad todo-terreno [103].
- Integración nativa con ROS 2: A través de plugins específicos, facilita el desarrollo de pipelines simulación-realidad, clave en proyectos ROS [42, 91].
- Amplia comunidad y soporte académico: Es el estándar de facto en investigación y enseñanza en robótica, con respaldo activo de universidades y laboratorios [41, 103].

Contras

- Curva de aprendizaje pronunciada: La configuración avanzada (plugins, modelos URDF/SDF, motores físicos) puede ser compleja para usuarios no expertos [91].
- Interfaz menos moderna: A pesar de mejoras en la versión Gazebo Sim, su interfaz es menos intuitiva que en simuladores comerciales o motores de juegos [91, 103].
- Fin de soporte para Gazebo Classic: Desde enero de 2025, Gazebo Classic ha sido oficialmente discontinuado, lo que requiere migración a la nueva arquitectura [41, 103].

1.3.2. Webots

Webots es un simulador de código abierto bajo licencia Apache 2.0, desarrollado por Cyberbotics desde 1998 [25, 28]. Está diseñado para ofrecer un entorno visual amigable y completo para la simulación de robots móviles, manipuladores y entornos complejos. A diferencia de Gazebo, Webots tiene un enfoque centrado en la usabilidad desde su interfaz gráfica, permitiendo crear escenas, importar modelos CAD y mapas de OpenStreetMap sin necesidad de scripting complejo [28, 90, 96].

La integración con ROS 2 ha mejorado considerablemente mediante el paquete `webots_ros2`, permitiendo simular nodos ROS de forma realista [26, 90].

Pros

- Ligero en recursos: Utiliza menos CPU y memoria que Gazebo, permitiendo correr simulaciones en ordenadores de gama media [81, 90].

- Biblioteca extensa y soporte para exteriores: Incluye modelos de coches, drones, tractores, y simula entornos realistas con terreno y clima [28,90].
- Desarrollo activo y compatibilidad con ROS 2: Si bien su comunidad es menor, Cyberbotics mantiene actualizaciones frecuentes y soporte ROS oficial [26].

Contras

- Documentación técnica limitada: Aunque ofrece buenos tutoriales para principiantes, la documentación para temas avanzados o toolchains externas es escasa [81].
- Menor precisión física: Las simulaciones de colisiones y dinámica son más simples que en Gazebo, limitando su uso en proyectos físicos exigentes [81,82].
- Comunidad más reducida: Menor presencia en foros especializados en ROS, lo que puede dificultar la resolución de problemas complejos [90,92].

1.3.3. NVIDIA Isaac Sim

Isaac Sim es el simulador de NVIDIA basado en la plataforma Omniverse, diseñado para tareas de robótica y visión artificial en entornos físicamente realistas [58,60]. Su arquitectura está enfocada en aprovechar al máximo la aceleración por GPU (RT Cores y Tensor Cores), permitiendo simulaciones masivas y generación de datos sintéticos para entrenamiento de IA [58,68].

Utiliza el lenguaje de escena USD (Universal Scene Description) y cuenta con integración parcial con ROS 2 mediante adaptadores específicos [56,61]. Está orientado principalmente a simulaciones avanzadas para aprendizaje por refuerzo, percepción y manipulación robótica compleja.

Pros

- Renderizado fotorrealista: Gracias a RTX y Omniverse, ofrece iluminación y materiales realistas, útil para proyectos de visión por computador [58,60].
- Física acelerada por GPU: Aumenta el rendimiento en simulaciones densas, permitiendo entrenamientos paralelos en entornos de IA (Isaac Gym, Isaac Cortex) [58,68].
- Generación de datos sintéticos: Puede generar grandes volúmenes de imágenes anotadas para redes neuronales, integrándose con frameworks como PyTorch y Isaac ROS [58,60].

Contras

- Requisitos muy altos de hardware: Necesita GPUs con RT Cores y al menos 32 GB de RAM. Para escenarios complejos se recomienda una GPU con 48 GB de VRAM [58].
- Integración ROS menos directa: Aunque es posible simular nodos ROS 2, su configuración requiere conocimientos avanzados de Omniverse y USD [56,61].
- Dependencia de software propietario: Parte del stack es cerrado, lo que dificulta modificaciones profundas o auditoría completa del código [58,60].

1.4. La definición del problema: Coste computacional asociado a la simulación

Además de la explicación detallada anterior, en esta sección se expondrá lo relacionado con el coste computacional de la simulación incluyendo el coste computacional de cada simulador según los desarrolladores y usuarios a lo largo de los últimos años, las opciones de lanzamiento con o sin GPU, con o sin GUI y de tiempo real de cada simulador.

1.4.1. Coste computacional por simulador

Se expondrá un breve resumen por cada simulador respecto a su coste computacional según los desarrolladores y usuarios a lo largo de los últimos años.

Coste computacional: Gazebo

Gazebo tiende a ser el simulador más exigente en cuanto a CPU, especialmente en escenas grandes o con múltiples sensores activos. Un estudio comparativo mostró que supera en uso de CPU a simuladores como Webots o V-REP, manteniendo un consumo intermedio-alto de RAM [2,91,103].

Coste computacional: Webots

Webots es uno de los simuladores más eficientes: sus requerimientos mínimos (CPU dual-core de 2 GHz y 2 GB de RAM) permiten ejecutar simulaciones en equipos básicos. En pruebas comparativas, fue el simulador con menor uso de CPU y memoria [1,28,90].

Coste computacional: Isaac Sim

Es el simulador más exigente en términos de recursos. A pesar de correr con Omniverse Cloud, para simulaciones locales requiere GPUs profesionales de gama alta (RTX A6000 o similar). El uso masivo de GPU permite entrenamiento paralelo pero a un coste de consumo considerable [58, 69].

1.4.2. Lanzar con/sin GUI

En este apartado se comprobará si los 3 simuladores tienen la opción de lanzarse con o sin interfaz gráfica. Lanzar un simulador con GUI permite visualizar en tiempo real el comportamiento del robot, lo cual es útil para depuración, desarrollo y demostraciones, pero implica un mayor consumo de recursos, especialmente GPU y CPU, lo que puede afectar negativamente al rendimiento de la simulación. En cambio, ejecutarlo sin GUI elimina la carga del renderizado gráfico, permitiendo una ejecución más eficiente y rápida, ideal para experimentos automatizados, benchmarking y entornos sin interfaz gráfica. Por tanto, usar GUI es apropiado en fases de desarrollo o validación visual, mientras que prescindir de ella es preferible para obtener mediciones objetivas y reproducibles.

Ignition Gazebo

En el caso de Gazebo nos da la opción de lanzar solo el servidor de simulación sin GUI haciendo uso de la opción `-s` o `-server-only`. Podemos ver como el comando sería algo así:

```
$ ign gazebo -s empty.sdf
```

Para comprobar que está funcionando correctamente sin GUI podemos comprobar que el proceso esta corriendo con:

```
$ ps aux | grep ign
```

O también consultar los topics activos de Gazebo con:

```
$ ign topic -l
```

Podemos ver esto en la siguiente imagen:

```
jorge_casatejada@jorgeCasatejada:~$ ign gazebo -s empty.sdf
jorge_casatejada@jorgeCasatejada:~$ ign topic -l
/clock
/gazebo/resource_paths
/stats
/world/empty/clock
/world/empty/dynamic_pose/info
/world/empty/pose/info
/world/empty/scene/deletion
/world/empty/scene/info
/world/empty/state
/world/empty/stats
jorge_casatejada@jorgeCasatejada:~$
jorge_casatejada@jorgeCasatejada:~$ ps aux | grep ign
root      945  0.0  0.1 120816 22940 ?        Ssl  10:15   0:00 /usr/bin/python3 /usr/share
/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
jorge_c+  36515 11.0  0.6 994304 104840 pts/5    Sl+  11:35   0:02 ign gazebo -s empty.sdf
jorge_c+  36720  0.0  0.0 11852 2560 pts/7     S+   11:36   0:00 grep --color=auto ign
```

Figura 1.1: Ejecución de Ignition Gazebo sin GUI

Webots

Webots también ofrece la posibilidad de lanzar el simulador sin GUI de 2 maneras, la primera de ellas haciendo uso de los argumentos de la línea de comandos especificados en su documentación [15] y la segunda, en caso de hacer uso de ROS 2, con su nodo WebotsLauncher en el cual uno de sus argumentos nos permite desactivar la GUI como se expone aquí. Este argumento del nodo WebotsLauncher hace uso de los siguientes argumentos de la línea de comandos:

- `-no-rendering`
- `-stdout`
- `-stderr`
- `-minimize`

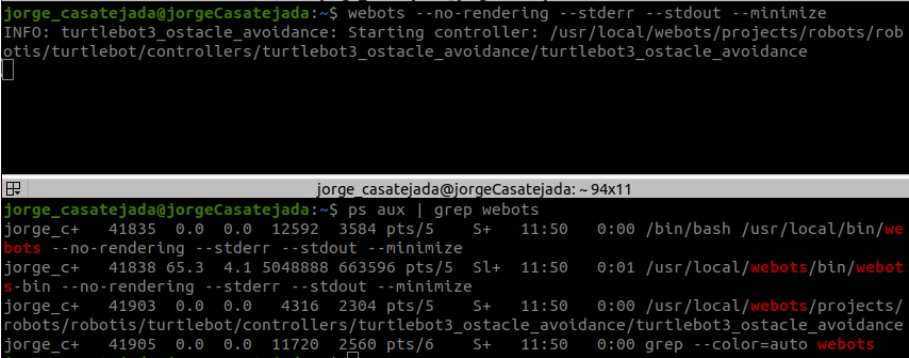
Es decir, podemos lanzar el simulador Webots sin GUI de la siguiente manera:

```
$ webots --no-rendering --stderr --stdout --minimize
```

Para comprobar que está funcionando correctamente podemos observar los logs por consola además de comprobar si su proceso está corriendo con:

```
$ ps aux | grep webots
```

Podemos ver todo esto en la siguiente imagen:



```
jorge_casatejada@jorgeCasatejada:~$ webots --no-rendering --stderr --stdout --minimize
INFO: turtlebot3_ostacle_avoidance: Starting controller: /usr/local/webots/projects/robots/robotis/turtlebot/controllers/turtlebot3_ostacle_avoidance/turtlebot3_ostacle_avoidance

jorge_casatejada@jorgeCasatejada:~$ ps aux | grep webots
jorge_c+ 41835 0.0 0.0 12592 3584 pts/5 S+ 11:50 0:00 /bin/bash /usr/local/bin/webots --no-rendering --stderr --stdout --minimize
jorge_c+ 41838 65.3 4.1 5048888 663596 pts/5 Sl+ 11:50 0:01 /usr/local/webots/bin/webots --no-rendering --stderr --stdout --minimize
jorge_c+ 41903 0.0 0.0 4316 2304 pts/5 S+ 11:50 0:00 /usr/local/webots/projects/robots/robotis/turtlebot/controllers/turtlebot3_ostacle_avoidance/turtlebot3_ostacle_avoidance
jorge_c+ 41905 0.0 0.0 11720 2560 pts/6 S+ 11:50 0:00 grep --color=auto webots
```

Figura 1.2: Ejecución de Webots sin GUI

Isaac Sim

Isaac Sim ofrece la alternativa de lanzar el simulador sin GUI local a través de su opción de lanzamiento streaming la cual lanza Isaac Sim completamente sin GUI local (modo headless + streaming remoto) y:

- No abre ninguna ventana localmente.
- Permite conectarse remotamente con el cliente de NVIDIA Omniverse (streaming).
- Es ideal para servidores o estaciones sin entorno gráfico.

Es decir, podemos lanzar esta alternativa con el comando:

```
$ ./isaac-sim.streaming.sh
```

Y comprobar que el proceso del simulador está corriendo con:

```
$ ps aux | grep isaac
```

O ver el consumo de la gráfica Nvidia con:

```
$ nvidia-smi
```

Podemos ver todo esto en la siguiente imagen:

```
jorge_casatejada@jorgeCasatejada: ~
jorge_casatejada@jorgeCasatejada: ~/isaacsim 94x19
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : TLAS limit buff
er size 7508933632
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : TLAS limit : va
lid false, within: false
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : TLAS limit : de
crement: 167690, decrement size: 7433845248
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : New limit 95742
51 (slope: 447, intercept: 13179904)
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : TLAS limit buff
er size 4287216384
2025-06-28 10:33:03 [12,367ms] [Warning] [rtx.scenedb.plugin] SceneDbContext : TLAS limit : va
lid true, within: true
[12.406s] app ready
2025-06-28 10:33:03 [12,454ms] [Warning] [omni.usd-abi.plugin] No setting was found for '/rtx-
defaults-transient/meshlights/forceDisable'
2025-06-28 10:33:03 [12,489ms] [Warning] [omni.usd-abi.plugin] No setting was found for '/rtx-
defaults/post/dlss/execMode'
[14.968s] Isaac Sim Full Streaming App is loaded.

jorge_casatejada@jorgeCasatejada: ~ 94x5
jorge_casatejada@jorgeCasatejada:~$ ps aux | grep isaac
jorge_c+ 57545 183 28.1 90068968 4516736 pts/5 Sl+ 12:32 2:22 ./kit/kit ./apps/isaacsim.e
xp.full.streaming.kit --ext-folder ./apps --no-window
jorge_c+ 58346 0.0 0.0 11720 2560 pts/6 S+ 12:34 0:00 grep --color=auto isaac
jorge_casatejada@jorgeCasatejada:~$

jorge_casatejada@jorgeCasatejada: ~ 94x23
jorge_casatejada@jorgeCasatejada:~$ nvidia-smi
Sat Jun 28 12:34:06 2025
+-----+
| NVIDIA-SMI 570.153.02      Driver Version: 570.153.02      CUDA Version: 12.8      |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id  Disp.A | Volatile Uncorr. ECC | | |
| Fan  Temp  Perf    Pwr:Usage/Cap       |           |         Memory-Usage | GPU-Util  Compute M. |
|              |              |   /     /              |           |         MIG M.       |
+-----+-----+-----+-----+-----+-----+
| 0     NVIDIA GeForce RTX 4050 ...      Off      | 00000000:01:00:0 Off |           |       N/A           | |
| N/A   48C    P4      9W / 60W         | 1589MiB / 6141MiB |           | 39%      Default   |
|              |              |           |           |         N/A         |
+-----+-----+-----+-----+-----+-----+

Processes:
+-----+-----+-----+-----+-----+-----+
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
| ID   ID   ID           |          |           | Usage                     |
+-----+-----+-----+-----+-----+-----+
| 0     N/A  N/A        1522     G     /usr/lib/xorg/Xorg                22MiB   |
| 0     N/A  N/A        57545    C+G    ./kit/kit                        1507MiB |
+-----+-----+-----+-----+-----+-----+
jorge_casatejada@jorgeCasatejada:~$
```

Figura 1.3: Ejecución de Isaac Sim sin GUI

1.4.3. Lanzar con/sin GPU

En este apartado se analizará si los tres simuladores empleados permiten su ejecución con o sin el uso explícito de la GPU NVIDIA. El uso de GPU dedicada, como las de NVIDIA, permite acelerar el renderizado gráfico y mejorar el rendimiento visual del simulador, lo cual es especialmente beneficioso en entornos complejos o con modelos detallados.

Por otro lado, ejecutar los simuladores sin utilizar la GPU NVIDIA permite mantener una mayor portabilidad y eficiencia en recursos, facilitando la ejecución en entornos con hardware limitado.

En mi sistema, el renderizador especificado se encuentra en modo *on-demand*, es decir, el sistema utiliza por defecto la GPU integrada (Intel) para la mayoría

de tareas, y solo emplea la GPU NVIDIA cuando se le indica explícitamente. Esta configuración puede comprobarse mediante el siguiente comando:

```
$ prime-select query
```

Gazebo Classic, Ignition Gazebo y Webots

Con el renderizado en modo *on-demand*, estos simuladores no hacen uso de la GPU NVIDIA, lo que indica que puede funcionar sin esta, como se observa en las siguientes imágenes:

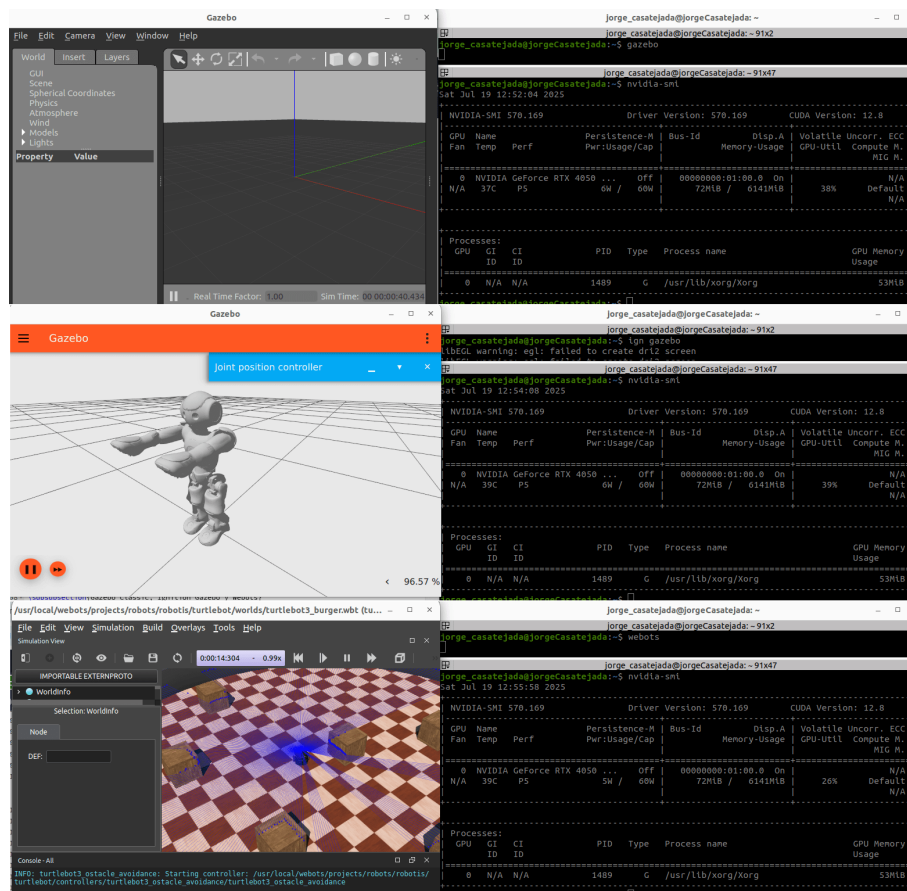


Figura 1.4: Simuladores sin GPU

Para forzar el uso de la GPU NVIDIA y mejorar el rendimiento gráfico, se pueden utilizar las siguientes variables de entorno:

```
$ __NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia gazebo
$ __NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia ign gazebo
$ __NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia webots
```

El uso de GPU se evidencia en:

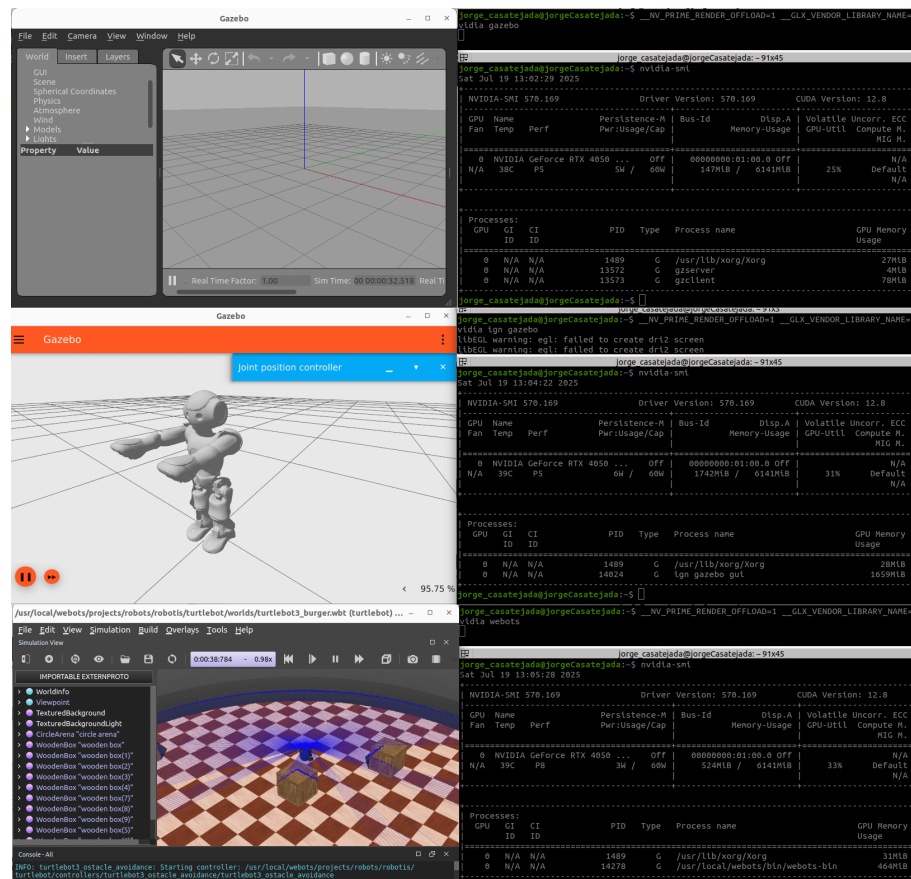


Figura 1.5: Simuladores con GPU

Isaac Sim

Isaac Sim, a diferencia de los anteriores, sí requiere el uso obligatorio de la GPU NVIDIA para su ejecución, además como mínimo se necesita una GPU NVIDIA GeForce RTX 4080 con al menos 16 GB de VRAM como se indica en su página oficial.

Cambio de Renderizador

En caso de que el sistema tenga configurado como predeterminado el uso de la GPU NVIDIA (modo *nvidia*), también es posible deshabilitarla temporalmente para la ejecución de los simuladores mediante variables de entorno:

```
$ LIBGL_ALWAYS_SOFTWARE=1 gazebo
$ LIBGL_ALWAYS_SOFTWARE=1 ign gazebo
$ LIBGL_ALWAYS_SOFTWARE=1 WEBOTS_FORCE_SOFTWARE_RENDERING=1 webots
```

1.4.4. Tiempo Real

El tiempo real aplicado a simulación se refiere a la relación entre el tiempo simulado y el tiempo de reloj del mundo real. Esta relación se mide comúnmente mediante el Real Time Factor (RTF), que indica cuántos segundos de simulación transcurren por cada segundo real. Un RTF igual a 1.0 implica que la simulación se ejecuta en tiempo real, mientras que un valor inferior indica una simulación más lenta que el mundo real.

La ejecución en tiempo real depende de múltiples factores como la complejidad del entorno, los modelos físicos, la frecuencia de actualización de sensores, el número de agentes, y si se está usando GUI o modo headless.

En este apartado se comprobará las opciones disponibles respecto a tiempo real en los 3 simuladores.

Gazebo Classic

Gazebo Classic [71] permite modificar la velocidad de simulación en relación con el tiempo real mediante parámetros como `real_time_update_rate` y `max_step_size`. Esto hace posible ajustar el Real Time Factor (RTF), es decir, la relación entre tiempo simulado y tiempo real. Por ejemplo, al ejecutar:

```
$ gz physics -u 2000
```

La simulación se ejecutará al doble de velocidad del tiempo real ($RTF \approx 2$), como vemos aquí:

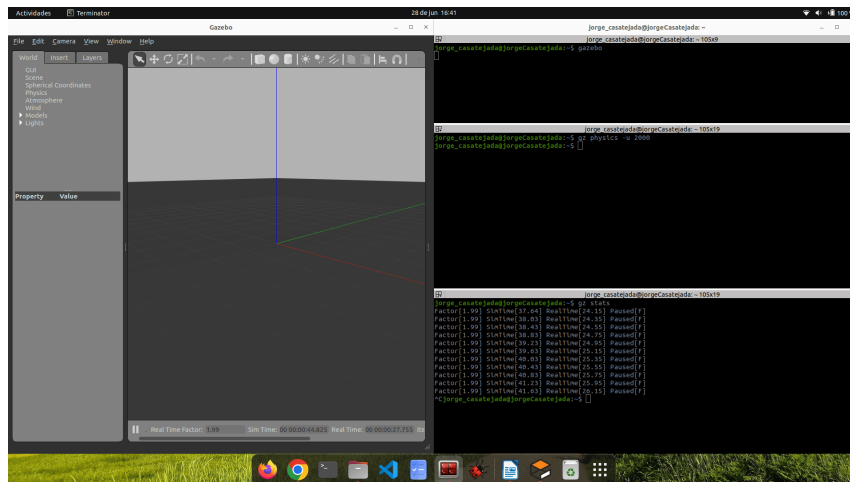


Figura 1.6: Gazebo Classic con $RTF \approx 2$

Mientras que si ejecutamos:

```
$ gz physics -u 500
```

La velocidad de simulación se reducirá a la mitad del tiempo real ($RTF \approx 0.5$):

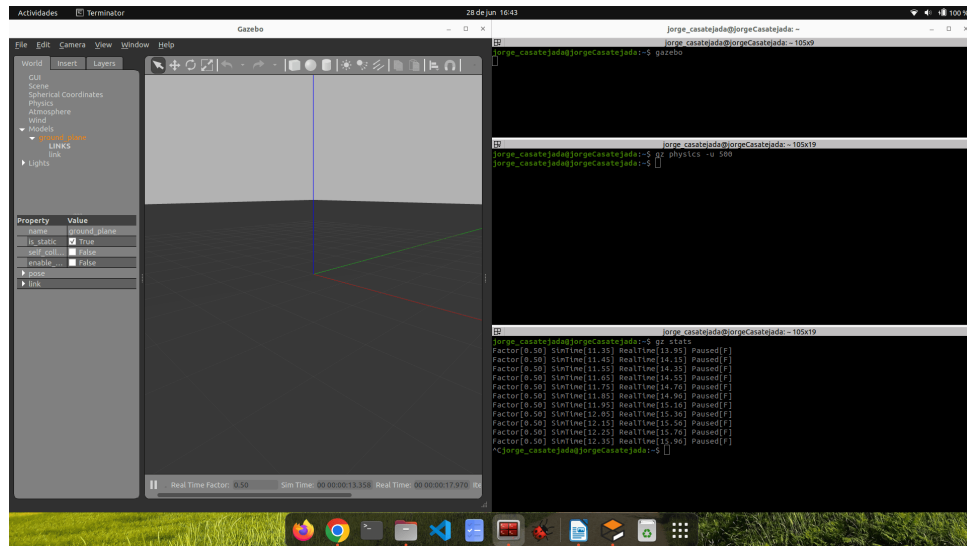


Figura 1.7: Gazebo Classic con $RTF \approx 0.5$

En ambos casos siempre será en el caso de que el hardware lo permita. También se pueden cambiar el parámetro `real_time_factor` en los archivo del mundo (.world) a los que queramos modificar el RTF o directamente en el simulador, como se muestran en la siguiente imagen:

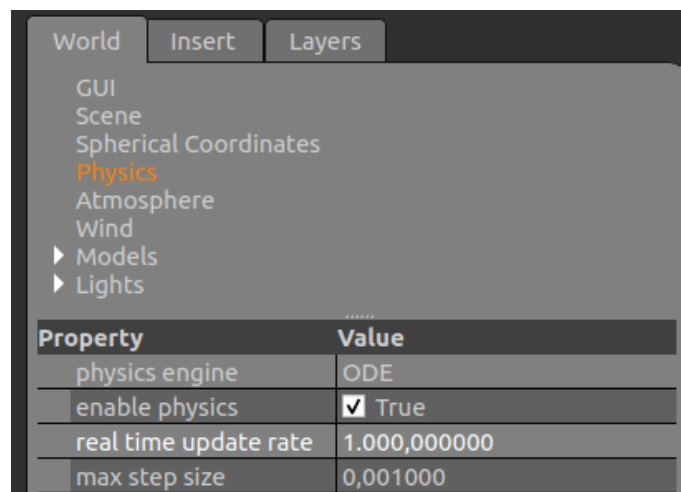


Figura 1.8: Ajuste de RTF en Gazebo Classic

Añadir que también se puede ir paso a paso, es decir, simular cada step de la simulación sin tener corriendo la simulación constantemente.

Ignition Gazebo

En Ignition Gazebo [72,73] también se permite modificar la velocidad de simulación en relación con el tiempo real mediante la configuración de parámetros físicos como `real_time_update_rate` y `max_step_size`. Al igual que en Gazebo Classic, esto permite ajustar el Real Time Factor (RTF), es decir, la relación entre el tiempo simulado y el tiempo real.

En Ignition, los parámetros pueden cambiarse en tiempo de ejecución mediante servicios. Por ejemplo, al ejecutar:

```
$ ign service -s /world/world_demo/set_physics \
--reqtype ignition.msgs.Physics \
--reptype ignition.msgs.Boolean \
--req 'real_time_factor: 2' \
--timeout 2000
```

La simulación intentará ejecutarse al doble de velocidad del tiempo real ($RTF \approx 2$) como se ve aquí:

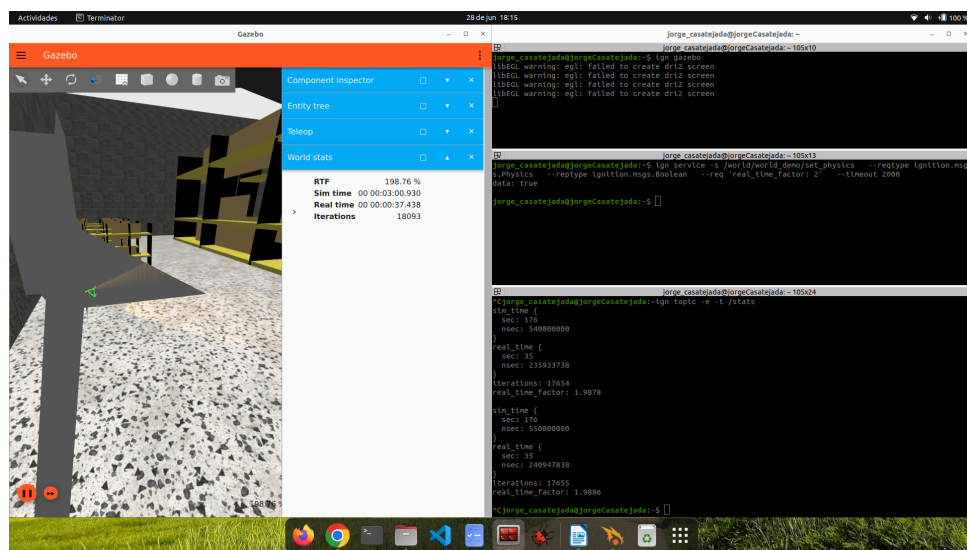


Figura 1.9: Ignition Gazebo con $RTF \approx 2$

Mientras que si disminuimos el `real_time_factor` a 500:

```
$ ign service -s /world/world_demo/set_physics \
--reqtype ignition.msgs.Physics \
--reptype ignition.msgs.Boolean \
```



```
--req 'real_time_factor: 0,5' \
--timeout 2000
```

La velocidad de simulación se reducirá a la mitad del tiempo real ($RTF \approx 0.5$):

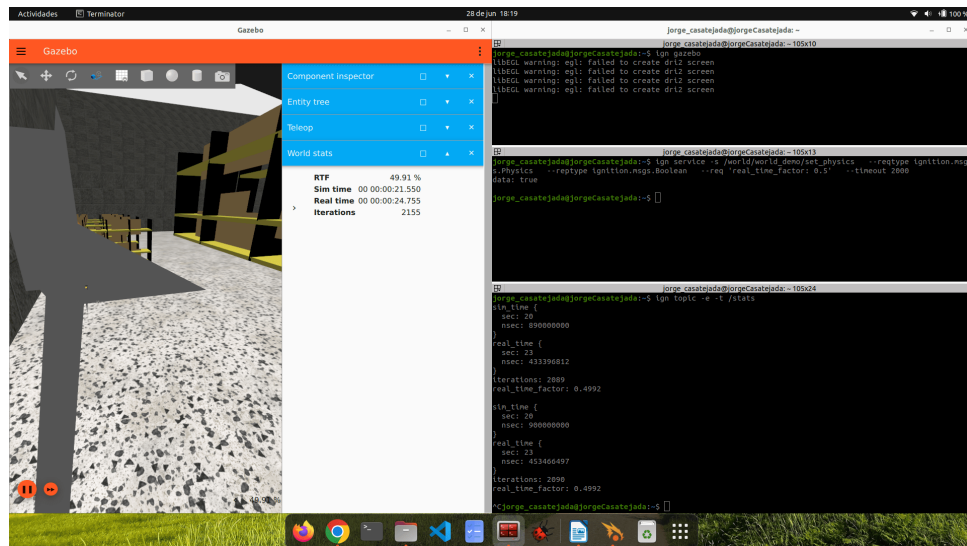


Figura 1.10: Ignition Gazebo con $RTF \approx 0.5$

Al igual que antes, siempre será en el caso de que el hardware lo permita. También se pueden cambiar el parámetro `real_time_factor` en los archivo del mundo (.sdf) a los que queramos modificar el RTF o directamente en el simulador, como se muestran en la siguiente imagen:

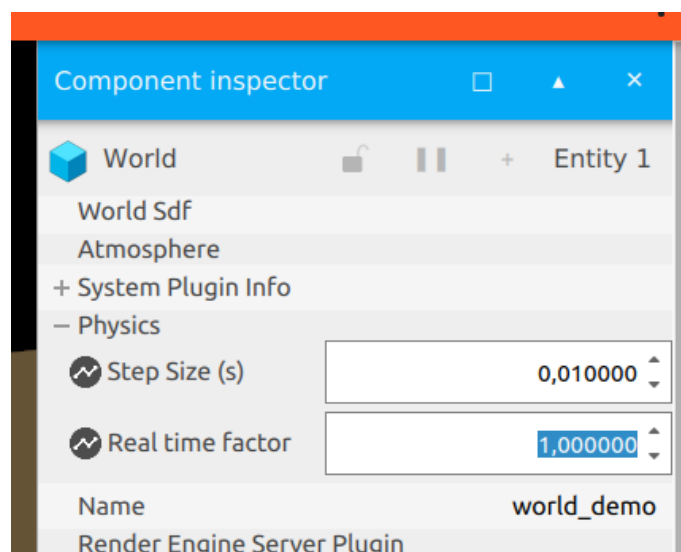


Figura 1.11: Ajuste de RTF en Ignition Gazebo

Añadir que, al igual que en Gazebo Classic, también se puede ir paso a paso, es decir, simular cada step de la simulación sin tener corriendo la simulación constantemente.

Webots

En Webots no se puede especificar un valor de `real_time_factor` explícitamente pero nos permite mediante el argumento `-mode` especificar entre 3 opciones [15]:

- **pause**: comienza con la simulación parada.
- **realtime**: ejecuta la simulación intentando mantener $RTF = 1$ (por defecto).
- **fast**: Ejecuta la simulación tan rápido como el hardware lo permita

Al estar en pausa nos permite simular paso a paso la simulación al igual que con Gazebo.

Podemos ver como afecta el modo **realtime** en la siguiente imagen:

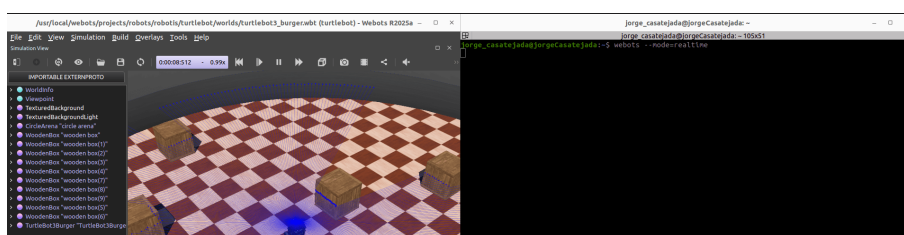


Figura 1.12: Webots en modo realtime ($RTF \approx 1$)

Se ve como en el simulador, en el recuadro morado, el factor de tiempo real es de prácticamente 1. Podemos ver el comportamiento del modo **fast** en la siguiente imagen:

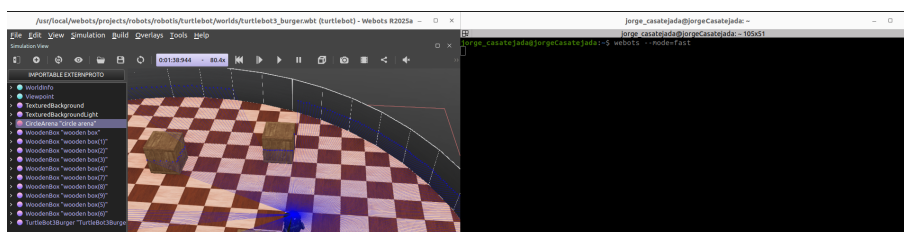


Figura 1.13: Webots en modo fast ($RTF \approx 80$)

Vemos como llega aproximadamente a un factor de tiempo real de 80.

Otra opción [24] es modificando el campo `basicTimeStep` de `WorldInfo` del fichero del mundo que vayamos a lanzar (.wbt) podemos hacer que el paso físico sea

más pequeño y se ralentice o acelera la simulación. Por ejemplo cambiando el valor a 0.5:

```
WorldInfo {  
    basicTimeStep 0.5  
}
```

Figura 1.14: Configuración de basicTimeStep en Webots

Podemos ver como el factor de tiempo real disminuye:

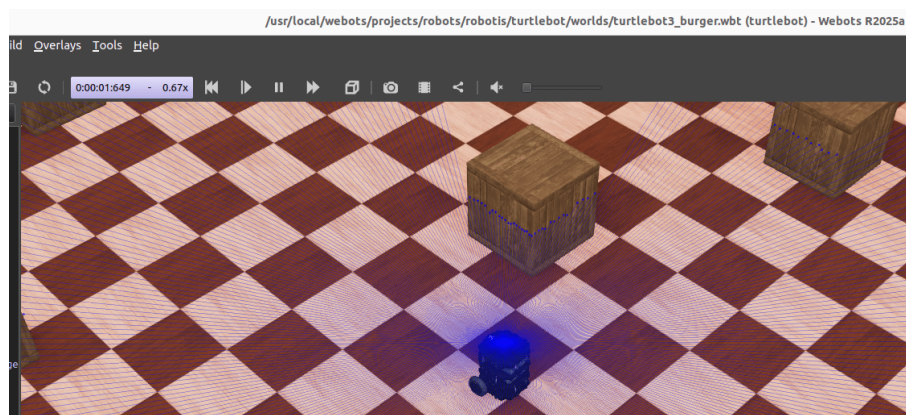


Figura 1.15: Webots con RTF reducido

El claro inconveniente de Webots es no poder fijar este factor de manera explícita.

Isaac Sim

En Isaac Sim las opciones [59] que nos da son:

- RTX – Real-Time mode
 - Modo de renderizado en tiempo real de alta calidad.
- RTX – Interactive (Path Tracing) mode
 - El modo de renderizado de mayor calidad y precisión física.

Por defecto el Real-Time mode es el que se lanza con el simulador el cual intentará siempre que haya un real time factor de 1. El valor de este factor se puede obtener a partir de un nodo que ofrece Isaac Sim, el nodo Isaac Real Time Factor [57] el cual simplemente calcula el factor de tiempo real y lo devuelve. También está disponible para Isaac Sim ROS 2 para poder publicarlo en un topic fácilmente [62].

Para poder disminuir el real time factor lo que podemos es aumentar el parámetro **Time Steps Per Second** en la **Physics Scene** del proyecto que ejecutemos ya que mientras más alto sea este valor más le costará alcanzar el tiempo real al simulador. Lo podemos ver en la siguiente imagen en la que se aumenta a 5000, lo que hace que el RTF se reduzca de 1 a 0.8 aproximadamente. Esto dependerá del hardware disponible y lo complejo del proyecto:

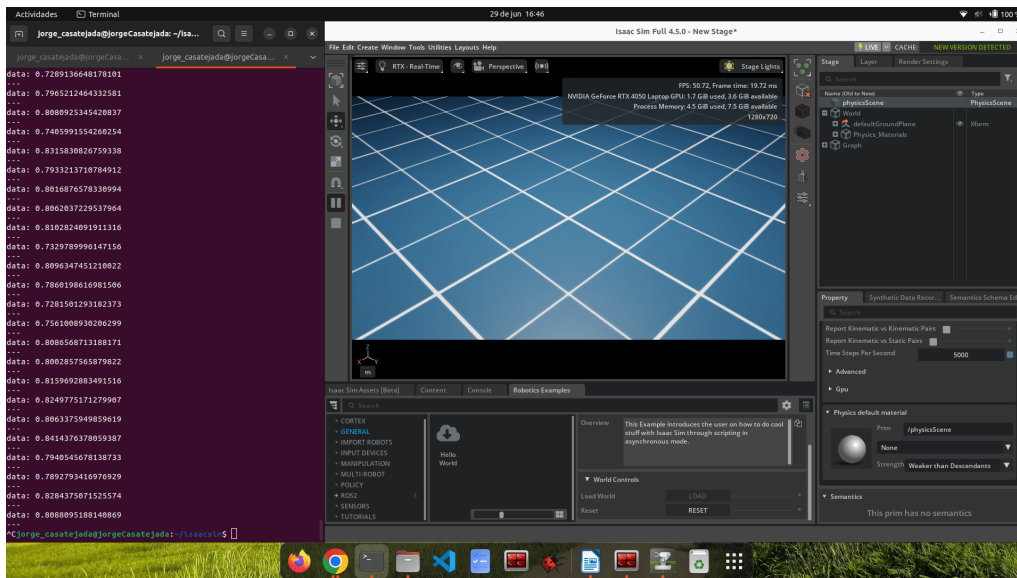


Figura 1.16: Isaac Sim con $RTF \approx 0.8$

Por otra parte [66, 67] para lograr aumentar el real time factor a más de uno se tendrá que usar un standalone script de python utilizando `SimulationApp` y `python.sh` [63] ya que, como he dicho antes, el simulador intentará siempre correr a tiempo real.

1.5. Tabla comparativa resumen

Por último se presenta una tabla comparativa a modo de resumen con los aspectos más importantes de los 3 simuladores.

Tabla 1.2: Tabla comparativa resumen de simuladores robóticos

Aspecto	Gazebo	Webots	Isaac Sim
---------	--------	--------	-----------

Pros	• Física Realista • Integración nativa con ROS • Amplia comunidad	• Ligero en recursos • Fácil de usar • Versátil para exteriores	• Fotorrealismo RTX • Generación de datos sintéticos • Aceleración por GPU
Contras	• Consumo intensivo de CPU • Curva de aprendizaje alta	• Comunidad relativamente pequeña • Física menos precisa	• Requisitos de hardware muy exigentes • Dependencia de Omniverse
Coste computacional	Alto (CPU/RAM)	Bajo (CPU/RAM)	Muy alto (GPU/-VRAM y RAM)
Comunidad	Muy activa (ROS 2)	Mediana, con foco educativo	Grande pero menos enfocada en ROS
Escalabilidad	Buen RTF en entornos grandes	Webots.HPC en clusters	Entrenamiento paralelo masivo en GPU
Exteriores	• Terrenos irregulares • Plugins de clima	• Integración con OSM • Simulación meteorológica	Escenarios urbanos dinámicos (GRADE)
Coste monetario	Gratis y open-source	Gratis y open-source	• Gratuito para desarrolladores • Pago en empresas
GPU	No es necesaria (mejora el rendimiento)	No es necesaria (mejora el rendimiento)	Obligatoria
GUI	• gzserver (sin GUI) • ign gazebo -s (sin GUI)	• -no-rendering • -stdout • -stderr • -minimize	isaac-sim.streaming.sh (headless)
Tiempo real	• Ajuste preciso de RTF • gz physics -u [valor] • Modificación en .world	• Modos: pause/realttime/fast • Ajuste mediante basicTimeStep	• RTF controlado por Time Steps Per Second • Modos RTX/Path Tracing • Scripts Python para RTF > 1

Capítulo 2

Gestión de proyecto software

En este capítulo, se describen los aspectos relacionados con la gestión del proyecto, incluyendo la planificación inicial del proyecto, el presupuesto y la gestión de recursos.

2.1. Alcance del proyecto

El resultado final de este proyecto consiste en encontrar una alternativa al simulador de Gazebo que permita la simulación de robots en exteriores bajo ROS 2 y haga un menor uso de recursos computacionales ya que Gazebo presenta limitaciones en cuanto a los recursos computacionales necesarios para lanzar simulaciones complejas y cambiantes propios de la robótica de exteriores, como terrenos irregulares, vegetación dinámica o condiciones meteorológicas.

Para ello previamente se realizará una exploración de las posibles alternativas que existen en el ámbito de los simuladores de robótica para después hacer una comparativa, en términos de recursos computacionales, de las mejores alternativas encontradas. Para esta comparación se medirá el rendimiento de los simuladores, incluyendo Gazebo, en un entorno controlado en el que se ejecutará un ejemplo de un robot el cual haga uso de ROS 2 y Nav2. Una vez medido el uso de recursos computacionales se procederá a la comparación de los resultados para encontrar la mejor alternativa a Gazebo.

Por último se desarrollará un proyecto de ejemplo en la mejor alternativa. Este ejemplo será de simulación de robots en exteriores bajo ROS 2 y Nav2 para poder comprobar de manera práctica los resultados obtenidos, además de estudiar en más profundidad la mejor alternativa.

2.2. Plan de trabajo

Se ha desarrollado una planificación inicial que incluye desde las primeras fases de la exploración de información de los simuladores y la comparativa entre simuladores hasta el desarrollo de un ejemplo de la mejor alternativa a Gazebo encontrada. Se ha adaptado este plan tradicional al uso de la metodología de desarrollo ágil Scrum [94] para la gestión del desarrollo del proyecto. En esta sección se mostrarán las tareas que se han identificado, su estimación y su planificación.

2.2.1. Identificación de tareas

Las tareas identificadas han sido las siguientes:

- **Tarea 1:** Este grupo de tareas se centra en la búsqueda de información y análisis inicial exploratorio para recabar datos y distintas opciones sobre los distintos simuladores disponibles en robótica. Está formado por las siguientes tareas:
 - **Tarea 1.1:** Exploración de los principales simuladores para robótica. Además de esta exploración también se extraerá la información más relevante de cada uno de ellos.
 - **Tarea 1.2:** Exploración de los principales simuladores que podrían sustituir a Gazebo, es decir, que sean compatibles con ROS 2. Además de esta exploración también se extraerá información más específica y detallada de cada uno de ellos.
- **Tarea 2:** Este grupo de tareas consiste en todo lo relacionado con la preparación de los simuladores a comparar. Está formado por las siguientes tareas:
 - **Tarea 2.1:** Instalación de los simuladores y realización de los tutoriales de instalación de cada uno de ellos.
 - **Tarea 2.2:** Preparación de los ejemplos a usar posteriormente en las mediciones, explorando distintas opciones dependiendo de cada simulador, además de realizar los tutoriales correspondientes para hacer uso de los mismos.
 - **Tarea 2.3:** Exploración del lanzamiento de los simuladores y el uso de GPU. Buscar información y probar cómo lanzar cada simulador haciendo uso de GPU o sin ella si es posible.

- **Tarea 2.4:** Exploración del lanzamiento de los simuladores y el uso de GUI (Interfaz Gráfica de Usuario). Buscar información y probar cómo lanzar cada simulador haciendo uso de GUI o sin ella si es posible.
- **Tarea 2.5:** Exploración del lanzamiento de los simuladores y el uso de tiempo real. Buscar información y probar cómo lanzar cada simulador con las opciones de tiempo real disponibles para cada uno de ellos.
- **Tarea 3:** Este grupo de tareas congrega todo lo relacionado con la medición de recursos computacionales de cada simulador y su comparación.
 - **Tarea 3.1:** Elección de métricas. Consistirá en investigar y decidir qué métricas se van a recopilar para su posterior uso.
 - **Tarea 3.2:** Desarrollo de los scripts de medición. Más específicamente, realizar los scripts necesarios para poder automatizar el lanzamiento de los simuladores con sus ejemplos y el comienzo de las mediciones. Se deberá conseguir que con estos scripts los simuladores se inicien en el modo indicado por el usuario, los robots naveguen autónomamente y las métricas sean monitorizadas.
 - **Tarea 3.3:** Diseño de escenarios. Consiste en pensar en que escenarios se probarán los distintos simuladores.
 - **Tarea 3.4:** Realización de las mediciones. Esta tarea consiste en realizar la medición de uso de recursos computacionales haciendo uso de los scripts anteriores. Se tendrán que realizar varias repeticiones para cada escenario propuesto para que los datos sean más consistentes.
 - **Tarea 3.5:** Comparación de mediciones. Se deberán obtener los datos estadísticos, a través de JASP, para su posterior análisis exhaustivo que permita tanto la comparación entre modos de un simulador como la comparación entre simuladores. Hay que tener en cuenta que se necesitará una previa preparación de los datos para poder usarlos en JASP.
- **Tarea 4:** Análisis exhaustivo de los datos. Una vez realizadas todas las mediciones y obtenidos los datos estadísticos, se procederá al análisis de los mismos para poder formar una conclusión de los resultados obtenidos.
- **Tarea 5:** Desarrollo de un ejemplo de ROS 2 en un entorno de exteriores en el simulador con mejores resultados respecto a los recursos computacionales.

Esta tarea incluye realizar una guía paso a paso explicando el proceso seguido para el desarrollo.

- **Tarea 6:** Documentación. A lo largo de todo el desarrollo del proyecto se irán desarrollando las partes respectivas de la documentación del Trabajo de Fin de Máster, tanto como borradores rápidos como su redacción más detallada y pulida.

2.2.2. Estimación de tareas

El proyecto tiene una duración de 276 horas, equivalentes a 92 días laborables, es decir, 5 meses y 1 semana si se dedican 4 días a las semana y 3 horas diarias de trabajo cada uno de esos 4 días. De esta forma se presenta la estimación de las tareas descritas anteriormente, especificando también sus fechas de inicio y fin, teniendo en cuenta que se trabajarán 4 días a la semana, así como las dependencias entre ellas.

Tareas	Días	Fecha inicio	Fecha fin	Predecesoras
Tarea 1	10	24/02/2025	12/03/2025	-
Tarea 1.1	3	24/02/2025	28/02/2025	-
Tarea 1.2	7	28/02/2025	12/03/2025	-
Tarea 2	20	12/03/2025	16/04/2025	Tarea 1.2
Tarea 2.1	10	12/03/2025	30/03/2025	-
Tarea 2.2	4	30/03/2025	06/04/2025	Tarea 2.1
Tarea 2.3	1	06/04/2025	07/04/2025	Tarea 2.1
Tarea 2.4	1	07/04/2025	09/04/2025	Tarea 2.1
Tarea 2.5	4	09/04/2025	16/04/2025	Tarea 2.1
Tarea 3	31	16/04/2025	09/06/2025	Tarea 2
Tarea 3.1	2	16/04/2025	20/04/2025	-
Tarea 3.2	5	20/04/2025	28/04/2025	Tarea 3.1
Tarea 3.3	1	28/04/2025	30/04/2025	Tarea 3.1
Tarea 3.4	20	30/04/2025	04/06/2025	Tarea 3.2, Tarea 3.3
Tarea 3.5	3	04/06/2025	09/06/2025	Tarea 3.4
Tarea 4	9	09/06/2025	25/06/2025	Tarea 3
Tarea 5	22	25/06/2025	03/08/2025	Tarea 2.1
Tarea 6	92	24/02/2025	03/08/2025	-

Tabla 2.1: Estimación de tareas

2.2.3. Planificación de tareas

En la figura 2.1 se muestra el diagrama de Gantt que se corresponde con las tareas definidas anteriormente en la Tabla 2.1. En este diagrama se especifican las dependencias y solapamientos entre tareas. Esta planificación resulta en una duración de 23 semanas, 5 meses y 1 semana, que comienza el 24 de febrero de 2025 y finaliza el 3 de agosto de 2025 con un trabajo de 3 horas diarias 4 días a la semana y teniendo en cuenta que de los 4 días de la semana establecidos para el trabajo serán los lunes, miércoles, viernes y domingo.

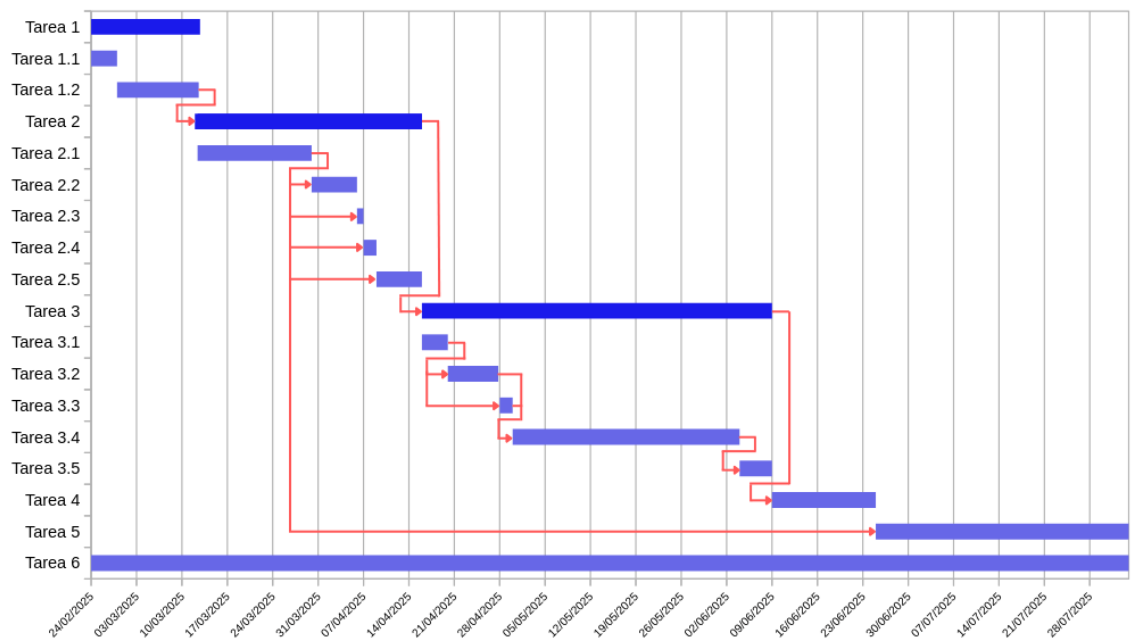


Figura 2.1: Diagrama de Gantt de la planificación inicial de las tareas

2.2.4. Iteraciones de Scrum

En esta sección se presentan las iteraciones de Scrum que se han llevado a cabo durante el desarrollo del proyecto. La duración de las iteraciones depende de las tareas a realizar en cada una de ellas, por lo que son de duración variables. En la tabla 2.2 podemos observar las iteraciones que se han llevado a cabo.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
1	24/02/25	12/03/25	Tarea 1.1	Buscar información general sobre los simuladores más conocidos para robótica.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
			Tarea 1.1	Identificar sus principales características, ventajas y desventajas.
			Tarea 1.1	Resumir la información más relevante de cada simulador
			Tarea 1.2	Seleccionar simuladores que soporten integración directa con ROS 2.
			Tarea 1.2	Analizar características específicas como soporte para sensores, controladores y navegación.
			Tarea 1.2	Crear un resumen con la información más relevante para comparación futura.
2	12/03/25	30/03/25	Tarea 2.1	Descargar y configurar cada simulador en el entorno de trabajo.
			Tarea 2.1	Seguir los tutoriales oficiales o comunitarios para cada simulador.
			Tarea 2.1	Verificar que cada instalación funcione correctamente mediante pruebas básicas.
			Tarea 2.1	Documentar los pasos seguidos y posibles problemas encontrados con sus soluciones.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
3	30/03/25	09/04/25	Tarea 2.2	Identificar ejemplos de navegación o simulación incluidos en cada simulador.
			Tarea 2.2	Probar los ejemplos para comprobar su correcto funcionamiento.
			Tarea 2.2	Documentar los pasos seguidos y posibles problemas encontrados con sus soluciones.
			Tarea 2.3	Investigar si cada simulador soporta ejecución acelerada por GPU.
			Tarea 2.3	Probar configuraciones de lanzamiento con y sin GPU activada.
			Tarea 2.3	Anotar los comandos, parámetros o archivos de configuración necesarios para el uso de GPU o no GPU.
			Tarea 2.4	Investigar cómo lanzar cada simulador con GUI activa o inactiva.
			Tarea 2.4	Probar configuraciones de lanzamiento con y sin GUI activada.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
			Tarea 2.4	Anotar los comandos, parámetros o archivos de configuración necesarios para el uso de GUI o no GUI.
			Tarea 2.5	Buscar documentación sobre tiempo real y su influencia en la simulación.
			Tarea 2.5	Buscar documentación sobre el soporte de tiempo real en cada simulador.
			Tarea 2.5	Probar los distintos modos de ejecución con control de tiempo
			Tarea 2.5	Anotar los comandos, parámetros o archivos de configuración necesarios para modificar el uso de tiempo real.
4	16/04/25	30/04/25	Tarea 3.1	Investigar qué métricas son relevantes para evaluar rendimiento en simuladores.
			Tarea 3.1	Seleccionar las herramientas y técnicas para medir CPU, GPU, RAM y tiempo real.
			Tarea 3.2	Crear scripts que permitan lanzar automáticamente los simuladores con parámetros definidos.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
			Tarea 3.2	Integrar la ejecución de ejemplos con el inicio de la navegación autónoma del robot.
			Tarea 3.2	Programar la monitorización de métricas como CPU, RAM, GPU y RTF durante la simulación.
			Tarea 3.2	Comprobar que la salida de los scripts tiene sentido y es válida.
			Tarea 3.3	Diseñar los escenarios que permitan la comparación entre simuladores.
			Tarea 3.3	Asegurarse de que los escenarios sean comparables entre simuladores.
5	30/04/25	04/06/25	Tarea 3.4	Ejecutar los scripts de medición con cada escenario y simulador configurado.
			Tarea 3.4	Repetir cada prueba múltiples veces para obtener resultados consistentes.
			Tarea 3.4	Guardar los resultados en archivos organizados por simulador, escenario y repetición.
			Tarea 3.4	Verificar que los datos registrados sean válidos y estén completos.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
6	04/06/25	25/06/25	Tarea 3.5	Cargar los datos en JASP y calcular medidas estadísticas.
			Tarea 3.5	Comparar los resultados entre distintos modos del mismo simulador.
			Tarea 3.5	Realizar comparaciones cruzadas entre simuladores para extraer diferencias clave.
			Tarea 4	Relacionar el rendimiento con las características técnicas de cada simulador.
			Tarea 4	Elaborar conclusiones basadas en evidencia objetiva y respaldada por los datos.
			Tarea 4	Redactar un resumen detallado con hallazgos principales y recomendaciones.
7	25/06/25	03/08/25	Tarea 5	Seleccionar el simulador con mejores resultados en consumo de recursos.
			Tarea 5	Diseñar un escenario de exteriores compatible con un robot de exteriores y ROS 2.
			Tarea 5	Desarrollar un ejemplo funcional de ROS 2 con navegación autónoma en dicho entorno.

Iteración	Fecha inicio	Fecha fin	Tareas	Subtarea
			Tarea 5	Elaborar una guía paso a paso que documente todo el proceso desde cero.
			Tarea 6	Completar la memoria del Trabajo de Fin de Máster.

Tabla 2.2: Iteraciones de Scrum

2.3. Gestión de recursos

En esta sección se presentan los recursos que han sido usados en el proyecto, exponiendo sus costes y características.

2.3.1. Especificación de recursos

Los recursos usados en el proyecto son los siguientes:

- **Ordenador portátil personal:**

- Procesador: Intel Core i7-13620H
- Memoria RAM: 16 GB
- Almacenamiento: 512 GB
- Tarjeta gráfica: NVIDIA RTX 4050 (6 GB)
- Sistema operativo: Ubuntu 22.04 LTS

- **Software(acceso libre o licencia educativa):**

- Ignition Gazebo
- Gazebo Classic
- Webots
- Isaac Sim
- Visual Studio Code
- Python

- ROS 2
- Nav 2
- RViz
- JASP
- Git
- Github
- Overleaf
- LibreOffice Writer & calc
- Google Drive, Gmail, Meet & Chrome
- Discord
- Draw.io

2.3.2. Presupuesto

En esta sección se muestra el presupuesto estimado para el proyecto.

Coste de personal

Para la realización de este proyecto se ha necesitado de un Ingeniero Informático y de un Scrum Master. Para obtener los salarios de ambos se ha hecho uso de la página de indeed [33], de la cual se obtienen los salarios medios. Además, para calcular el coste del personal es necesario aplicar las tasas de la Seguridad Social [3] al salario base que se presentan en la tabla 2.3.

Tasa	Valor
Contingencias comunes	23,60 %
Cuota de desempleo (empresa)	5,50 %
Cuota de formación profesional	0,60 %
Fogasa	0,20 %
Accidentes de trabajo y enfermedades profesionales	1,50 %
Mecanismo de Equidad Intergeneracional (MEI)	0,80 %
Total cuotas empresariales	32,20 %

Tabla 2.3: Tasas de la Seguridad Social Española

En la tabla 2.4 se presenta el coste del personal del proyecto, habiendo obtenido el coste por hora estimado del perfil de Ingeniero Informático a partir del salario medio publicado por el portal de empleo Indeed en su sección de salarios en España [34]. De forma análoga, el coste del perfil de Scrum Master se ha extraído del mismo portal, utilizando las cifras salariales actualizadas para dicha categoría profesional [35]. Se tendrá en cuenta la duración de 5 meses y 1 semana, con un total de 92 días laborables y 3 horas diarias. El coste por hora se calcula aplicando la fórmula:

$$\text{Coste/hora} = \text{Salario/hora} \times (1 + \text{Tasa total empresarial}) \quad (2.1)$$

siendo esta del 32,20 % según las cotizaciones vigentes.

Puesto	Salario/hora (€)	Coste/hora (€)	Horas trabajadas	Total (€)
Ingeniero Informático	16,73	22,28	276	6.150,48
Scrum Master	17,24	22,96	40	918,55
Total				7.069,03

Tabla 2.4: Coste del personal

Coste de hardware

En este apartado se presentan los costes del hardware empleado. En este Trabajo de Fin de Máster el único recurso hardware utilizado ha sido el portátil personal, que tiene el siguiente coste:

- El coste sin IVA de este portátil es de 612,91 €.
- El tiempo para amortizarlo es de 3 años.
- Se ha utilizado durante 5 meses y 1 semana, por lo que su coste estimado sin IVA es de 89,42 €.

Por lo tanto, el coste total de hardware es de **89,42 €**.

Costes indirectos industrial

En este apartado se presenta el cálculo de los costes indirectos. Estos costes incluyen la luz, el alquiler, el agua, la red y otros conceptos que no se tienen en cuenta de forma directa en el proyecto. Según el Artículo 55 de los Presupuestos de la Universidad de León [31], es el 15 % y se calcula sobre el coste total del proyecto

sin IVA. De esta manera, el beneficio industrial de este proyecto se ve representado en la tabla 2.5.

Concepto	Valor (€)
Coste del personal	7.069,03
Coste del hardware	89,42
Subtotal	7.158,45
Costes indirectos (15 %)	1.073,77

Tabla 2.5: Costes indirectos

Beneficio industrial

En este apartado se presenta el beneficio industrial, el cual es el porcentaje que se obtiene de beneficio con los resultados. Según el BOE [30], es del 6 % y se calcula sobre el coste total del proyecto sin IVA, por lo que el beneficio industrial obtenido en este proyecto se presenta en la tabla 2.6.

Concepto	Valor (€)
Coste del personal	7.069,03
Coste del hardware	89,42
Subtotal	7.158,45
Beneficio industrial (6 %)	429,51

Tabla 2.6: Beneficio industrial

Coste total

Una vez calculados los costes de personal, los costes del hardware, los costes indirectos y el beneficio industrial se realiza el cálculo del coste total. En este cálculo se aplica el IVA y está presentado en la tabla 2.7.

Concepto	Valor (€)
Coste del personal	7.069,03
Coste del hardware	89,42
Costes indirectos (15 %)	1.073,77
Beneficio industrial (6 %)	429,51
Subtotal	8.661,73
IVA (21 %)	1.818,96
Total	10.480,69

Tabla 2.7: Coste total

2.4. Gestión de riesgos

En esta sección se detalla el proceso seguido para la gestión de riesgos, comenzando por la identificación de las principales amenazas y concluyendo con la descripción de las estrategias de mitigación diseñadas para abordarlas.

2.4.1. Identificación de riesgos

En el desarrollo del proyecto se han identificado los siguientes riesgos potenciales que podrían afectar a su correcta ejecución:

- **Elevado consumo de recursos computacionales en simulación:** El proyecto implica la evaluación de simuladores de robótica, algunos de los cuales, como Ignition Gazebo e Isaac Sim, son conocidos por su alta demanda de CPU, GPU y memoria RAM. El hardware utilizado podría no ser suficiente para ejecutar las simulaciones de manera fluida, lo que podría provocar fallos en el sistema. Posibilidad de ocurrencia: *Alta*. Impacto: *Alto*.
- **Inestabilidad o falta de soporte en herramientas de simulación:** Al depender de software de código abierto y en constante desarrollo, existe el riesgo de encontrar errores (bugs), problemas de compatibilidad entre versiones (simulador, ROS 2, drivers de GPU) o documentación insuficiente para casos de uso específicos. Estos problemas podrían bloquear el progreso o requerir una cantidad de tiempo no planificada para su resolución. Posibilidad de ocurrencia: *Media*. Impacto: *Alto*.
- **Falta de disponibilidad de hardware:** El desarrollo del proyecto se realiza en un único ordenador portátil personal. Una avería de hardware, robo o

pérdida del equipo supondría un punto único de fallo que podría detener por completo el avance del proyecto. Posibilidad de ocurrencia: *Baja*. Impacto: *Alto*.

- **Sobrecarga de trabajo en el equipo de desarrollo:** Al ser un proyecto realizado por una única persona, existe el riesgo de subestimar el tiempo necesario para completar todas las tareas (investigación, implementación, experimentación, análisis y redacción). La aparición de dificultades imprevistas podría generar un efecto dominó en la planificación, comprometiendo los plazos de entrega. Posibilidad de ocurrencia: *Media*. Impacto: *Alto*.

2.4.2. Mitigación de riesgos

Para cada uno de los riesgos identificados en la sección anterior, se ha diseñado un plan de mitigación específico con el fin de minimizar su impacto:

- **Elevado consumo de recursos computacionales en simulación:**
 1. **Validación previa de requisitos:** Antes de comenzar la fase experimental, se verificó que el hardware disponible cumplía con los requisitos mínimos y recomendados de cada simulador.
 2. **Optimización de escenarios:** Se utilizarán mundos y modelos de robots de complejidad controlada para evitar una sobrecarga innecesaria.
 3. **Ejecución en modo sin interfaz gráfica (*headless*):** Como parte de la propia evaluación, se realizarán pruebas sin la GUI, lo que reduce drásticamente la carga sobre la GPU y la CPU.
 4. **Gestión de procesos:** Durante la ejecución de las pruebas, se cerrarán todas las aplicaciones no esenciales para asegurar que los simuladores dispongan de la máxima cantidad de recursos posible.
- **Inestabilidad o falta de soporte en herramientas de simulación:**
 1. **Uso de versiones estables (LTS):** Se ha optado por utilizar versiones de soporte a largo plazo como Ubuntu 22.04 y ROS 2 Humble, que garantizan una mayor estabilidad.
 2. **Recursos comunitarios:** Se recurrirá activamente a la documentación oficial, foros de la comunidad y repositorios de GitHub para buscar soluciones.

3. **Aislamiento de problemas:** En caso de error, se crearán casos de prueba mínimos para aislar la causa del problema y facilitar su diagnóstico.

- **Falta de disponibilidad de hardware:**

1. **Control de versiones:** Todo el código fuente, scripts y archivos de configuración se gestionan a través de un repositorio Git alojado en GitHub. Además la documentación se gestionará a través de Overleaf.
2. **Copias de seguridad en la nube:** Los datos experimentales, resultados y la memoria del TFM se guardarán en un servicio de almacenamiento en la nube (Google Drive).
3. **Plan de contingencia:** La combinación de Git y Google Drive permitiría restaurar el entorno de trabajo en un equipo alternativo en un tiempo reducido.

- **Sobrecarga de trabajo en el equipo de desarrollo:**

1. **Metodología ágil (Scrum):** Se ha adoptado un enfoque de desarrollo ágil, dividiendo el trabajo en iteraciones cortas (sprints) con objetivos claros y entregables definidos.
2. **Priorización de tareas:** Se han priorizado las tareas esenciales para cumplir los objetivos principales del TFM, simplificando aspectos secundarios si fuera necesario.
3. **Seguimiento regular con el tutor:** Se mantendrán reuniones para revisar el progreso, validar resultados y reajustar la planificación, actuando como mecanismo de control.

2.5. Legislación y normativa

En el marco de ejecución de este proyecto, se ha llevado a cabo un riguroso cumplimiento de la legislación y normativa vigente. A continuación, se detalla cómo el proyecto se ajusta y adhiere a las leyes pertinentes:

- **Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales [10]**

Este proyecto respeta plenamente la Ley Orgánica 3/2018, la cual reconoce el derecho fundamental a la protección de datos personales. La creación del dataset se ha desarrollado de acuerdo con las disposiciones de la ley, asegurando

la legalidad en el tratamiento de datos. Los permisos explícitos obtenidos para el uso de imágenes garantizan la privacidad de los individuos. Además, las pruebas y el desarrollo se realizaron en un entorno controlado para evitar perjuicios en materia de datos a terceros. Aunque las imágenes capturadas por la cámara del robot no se almacenan, se garantiza que el procesamiento cumple con los principios de protección de datos establecidos en la ley.

- **Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD) [101]**

La creación del dataset en este proyecto se ajusta a los principios del RGPD, asegurando la legalidad y transparencia en el tratamiento de datos personales. Se han implementado medidas técnicas y organizativas para garantizar la seguridad y privacidad de los datos presentes en el dataset, cumpliendo así con las normativas del RGPD.

- **Propuesta de Reglamento del Parlamento Europeo y del Consejo sobre Inteligencia Artificial (PRLIA) [5]**

El proyecto ha sido diseñado y ejecutado considerando de manera integral las disposiciones y requisitos establecidos por la PRLIA. Un documento legislativo propuesto por la Unión Europea para regular el uso de la inteligencia artificial en diversos ámbitos. Esta propuesta tiene como objetivo establecer normas armonizadas en materia de inteligencia artificial para garantizar la seguridad, ética y transparencia en su desarrollo y aplicación. Aunque el proyecto no involucra una IA de alto riesgo, se han tenido en cuenta las recomendaciones éticas reflejadas en el marco ético del proyecto. No se ha incumplido ninguna disposición de la PRLIA, ya que no se han violado derechos fundamentales ni se han utilizado técnicas subliminales o vulnerabilidades de grupos específicos.

- **Real Decreto Legislativo 1/1996 sobre Propiedad Intelectual [50]**

En conformidad con el Real Decreto Legislativo 1/1996, el proyecto respeta la normativa sobre propiedad intelectual. Se ha optado por utilizar únicamente código y herramientas de software libre y de código abierto para garantizar el cumplimiento de la normativa en materia de propiedad intelectual.

Capítulo 3

Solución

En este capítulo, se presentan los diferentes aspectos de la solución propuesta, incluyendo la descripción de la solución, su desarrollo y el resultado de la misma.

3.1. Metodologías de evaluación

El objetivo principal de este TFM es identificar un simulador que presente un menor uso de recursos computacionales en comparación con Gazebo, sin comprometer significativamente la funcionalidad necesaria para el desarrollo de sistemas robóticos con ROS 2. Para ello, se seleccionan los tres simuladores ya vistos anteriormente del ámbito de la simulación robótica y ROS 2: **Gazebo**, **Webots** e **Isaac Sim**.

Con el fin de garantizar una comparación justa y objetiva, los tres simuladores se han evaluado bajo condiciones controladas y equivalentes. Las pruebas se han llevado a cabo en el mismo equipo, ejecutando tareas similares para cada entorno y monitorizando métricas relevantes relacionadas con el rendimiento del sistema.

Una vez obtenidas todas las mediciones, se procederá a realizar un análisis estadístico de los resultados con el propósito de identificar si existen diferencias significativas entre los simuladores. Este análisis permitirá determinar si alguna de las alternativas evaluadas puede considerarse una opción viable y más eficiente que Gazebo en términos de consumo de recursos computacionales.

3.2. El proceso de desarrollo

3.2.1. Análisis

El proceso de desarrollo comenzó con un análisis detallado de las necesidades y condiciones para llevar a cabo la comparación entre simuladores. Se identificaron los requisitos funcionales y no funcionales que debía cumplir la solución para garantizar la validez y reproducibilidad de los experimentos.

Definición de requisitos

■ Requisitos funcionales (RF)

- **RF1:** Implementar el código necesario que automatice el lanzamiento del simulador y el inicio de la navegación a diferentes waypoints.
- **RF2:** Implementar el código necesario que automatice la medición del consumo energético (batería) y almacene los datos recogidos.
- **RF3:** Implementar el código necesario que automatice la medición de métricas de uso de CPU, memoria RAM y GPU, y almacene dicha información.
- **RF4:** Implementar el código necesario que automatice la medición del factor de tiempo real y almacene los datos recogidos.
- **RF5:** Implementar un script en Python que integre y coordine los requisitos anteriores para automatizar completamente el proceso de medición de los simuladores.
- **RF6:** Adaptar los scripts de automatización a las particularidades y diferencias de interfaz o comandos de cada simulador para asegurar una ejecución correcta y homogénea en todos los entornos evaluados.
- **RF7:** Desarrollar un ejemplo funcional en el simulador alternativo a Gazebo, acompañado de un tutorial paso a paso para facilitar su uso y replicación.
- **RF8:** Realizar el análisis estadístico de los datos obtenidos utilizando la herramienta JASP para facilitar la comparación objetiva entre simuladores.

■ Requisitos no funcionales (RNF)

- **RNF1:** Asegurar la reproducibilidad de las pruebas mediante la automatización completa y la definición clara de las condiciones de ejecución.
- **RNF2:** Garantizar que los scripts sean robustos frente a posibles fallos o excepciones durante la ejecución para evitar interrupciones no deseadas.
- **RNF3:** Diseñar los scripts con modularidad para facilitar su mantenimiento, adaptación y extensión a diferentes simuladores o futuras versiones.
- **RNF4:** Optimizar la eficiencia de los scripts para minimizar la carga adicional que puedan generar durante las mediciones del rendimiento.
- **RNF5:** Garantizar la compatibilidad de los scripts y del entorno de ejecución con el hardware y sistema operativo usados en las pruebas.

Especificación de requisitos

CU-01	Automatizar el lanzamiento y navegación en el simulador	
Versión	1.0	
Descripción	El sistema ejecuta un script en Python que lanza el simulador seleccionado y ordena al robot simulado navegar a diferentes waypoints predefinidos.	
Actores	Usuario, Script de automatización	
Precondición	El simulador debe estar instalado y configurado correctamente en el equipo.	
Flujo básico	Paso	Acción
	1	El usuario ejecuta el script de automatización.
	2	El script lanza el simulador seleccionado.
	3	El script inicializa el nodo de navegación y envía los comandos para que el robot simulado se desplace a los waypoints.
	4	El script monitoriza el progreso de la navegación.
	5	Al finalizar la navegación, el script detiene el simulador.

Flujo alternativo	1a	Si el simulador no se lanza correctamente, el script registra el error y finaliza la ejecución.
	3a	Si el robot simulado no puede llegar a un waypoint, el script intenta con el siguiente waypoint o termina la prueba.
Postcondición	El simulador se detiene y el script ha registrado el resultado de la navegación para su posterior análisis.	

Tabla 3.1: Caso de uso 1: Automatización de lanzamiento y navegación

CU-02	Medición y registro del consumo energético	
Versión	1.0	
Descripción	El sistema ejecuta un script en Python que automatiza la medición del consumo energético durante la ejecución del simulador y guarda los datos obtenidos.	
Actores	Usuario, Script de medición energética	
Precondición	Se debe contar con las herramientas necesarias para medir el consumo energético (powerstat).	
Flujo básico	Paso	Acción
	1	El usuario ejecuta el script de medición energética.
	2	El script inicia la monitorización del consumo energético durante la simulación.
	3	El script recopila y almacena los datos del consumo energético.
	4	Al finalizar la simulación, el script detiene la monitorización.
	5	Los datos son guardados para su posterior análisis.
Flujo alternativo	2a	Si la monitorización del consumo no es posible, el script notifica el fallo y termina la ejecución.
Postcondición	Se dispone de un conjunto de datos sobre el consumo energético para su análisis.	

Tabla 3.2: Caso de uso 2: Medición y registro del consumo energético

CU-03	Medición y registro de métricas de CPU, RAM y GPU	
Versión	1.0	
Descripción	El sistema ejecuta un script en Python que mide y registra el uso de CPU, memoria RAM y GPU durante la ejecución del simulador.	
Actores	Usuario, Script de medición de recursos	
Precondición	El sistema debe contar con las herramientas necesarias para monitorizar recursos y estar correctamente configurado.	
Flujo básico	Paso	Acción
	1	El usuario ejecuta el script de medición de recursos.
	2	El script inicia la monitorización de CPU, RAM y GPU durante la simulación.
	3	El script recopila y almacena los datos de uso de recursos.
	4	Al finalizar la simulación, el script detiene la monitorización.
	5	Los datos son guardados para su posterior análisis.
Flujo alternativo	2a	Si la monitorización de recursos no es posible, el script notifica el fallo y termina la ejecución.
Postcondición	Se dispone de un conjunto de datos sobre uso de CPU, RAM y GPU para su análisis.	

Tabla 3.3: Caso de uso 3: Medición y registro de métricas de CPU, RAM y GPU

CU-07	Medición y registro del factor de tiempo real (RTF)	
Versión	1.0	
Descripción	El sistema ejecuta un script en Python que automatiza la medición del factor de tiempo real (RTF) durante la simulación, recopilando y almacenando los datos para su posterior análisis.	
Actores	Usuario, Script de medición de RTF	

Precondición	El simulador debe estar en ejecución y publicar el dato del factor de tiempo real en el topic correspondiente.	
Flujo básico	Paso	Acción
	1	El usuario ejecuta el script de medición del RTF.
	2	El script se suscribe al topic donde se publica el factor de tiempo real.
	3	El script recopila periódicamente los datos del RTF durante la simulación.
	4	El script almacena los datos recogidos en un archivo para su análisis.
	5	Al finalizar la simulación, el script detiene la recopilación y guarda el archivo final.
Flujo alternativo	2a	Si el topic de RTF no está disponible, el script notifica el error y finaliza la ejecución.
	3a	Si hay pérdida de datos, el script continúa recopilando cuando la conexión se restablece.
Postcondición	Se dispone de un conjunto de datos sobre el factor de tiempo real para su análisis posterior.	

Tabla 3.4: Caso de uso 4: Medición y registro del factor de tiempo real (RTF)

CU-04	Integración y automatización completa del proceso de medición
Versión	1.0
Descripción	El sistema ejecuta un script en Python que integra y coordina los scripts de lanzamiento, navegación y medición para automatizar completamente el proceso de evaluación de los simuladores.
Actores	Usuario, Script integrador
Precondición	Todos los scripts individuales deben estar implementados y probados.

Flujo básico	Paso	Acción
	1	El usuario ejecuta el script integrador.
	2	El script lanza el simulador y comienza la navegación.
	3	Durante la simulación, se ejecutan en paralelo las mediciones de consumo energético y uso de recursos.
	4	El script monitoriza el progreso y coordina la recogida de datos.
Flujo alternativo	5	Al finalizar la prueba, el script detiene todos los procesos y almacena los datos recopilados.
	2a	Si el simulador no se inicia correctamente, el script registra el error y termina la ejecución.
	3a	Si alguna medición falla, el script intenta continuar con las demás y notifica la incidencia.
Postcondición	Se dispone de un conjunto completo de datos de la prueba para análisis posterior.	

Tabla 3.5: Caso de uso 5: Integración y automatización completa del proceso

CU-05	Desarrollo y documentación de un ejemplo y tutorial para simulador alternativo	
Versión	1.0	
Descripción	Se desarrolla un ejemplo funcional en el simulador alternativo a Gazebo, acompañado de un tutorial paso a paso para facilitar su uso y replicación.	
Actores	Usuario, Desarrollador	
Precondición	El simulador alternativo debe estar instalado y operativo.	
Flujo básico	Paso	Acción
	1	El desarrollador crea un escenario básico en el simulador alternativo.
	2	Se configura la navegación para un robot simulado dentro del escenario.
	3	Se documenta el proceso paso a paso en un tutorial accesible.

Flujo alternativo	3a	Si el tutorial no es claro, se revisa y mejora el contenido.
Postcondición	Se dispone de un ejemplo y tutorial funcional para el simulador alternativo.	

Tabla 3.6: Caso de uso 6: Desarrollo y documentación de ejemplo y tutorial

CU-06	Análisis estadístico de los datos obtenidos	
Versión	1.0	
Descripción	El sistema utiliza la herramienta JASP para realizar el análisis estadístico de los datos recopilados en las pruebas con los diferentes simuladores, facilitando la comparación objetiva.	
Actores	Analista, Usuario	
Precondición	Los datos de las pruebas deben estar completos y almacenados correctamente.	
Flujo básico	Paso	Acción
	1	El analista importa los datos en JASP.
	2	Se seleccionan las pruebas estadísticas adecuadas según los objetivos.
	3	Se ejecuta el análisis y se interpretan los resultados.
	4	Se generan informes con conclusiones y recomendaciones.
Flujo alternativo	3a	Si los datos presentan inconsistencias, se revisan y limpian antes del análisis.
Postcondición	Se dispone de un informe estadístico que facilita la comparación entre simuladores.	

Tabla 3.7: Caso de uso 7: Análisis estadístico de los datos

3.2.2. Diseño

Descripción Detallada

Con el objetivo de realizar una comparación rigurosa entre los simuladores Ignition Gazebo, Webots e Isaac Sim, se han definido una serie de escenarios controlados

que combinan distintas condiciones de ejecución. Estas condiciones permiten analizar cómo afectan factores como el uso de interfaz gráfica (GUI), la aceleración por GPU y el estado de carga del ordenador al rendimiento y consumo de recursos durante la simulación.

Combinaciones evaluadas

Para los simuladores Ignition Gazebo y Webots, se han considerado todas las combinaciones posibles entre los siguientes tres factores:

- Estado de energía del portátil: enchufado a la corriente, cargando y al 100 % de batería (se denominará "cargando") o desenchufado de la corriente y descargando (se denominará "sin cargar").
- Presencia de interfaz gráfica (GUI): habilitada o deshabilitada.
- Uso de GPU: habilitada o deshabilitada.

Esto da lugar a un total de 8 escenarios por simulador, los cuales se resumen a continuación:

Tabla 3.8: Escenarios considerados por simulador

Escenario	Condición	GUI	GPU
1	Cargando	Sí	No
2	Cargando	Sí	Sí
3	Cargando	No	No
4	Cargando	No	Sí
5	Sin Cargar	Sí	No
6	Sin Cargar	Sí	Sí
7	Sin Cargar	No	No
8	Sin Cargar	No	Sí

En el caso de Isaac Sim, dado que requiere obligatoriamente el uso de GPU, se han evaluado únicamente los 4 escenarios posibles combinando estado de energía y uso de GUI:

Tabla 3.9: Escenarios considerados Isaac Sim

Escenario	Condición	GUI	GPU
1	Cargando	Sí	Sí
2	Cargando	No	Sí
3	Sin Cargar	Sí	Sí
4	Sin Cargar	No	Sí

Protocolo de ejecución

Cada uno de los escenarios ha sido evaluado mediante cinco simulaciones independientes para garantizar la consistencia de los resultados, más concretamente se han seguido los siguientes pasos:

1. Encendido del ordenador.
2. Abrir terminal.
3. Navegar al directorio indicado.
4. Ejecutar script de medición.
5. Apagado del ordenador.

En cada simulación, el robot correspondiente (TurtleBot o Carter, según el simulador) ejecuta una tarea de navegación autónoma entre waypoints utilizando el framework de navegación Nav2. Esta actividad permite obtener medidas representativas del uso real en aplicaciones robóticas.

Los datos se recopilan una vez por segundo durante la ejecución de cada simulación. Las métricas monitorizadas incluyen:

- Uso de CPU.
- Uso de RAM.
- Uso de GPU .
- Real Time Factor (RTF).
- Consumo energético.

Estas métricas serán explicadas con detalle en el apartado 3.2.2. La duración de cada simulación depende del estado de carga:

- Simulaciones con el ordenador cargando: 5 minutos.

- Simulaciones con el ordenador sin cargar: 8 minutos.

Este ajuste se debe a la forma en que `powerstat` gestiona el retardo inicial cuando se mide el consumo de energía en batería. En concreto, `powerstat` introduce un retardo de 180 segundos antes de comenzar a registrar datos, con el objetivo de estabilizar las lecturas de batería y reducir el ruido asociado al cambio de estado energético.

Métricas y datos recopilados

Durante la evaluación de los simuladores, se han monitorizado diversas métricas del sistema para analizar su rendimiento y consumo. A continuación, se detallan las métricas recopiladas:

- **CPU:**
 - `timestamp`: Marca de tiempo en formato Año-Mes-Día Horas:Minutos:Segundos.
 - `cpu_percent`: Porcentaje total de uso de CPU (en todos los núcleos).
 - `cpu_user_percent`: Porcentaje de CPU usado por procesos en espacio de usuario (aplicaciones).
 - `cpu_system_percent`: Porcentaje de CPU usado por el sistema (kernel y procesos del SO).
 - `cpu_idle_percent`: Porcentaje de CPU que está inactiva.
 - `cpu_freq_MHz`: Frecuencia actual de la CPU en megahercios.
 - `load_avg_1`: Promedio de carga del sistema en el último minuto.
 - `load_avg_5`: Promedio de carga del sistema en los últimos 5 minutos.
 - `load_avg_15`: Promedio de carga del sistema en los últimos 15 minutos.
 - `total_tasks`: Número total de procesos activos en el sistema.
 - `total_threads`: Número total de hilos en ejecución activos.
 - `running_tasks`: Número de tareas en ejecución.
- **GPU:**
 - `timestamp`: Marca de tiempo en formato Año-Mes-Día Horas:Minutos:Segundos.
 - `gpu_util_percent`: Porcentaje de utilización de la GPU.
 - `gpu_mem_used_MB`: Memoria utilizada de la GPU en megabytes.

- `gpu_mem_total_MB`: Memoria total de la GPU en megabytes.
- `gpu_power_W`: Consumo de energía de la GPU en vatios.
- `gpu_temp_C`: Temperatura de la GPU en grados Celsius.

■ **Batería (Powerstat):**

- `Time`: Marca de tiempo en formato Horas:Minutos:Segundos.
- `User`: Porcentaje de CPU dedicado a procesos en modo usuario.
- `Nice`: Porcentaje de CPU usado por procesos en usuario con prioridad "nice" (baja prioridad).
- `Sys`: Porcentaje de CPU en modo kernel.
- `Idle`: Porcentaje de CPU en modo inactivo.
- `IO`: Porcentaje de CPU en operaciones de entrada/salida.
- `Run`: Número promedio de procesos en ejecución o listos para ejecutarse.
- `Ctxt_s`: Cambios de contexto por segundo.
- `IRQ_s`: Número de interrupciones por segundo.
- `Watts`: Consumo de energía en vatios.
- `TCPU`: Valor leído de la zona RAPL asociada a la CPU, representa temperatura del paquete CPU.
- `x86_pk`: Energía consumida por el paquete CPU completo (package: núcleos + GPU integrada), medida vía RAPL.
- `SEN2`, `SEN3`, `SEN4`, `SEN5`: Lecturas de sensores ACPI etiquetados como `SENx`, reflejan valores de temperatura.
- `INT340`: Sensor ACPI con identificador `INT340`, proporciona temperatura.
- `TCPU_P`: Porcentaje total de temperatura correspondiente a la CPU.
- `acpitz`: Sensor correspondiente a una "zona térmica ACPI" (ACPI Thermal Zone), que monitorea la temperatura cerca del zócalo de la CPU (CPU socket).
- `CPU Freq`: Frecuencia de la CPU en mega hercios.

■ **RAM:**

- **timestamp**: Marca de tiempo en formato Año-Mes-Día Horas:Minutos:Segundos.
- **total_MB**: Memoria total en megabytes.
- **used_MB**: Memoria utilizada en megabytes.
- **available_MB**: Memoria disponible en megabytes.
- **percent**: Porcentaje de memoria utilizada.
- **free_MB**: Memoria libre en megabytes.
- **active_MB**: Memoria activa en megabytes.
- **inactive_MB**: Memoria inactiva en megabytes.
- **buffers_MB**: Memoria utilizada por buffers en megabytes.
- **cached_MB**: Memoria utilizada por cachés en megabytes.
- **swap_total_MB**: Memoria swap total en megabytes.
- **swap_used_MB**: Memoria swap utilizada en megabytes.
- **swap_free_MB**: Memoria swap libre en megabytes.

■ **Real Time Factor (RTF):**

- **timestamp**: Marca de tiempo en formato Año-Mes-Día Horas:Minutos:Segundos.
- **real_time_factor**: Factor de tiempo real, calculado como el tiempo simulado dividido por el tiempo real transcurrido.
- **iterations**: Número de iteraciones realizadas durante la simulación.

Siendo la lista anterior una descripción detallada de cada métrica, a continuación se describe de manera más general las métricas:

CPU Las métricas de la CPU proporcionan información sobre el uso del procesador, siendo estas métricas esenciales para evaluar la eficiencia del simulador en términos de procesamiento.

GPU Las métricas de la GPU ofrecen datos sobre la utilización de la unidad de procesamiento gráfico, su consumo energético y temperatura. Son fundamentales para analizar el rendimiento gráfico y energético del simulador.

Batería (Powerstat) Powerstat es una herramienta que mide el consumo de energía de un portátil utilizando información de la batería ACPI. Servirá para la comparación de gasto de batería entre simuladores.

RAM Las métricas de la memoria RAM son cruciales para evaluar la eficiencia del simulador en términos de uso de memoria.

Real Time Factor (RTF) El RTF indica qué tan rápido o lento está funcionando la simulación en comparación con el tiempo real. Un RTF mayor que 1 indica que la simulación está corriendo más rápido que el tiempo real, mientras que un RTF menor que 1 indica que la simulación está corriendo más lento que el tiempo real.

Automatización del proceso

La ejecución de cada uno de los escenarios, así como la recogida de métricas, ha sido completamente automatizada mediante un script en Python desarrollado específicamente para este propósito.

Este script se describe en detalle en el apartado 3.2.3, y permite lanzar las simulaciones, configurar los parámetros correspondientes, iniciar la monitorización y almacenar los datos recogidos en formato CSV para su posterior análisis.

3.2.3. Implementación

En la implementación se incluyen todos los archivos que han sido usados para realizar las mediciones en la simuladores, incluyendo el lanzamiento del simulador, la navegación entre waypoint, la medición de CPU, GPU, RAM, factor de tiempo real y batería y el archivo que hace uso de todas estas funcionalidades para la automatización completa de las mediciones.

Launcher de los simuladores

Para unificar el arranque de Webots, Gazebo e Isaac Sim, cada simulador dispone de una clase `SimulatorLauncher` con:

- **Constructor:** recibe un `mode` (CPU, GPU y GUI) y carga un diccionario de comandos (`sim_cmd_map`).
- **launch():**
 1. Verifica el modo.
 2. Inicia el proceso con `subprocess.Popen(..., preexec_fn=os.setsid)`.
 3. Lanza un hilo que comprueba la aparición del topic `/initialpose`.

- **wait_until_ready():** bloquea hasta que el topic esté disponible (o expire el timeout).
- **is_running():** devuelve si el proceso sigue activo.
- **stop():** termina el proceso enviando primero **SIGTERM** y, si es necesario, **SIGKILL**, y limpia el evento de ready.

Con esta implementación compartida, se puede definir el mapa de comandos específico de cada simulador para disponer de un lanzador uniforme y reutilizable.

Control de navegación

Para gestionar la navegación en Webots y Gazebo se ha creado una clase `NavigationController` con la siguiente interfaz común:

- **Constructor:**
 - Inicializa ROS 2 (si no está ya activo) y crea un `TurtleBot4Navigator`.
 - Prepara variables internas para el hilo de navegación y la señal de parada.
- **Inicialización:**
 - `waitNav2()` (Webots) espera a que Nav2 esté activo.
 - `setInitialPose()` (Gazebo) fija la pose inicial antes de llamar a `waitUntilNav2Active()`.
- **start_navigation():**
 - Lanza un hilo daemon que recorre una lista de waypoints predefinidos.
 - Para cada waypoint:
 1. Obtiene un `PoseStamped` con coordenadas y dirección.
 2. Invoca `goToPose()` y espera a `isTaskComplete()`.
 3. Permite abortar la navegación si se solicita (`stopRequested`).
- **stop_navigation():**
 - Señala la parada y cancela la tarea con `cancelTask()`.
 - Espera unos segundos a que el hilo termine limpiamente.

Isaac Sim: el ejemplo oficial ya contiene la lógica de navegación entre waypoints, por lo que no es necesario implementar un controlador adicional.

Medición de métricas

Para uniformizar la recolección de datos en Webots, Gazebo e Isaac Sim se ha definido un módulo `MetricsCollector` que agrupa varios *collectors* específicos:

- **MetricsCollector:**

- Crea una carpeta de salida y mantiene un listado de colectores.
- `start_all(duration, power_warmup)` instancia y lanza en hilos:
 - `CPUCollector`, `RAMCollector`, `GPUCollector`, `PowerCollector`.
 - Devuelve el `PowerCollector` para esperar su *warmup*.
- `stop_all()` detiene todos los hilos y cierra archivos.

- **BaseCollector:** plantilla que gestiona:

- `setup()`: inicializa CSV y recursos.
- `collect_data()`: recoge una muestra periódica.
- `cleanup()`: cierra ficheros y procesos.

- **CPUCollector & RAMCollector:** usan `psutil` para volcar cada segundo:

- *CPU*
- *RAM*

- **GPUCollector:** ejecuta `nvidia-smi` para obtener datos de la GPU.

- **RTFCollector:**

- *Webots/Isaac Sim*: lanza el nodo `rtf_monitor` de ROS 2 y parsea su salida.
- *Ignition Gazebo*: suscribe a `/stats` con `ign topic` y extrae *real_time_factor* e iteraciones.

- **PowerCollector:**

- Ejecuta `powerstat` con intervalo y duración configurables.
- Espera un retardo de *warmup* antes de lanzar el simulador.
- Registra en un log y expone `wait_for_warmup()` para sincronización.

Con esta estructura, todos los simuladores generan ficheros CSV homogéneos que facilitan el análisis posterior de CPU, memoria, GPU, factor de tiempo real y consumo energético.

Integración y automatización completa

El script principal agrupa los componentes de lanzamiento, navegación y medición en un flujo único:

1. **Parseo de argumentos:** Se definen opciones para carpeta de salida, modo de simulador, duración, intervalo, warmup de PowerStat y flag `-no-power`.
2. **Manejo de señales:** Se captura `SIGINT` y `SIGTERM` para invocar una función `cleanup()` que detiene todos los procesos y hilos.
3. **Inicialización de componentes:**
 - `MetricsCollector` (y opcionalmente `PowerCollector` para warmup).
 - `SimulatorLauncher` con el modo seleccionado.
 - `NavigationController` (solo en Webots/Gazebo).
4. **Warmup de PowerStat:** Si no se desactiva, se espera al evento de `power_collector` `-.wait_for_warmup()` antes de seguir.
5. **Inicio de métricas y simulador:**
 - `metrics.start_all()` arranca CPU, RAM, GPU y, si procede, PowerStat.
 - `simulator.launch()` inicia el simulador; `wait_until_ready()` espera al tópico `/initialpose`.
6. **Recolección de RTF:** Se instancia `RTFCollector` y se lanza en un hilo para volcar factor de tiempo real.
7. **Navegación (Webots/Gazebo):**
 - Se espera a Nav2 (`waitNav2()` o `setInitialPose() + waitUntilNav2Active()`).
 - Tras un breve retardo, se llama `start_navigation()` para recorrer waypoints.
8. **Espera de finalización:** El script calcula el tiempo restante según la duración definida (excluyendo warmup) y duerme hasta completar la simulación.

9. **Limpieza final:** `cleanup()` detiene RTF, navegación, simulador y métricas; opcionalmente cierra ROS 2 con `rclpy.shutdown()`.

De este modo, con un único comando se levanta el entorno, se sincroniza el muestreo de energía, se monitoriza CPU, RAM, GPU, RTF y se ejecuta la navegación automatizada, generando un conjunto homogéneo de CSVs y logs para su análisis.

Conversión de `powerstat.log` a CSV

Para transformar el fichero de salida de `powerstat` en un CSV legible, se dispone del script `powerstat_to_csv.py`, que sigue este flujo:

1. **Parseo del log (`parse_powerstat_log`):**
 - Lee línea a línea hasta encontrar la cabecera (contiene “Time”, “User”, “Nice”, “Sys”, etc.).
 - A partir de la primera línea con marca temporal (HH:MM:SS) recoge filas de datos.
 - Finaliza al detectar la sección de resumen (línea de guiones).
2. **Guardado en CSV (`save_to_csv`):**
 - Crea un archivo `.csv` con el mismo nombre base que el log.
 - Escribe la cabecera y las filas de datos extraídas.
3. **Proceso principal (`main`):**
 - Acepta uno o varios ficheros `.log` como argumentos.
 - Para cada fichero:
 - Verifica su existencia.
 - Llama a `process_file`, que aplica el parseo y guardado.
 - Informa del número de archivos procesados correctamente.

Con ello se facilita el análisis posterior de consumo energético en herramientas de hojas de cálculo o scripts de procesamiento de datos.

Unión de CSV

Se han desarrollado múltiples ficheros para la unión de las mediciones que permitan su análisis en JASP. Concretamente se han desarrollado 4:

- **Unión mediciones Ignition:** usado para unir todas las mediciones del simulador Ignition Gazebo, añadiendo información para ayudar en el análisis como columnas para uso GUI, uso de GPU y estado de la batería.
- **Unión mediciones Webots:** usado para unir todas las mediciones del simulador Webots, añadiendo información para ayudar en el análisis como columnas para uso GUI, uso de GPU y estado de la batería.
- **Unión mediciones Isaac Sim:** usado para unir todas las mediciones del simulador Isaac Sim, añadiendo información para ayudar en el análisis como columnas para uso GUI y estado de la batería.
- **Unión de todas las mediciones:** usado para unir todas las mediciones de los tres simuladores, añadiendo información para ayudar en el análisis como columnas para escenario y simulador.

3.3. El producto del desarrollo

El resultado es una estructura de carpetas y ficheros organizada según escenario, modo de simulación y repetición:

- **Escenarios:**

Tabla 3.10: Matriz de escenarios y configuraciones evaluadas en los simuladores

Simulador	Escenario	GUI / SIN GPU	GUI / GPU	SIN GUI / SIN GPU	SIN GUI / GPU
Webots & Ignition Gazebo	Cargando	X	X	X	X
	Sin Cargar	X	X	X	X
Isaac Sim	Cargando (solo GPU)		X		X
	Sin Cargar (solo GPU)		X		X

- **Repeticiones:** para cada uno de los modos anteriores se realizan 5 ejecuciones independientes.
 - Se crea una carpeta por repetición, nombrada con `YYYYMMDD_HHMMSS_repN` o con un nombre personalizable, para mayor legibilidad se optó por representar en el nombre el escenario aplicado más el número de repetición.
- **Contenido de cada carpeta de repetición:**
 - `powerstat.log`
 - `powerstat.csv`
 - `cpu_usage.csv`

- `gpu_usage.csv`
 - `ram_usage.csv`
 - `rtf.csv`
- **Formato de los CSV generados:** El contenido y las cabeceras de cada uno de los ficheros CSV (`powerstat.csv`, `cpu_usage.csv`, `gpu_usage.csv`, `ram_usage.csv`, `rtf.csv`) ya se especifica en el apartado 3.2.2.

Capítulo 4

Evaluación

En este capítulo, se exponen los resultados de la solución elaborada. Esto incluye la evaluación de los datos obtenidos, su análisis y además un ejemplo que demuestre los resultados del mejor de los simuladores en cuanto a recursos computacionales.

Recordar que hay 2 estados de energía del portátil: enchufado a la corriente, cargando y al 100 % de batería (se denominará "cargando") o desenchufado de la corriente y descargando (se denominará "sin cargar").

4.1. Proceso de evaluación (Análisis Individual de Simuladores)

Primero, se expondrán los resultados elaborados a partir de los datos obtenidos para cada simulador. Estos son los datos de CPU, GPU, RAM, consumo de batería y factor de tiempo real para observar como varía su comportamiento respecto a los distintos escenarios, es decir, la diferencia entre que esté cargando el ordenador o sin cargar, esté usando GUI o no y esté usando GPU o no. Los datos al completo se encuentran en la sección B.1.

4.1.1. Ignition Gazebo

Comparación Ignition Gazebo respecto a estados de batería

El estado de carga de la batería muestra un impacto significativo en el rendimiento de Ignition Gazebo. En términos de CPU, se observa un mayor uso porcentual cuando el sistema no está cargando (*cpu_percent* medio de 39.7% sin cargar vs

35.4 % cargando), con frecuencias de CPU notablemente más bajas (2.39 GHz vs 3.63 GHz), lo que sugiere que el sistema limita el rendimiento para conservar energía.

La GPU presenta comportamientos diferenciados: mientras el uso de memoria se mantiene similar en ambos casos (172-174MB), la potencia consumida es mayor cuando el portátil está cargando (8.4 W vs 6.8 W), al igual que la temperatura (51.6°C vs 45.6°C), indicando un mayor rendimiento gráfico cuando hay alimentación externa.

En cuanto a la RAM, las diferencias son menos pronunciadas, aunque se aprecia un ligero mayor uso de memoria activa cuando el sistema está cargando (3845 MB vs 3823 MB), posiblemente relacionado con el mejor rendimiento general del sistema en este estado.

El aspecto más crítico se observa en el *Real Time Factor* (RTF), donde el rendimiento es significativamente mejor cuando el sistema está cargando (mediana de 0.663 vs 0.467), lo que indica que la simulación avanza más rápido respecto al tiempo real con alimentación externa. Esto se correlaciona con el mayor número de iteraciones completadas (32102 vs 21970 de media), confirmando que el estado de carga afecta directamente a la eficiencia de la simulación.

Estos resultados demuestran que para obtener el máximo rendimiento en Ignition Gazebo, especialmente en simulaciones largas o complejas, es recomendable utilizar el sistema cargando. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.1.

Comparación Ignition Gazebo respecto a GUI

El uso de la interfaz gráfica (GUI) en Ignition Gazebo muestra un impacto significativo en todos los aspectos del rendimiento del sistema. En cuanto al procesador, se observa un aumento del 53.6 % en el uso medio de CPU (de 29.6 % sin GUI a 45.6 % con GUI) y un incremento del 57.9 % en el porcentaje de uso en espacio de usuario. Paradójicamente, la frecuencia media del CPU disminuye ligeramente (de 3.05 GHz a 2.97 GHz), lo que sugiere que la carga gráfica adicional limita el escalado dinámico de frecuencia.

La GPU experimenta cambios notables: el uso medio de memoria se multiplica por 2.15 (de 109.5 MB a 235.9 MB), la potencia consumida aumenta en un 37.7 % (de 6.4 W a 8.8 W) y la temperatura sube 1.8°C en promedio. Estos valores confirman el mayor trabajo de renderizado requerido cuando se activa la interfaz gráfica.

En cuanto a la memoria RAM, el uso de memoria activa crece un 12.9 % (de 3602 MB a 4066 MB) y la memoria caché aumenta un 6.7 %, reflejando la mayor demanda de recursos del sistema para mantener la interfaz visual.

El impacto en el rendimiento de la simulación es particularmente relevante: el Real Time Factor (RTF) disminuye un 17.1 % (de 0.620 a 0.514) y el número medio de iteraciones completadas se reduce en un 18 %, indicando que la simulación avanza más lentamente respecto al tiempo real cuando se utiliza la GUI.

Finalmente, el consumo energético se ve notablemente afectado, con un aumento del 15.5 % en el consumo medio (de 39.97 W a 46.18 W) y temperaturas más elevadas tanto en CPU (+1.4°C) como en el paquete x86 (+1.5°C). Estos resultados demuestran que, aunque la GUI proporciona visualización interactiva, implica compromisos significativos en rendimiento y eficiencia energética. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.1.

Comparación Ignition Gazebo respecto a GPU

El uso de GPU en Ignition Gazebo presenta un impacto significativo en el rendimiento del sistema, mostrando tanto ventajas como desventajas según los componentes analizados. En el procesador, se observa un incremento moderado del 4.3 % en el uso de CPU (de 36.8 % a 38.4 %) y del 3.2 % en el uso en espacio de usuario, acompañado de un ligero aumento del 5.1 % en la frecuencia media del CPU (de 2.93 GHz a 3.09 GHz), indicando una mayor carga computacional cuando se activa la GPU.

El impacto más notable se evidencia en los propios recursos de la GPU: el uso de memoria pasa de un mínimo constante de 15 MB (sin GPU) a un promedio de 330.5 MB (con GPU), mientras la potencia consumida se multiplica por 4.9 (de 2.6 W a 12.6 W) y la temperatura aumenta en 3.4°C. Estos valores reflejan el intenso trabajo gráfico delegado al acelerador hardware.

En cuanto a memoria RAM, el uso activo aumenta un 21.1 % (de 3431 MB a 4156 MB), aunque curiosamente la memoria caché disminuye un 14.9 %, sugiriendo un patrón de acceso a memoria diferente cuando se emplea aceleración gráfica.

El beneficio más relevante aparece en el rendimiento de simulación: el Real Time Factor (RTF) mejora en un 29.1 % (de 0.495 a 0.639) y el número de iteraciones completadas aumenta en un 30.8 %, demostrando que la GPU acelera significativamente la ejecución de la simulación.

Este mayor rendimiento tiene su costo energético: el consumo medio se incrementa en un 34 % (de 36.8 W a 49.3 W) y las temperaturas del paquete x86 suben 6.4°C. Estos resultados indican que, aunque el uso de GPU mejora notablemente el rendimiento de la simulación, implica mayores demandas energéticas y térmicas que deben considerarse según los requisitos de cada aplicación. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.1.

4.1.2. Webots

Comparación Webots respecto a estados de batería

El estado de carga de la batería muestra un impacto moderado pero consistente en el rendimiento de Webots. En el procesador, se observa un incremento del 25.6 % en el uso medio de CPU cuando el sistema no está cargando (6.16 % vs 4.91 %), acompañado de una reducción del 21.2 % en la frecuencia media del procesador (1.60 GHz vs 2.03 GHz), lo que indica que el sistema limita el rendimiento para conservar energía durante la operación sin cargar.

La GPU presenta comportamientos interesantes: mientras el uso de memoria se mantiene prácticamente idéntico (304 MB en ambos casos), la potencia consumida es un 51.2 % mayor cuando el portátil está cargando (4.81 W vs 3.18 W), aunque las temperaturas solo muestran una diferencia mínima (43.0°C vs 41.3°C). Esto sugiere que Webots mantiene un uso similar de recursos gráficos independientemente del estado de carga.

El uso de memoria RAM muestra patrones consistentes entre ambos modos, con diferencias menores al 1 % en los valores medios para memoria activa, buffers y caché. Sin embargo, se aprecia una mayor variabilidad (desviación estándar) en los buffers cuando el sistema está cargando, posiblemente relacionado con estrategias diferentes de gestión de energía.

El aspecto más destacable es la mínima diferencia en el Real Time Factor (RTF), con valores medianos idénticos (0.956) y una diferencia media de solo 0.002 a favor del modo cargando. Las iteraciones completadas también muestran variaciones insignificantes (<0.13 %). Esto indica que Webots mantiene una eficiencia de simulación consistente independientemente del estado de carga de la batería.

Estos resultados demuestran que, aunque el estado de carga afecta el rendimiento del CPU y el consumo energético de la GPU, Webots es capaz de mantener una eficiencia de simulación estable en ambos casos, lo que lo hace particularmente ade-

cuado para uso móvil cuando la precisión temporal es crítica. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.2.

Comparación Webots respecto a GUI

El uso de la interfaz gráfica (GUI) en Webots muestra un impacto moderado pero sistemático en el rendimiento del sistema. En el procesador, se observa un incremento del 10.3 % en el uso medio de CPU (5.80 % con GUI vs 5.26 % sin GUI) y del 9.2 % en el uso en espacio de usuario, acompañado de un ligero aumento del 1.3 % en la frecuencia media del CPU (1.83 GHz vs 1.81 GHz). Estos valores sugieren que la GUI añade una carga computacional adicional, aunque relativamente pequeña.

El comportamiento de la GPU revela patrones contrastantes: mientras el uso medio de memoria es un 32.3 % menor con GUI (245.7 MB vs 363.0 MB), la potencia consumida aumenta en un 40.6 % (4.67 W vs 3.32 W) y la temperatura sube 1.75 °C. Esta aparente contradicción podría deberse a diferentes estrategias de renderizado o a una mayor eficiencia en el manejo de recursos gráficos cuando se usa la interfaz visual.

En cuanto a memoria RAM, las diferencias son mínimas ($<0.5\%$) en el uso de memoria activa, aunque se aprecia una mayor variabilidad en los buffers cuando no se utiliza GUI (desviación estándar de 67.1 MB vs 6.5 MB). La memoria caché muestra un comportamiento similar en ambos casos.

El Real Time Factor (RTF) presenta una diferencia mínima pero consistente (0.932 con GUI vs 0.928 sin GUI), con un incremento del 0.4 % en la mediana, mientras que las iteraciones completadas aumentan un 0.36 %. Estos valores indican que la GUI no perjudica significativamente el rendimiento de la simulación.

El consumo energético refleja un aumento del 8.1 % (30.37 W vs 28.09 W) cuando se activa la GUI, con temperaturas del paquete x86 2.3 °C más altas. Estos resultados demuestran que, aunque la interfaz gráfica en Webots introduce una sobrecarga moderada en términos de consumo energético y uso de CPU, su impacto en el rendimiento de simulación es mínimo, manteniendo su eficiencia temporal incluso con la interfaz visual activada. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.2.

Comparación Webots respecto a GPU

El uso de GPU en Webots muestra un impacto significativo en el consumo de recursos del sistema, aunque con efectos diferenciados en cada componente. En el

procesador, se observa una reducción mínima del 2.4 % en el uso medio de CPU (5.46 % con GPU vs 5.60 % sin GPU) y del 4.3 % en el uso en espacio de usuario, acompañado de una ligera disminución del 2.5 % en la frecuencia media del CPU (1.80 GHz vs 1.84 GHz). Estos valores sugieren que la GPU asume parte de la carga computacional, permitiendo un menor trabajo del procesador.

El análisis de la GPU revela cambios drásticos: el uso de memoria pasa de un mínimo constante de 15 MB (sin GPU) a un promedio de 594.7 MB (con GPU), mientras la potencia consumida se multiplica por 1.9 (5.23 W vs 2.77 W) y la temperatura aumenta 1.0°C. Estos resultados confirman la activación efectiva del hardware gráfico cuando se utiliza esta opción.

En cuanto a memoria RAM, el uso activo aumenta un 9.1 % (3048.8 MB vs 2794.2 MB) con GPU, mientras la memoria caché disminuye un 16.3 %, indicando un patrón distinto de gestión de memoria cuando se emplea aceleración gráfica.

El Real Time Factor (RTF) muestra una diferencia mínima pero consistente (0.927 con GPU vs 0.933 sin GPU), con una reducción del 0.6 % en la mediana. Las iteraciones completadas presentan una variación insignificante (<0.22 %), demostrando que la GPU no mejora sustancialmente el rendimiento de simulación en Webots.

El consumo energético aumenta un 7.9 % (30.34 W vs 28.12 W) con GPU activa, aunque las temperaturas del paquete x86 muestran solo un incremento de 0.6°C. Estos resultados indican que, aunque el uso de GPU en Webots incrementa el consumo de recursos gráficos y energía, su impacto en el rendimiento de simulación es marginal, sugiriendo que la implementación actual podría no estar optimizada para aprovechar completamente el hardware gráfico disponible. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.2.

4.1.3. Isaac Sim

Comparación Isaac Sim respecto a estados de batería

El estado de carga de la batería muestra un impacto significativo en el rendimiento de Isaac Sim, particularmente en el uso de recursos gráficos y eficiencia energética. En el procesador, se observa un incremento del 11.3 % en el uso medio de CPU cuando opera sin carga (19.43 % vs 17.46 %), acompañado de una reducción del 18.9 % en la frecuencia media del procesador (2.01 GHz vs 2.48 GHz).

La GPU presenta diferencias notables: mientras el uso de memoria se mantiene alto en ambos casos ($\approx 4.2GB$), la potencia consumida disminuye un 45.1 % (29.43 W vs 53.60 W) y la temperatura media baja 11.2°C (52.04°C vs 63.28°C) en modo batería. Estos valores reflejan una estrategia agresiva de conservación energética en el hardware gráfico.

El uso de memoria RAM activa es un 4.7 % menor sin carga (6.82 GB vs 7.16 GB), mientras la memoria caché disminuye un 4 % (5.25 GB vs 5.47 GB), sugiriendo una gestión más conservadora de recursos en modo batería.

El aspecto más crítico se observa en el Real Time Factor (RTF), que cae un 19.4 % (0.137 vs 0.170) cuando el sistema opera sin carga eléctrica, con un 21 % menos de iteraciones completadas. Esta reducción en el rendimiento de simulación está directamente correlacionada con las limitaciones energéticas.

Estos resultados demuestran que Isaac Sim es particularmente sensible al estado de carga, mostrando un equilibrio claro entre conservación energética y rendimiento. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.3.

Comparación Isaac Sim respecto a GUI

El uso de la interfaz gráfica (GUI) en Isaac Sim muestra un impacto moderado pero significativo en el rendimiento del sistema. En términos de CPU, se observa un ligero incremento del 2.2 % en el uso medio (18.64 % con GUI vs 18.24 % sin GUI) y del 3.3 % en el uso en espacio de usuario, mientras que la frecuencia del procesador se mantiene prácticamente igual (2.23 GHz vs 2.27 GHz), indicando que la carga adicional de la GUI es manejada eficientemente por el procesador.

La GPU presenta comportamientos diferenciados: aunque el uso de memoria es similar ($\approx 4.2GB$ en ambos casos), la potencia consumida disminuye un 5.8 % (40.36 W vs 42.82 W) con GUI activa, mientras que la temperatura media se reduce 1.7°C (56.83°C vs 58.56°C). Estos valores sugieren optimizaciones en el renderizado gráfico cuando se utiliza la interfaz visual.

El uso de memoria RAM muestra diferencias menores al 2 % en los valores medios para memoria activa y caché, aunque se aprecia una mayor variabilidad en los buffers cuando no se utiliza GUI. Esto indica que la GUI no representa una carga significativa para el sistema de memoria.

El aspecto más relevante aparece en el Real Time Factor (RTF), donde se observa una reducción del 8.1 % (0.148 vs 0.161) al activar la GUI, con un 8.5 % menos de iteraciones completadas. Esta diferencia, aunque moderada, demuestra que la interfaz gráfica introduce cierta sobrecarga en el rendimiento de simulación.

El consumo energético presenta una disminución del 8 % (58.56 W vs 63.64 W) con GUI activa, contradiciendo la tendencia esperada. Este comportamiento atípico podría deberse a optimizaciones específicas de Isaac Sim en el manejo de recursos gráficos.

En conclusión, aunque la GUI en Isaac Sim introduce una ligera reducción en el RTF, su impacto general en el sistema es moderado, manteniendo un buen equilibrio entre funcionalidad visual y rendimiento. El desglose completo de los datos que respaldan este análisis se encuentra en el apartado B.1.3.

4.2. Proceso de evaluación (Comparativa entre Simuladores)

Una vez comparados los simuladores individualmente, se expondrán los datos que servirán para la comparación entre simuladores en las mismas condiciones para determinar cual de ellos tiene mejores prestaciones respecto al uso de recursos computacionales. Los datos al completo se encuentran en la sección B.2.

4.2.1. Cargando - GUI - No GPU

La comparación entre Ignition Gazebo y Webots en modo GUI sin aceleración GPU revela diferencias significativas en el rendimiento y uso de recursos. Ignition muestra un uso de CPU notablemente superior (42.58 % vs 5.27 % en Webots), con una frecuencia media de procesador un 72.8 % más alta (3.68 GHz vs 2.13 GHz), indicando una mayor demanda computacional para su simulación.

En cuanto a memoria RAM, Ignition utiliza un 41.8 % más de memoria activa (3.96 GB vs 2.79 GB) y un 4.1 % más de memoria caché, reflejando su arquitectura más pesada. La carga media del sistema (`load_avg_5`) es 8.1 veces mayor en Ignition (5.53 vs 0.68), confirmando su mayor impacto en el sistema completo.

El rendimiento de simulación (RTF) muestra patrones opuestos: mientras Webots alcanza un excelente RTF medio de 0.944 (cercano a tiempo real), Ignition se mantiene en 0.546, con solo la mitad de eficiencia temporal.

Estos resultados demuestran que Webots ofrece un rendimiento más eficiente en escenarios sin GPU, con menor consumo de recursos y mejor RTF, mientras que Ignition requiere más capacidad de procesamiento. Los datos al completo se encuentran en la sección B.2.1.

4.2.2. Cargando - GUI - GPU

La comparación entre Ignition Gazebo, Isaac Sim y Webots en modo GUI con aceleración GPU revela diferencias significativas en el uso de recursos y rendimiento. Ignition muestra un uso más alto de CPU (42.66 %) frente a Isaac Sim (17.94 %) y Webots (4.91 %), con frecuencias de CPU más altas (3.43 GHz vs 2.48 GHz y 1.97 GHz respectivamente), indicando distintos enfoques en el balance carga computacional entre CPU y GPU.

El uso de GPU presenta contrastes marcados: Isaac Sim demanda casi el máximo de su capacidad (90.69 % de utilización media) con un consumo energético elevado (53.76 W), mientras Ignition y Webots muestran utilizaciones más moderadas (37.88 % y 33.02 %) con consumos significativamente menores (15.53 W y 7.11 W). Isaac Sim también destaca por su alto uso de memoria GPU (3.86 GB promedio), muy superior a Ignition (453 MB) y Webots (476 MB).

En memoria RAM, Isaac Sim requiere considerablemente más recursos (7.21 GB activos) comparado con Ignition (4.17 GB) y Webots (3.08 GB), reflejando su naturaleza más compleja. El rendimiento de simulación (RTF) muestra que Webots alcanza el mejor desempeño (0.926), seguido de Ignition (0.654) y finalmente Isaac Sim (0.170), evidenciando diferencias fundamentales en sus arquitecturas.

Estos resultados demuestran que Webots ofrece el mejor equilibrio entre uso de recursos y rendimiento, Ignition proporciona un balance aceptable, mientras Isaac Sim requiere significativamente más recursos hardware para lograr su funcionalidad avanzada. Los datos al completo se encuentran en la sección B.2.2.

4.2.3. Cargando - No GUI - No GPU

La comparación entre Ignition Gazebo y Webots en modo sin GUI y sin GPU revela diferencias significativas en eficiencia y uso de recursos. Ignition muestra un mayor uso de CPU (26.27 % vs 4.67 % en Webots) con frecuencias de procesador un 78 % más altas (3.60 GHz vs 2.02 GHz), indicando una mayor demanda computacional para sus simulaciones.

En cuanto a memoria RAM, ambos simuladores presentan consumos similares de memoria activa ($\approx 3.5GB$) y caché ($\approx 3.0GB$), sugiriendo que la ausencia de interfaz gráfica iguala sustancialmente sus requerimientos de memoria. Sin embargo, Ignition mantiene una carga media del sistema (`load_avg_5`) 4.4 veces mayor (2.60 vs 0.59), reflejando su arquitectura más pesada.

El rendimiento de simulación (RTF) muestra patrones contrastantes: Webots alcanza un excelente RTF medio de 0.930 (cercano a tiempo real), mientras Ignition se mantiene en 0.642. Esta notable diferencia del 44.9 % a favor de Webots demuestra su mayor eficiencia en escenarios sin aceleración gráfica.

Estos resultados destacan que Webots ofrece un rendimiento significativamente mejor en configuraciones mínimas, con menor impacto en el sistema y mayor eficiencia de simulación. Ignition, aunque más demandante, podría preferirse cuando se requieran características específicas de su motor de física, aceptando el mayor consumo de recursos. Los datos al completo se encuentran en la sección B.2.3.

4.2.4. Cargando - No GUI - GPU

La comparación entre Ignition Gazebo, Isaac Sim y Webots en modo sin GUI pero con GPU revela diferencias significativas en rendimiento y uso de recursos. Ignition muestra un uso alto de CPU (30.22 %) comparado con Isaac Sim (16.98 %) y Webots (4.79 %), y con frecuencias de CPU notablemente más altas (3.79 GHz vs 2.48 GHz y 2.01 GHz respectivamente), indicando distintos enfoques en la distribución de carga entre CPU y GPU.

El uso de GPU presenta contrastes marcados: Isaac Sim utiliza casi al máximo su capacidad (89.09 % de utilización media) con un alto consumo energético (53.44 W), mientras Ignition y Webots muestran utilizaciones más moderadas (33.84 % y 18.21 %) con consumos significativamente menores (11.17 W y 4.75 W). Isaac Sim también destaca por su elevado uso de memoria GPU (3.82 GB promedio), muy superior a Ignition (204 MB) y Webots (709 MB).

En memoria RAM, Isaac Sim requiere considerablemente más recursos (7.10 GB activos) comparado con Ignition (3.68 GB) y Webots (3.00 GB). El rendimiento de simulación (RTF) muestra que Webots alcanza el mejor desempeño (0.923), seguido de Ignition (0.812) y finalmente Isaac Sim (0.171), evidenciando diferencias fundamentales en sus arquitecturas.

Estos resultados demuestran que Webots ofrece el mejor equilibrio entre uso de recursos y rendimiento en esta configuración, Ignition proporciona un balance acep-

table con mayor carga en CPU, mientras Isaac Sim requiere significativamente más recursos hardware para lograr su funcionalidad. Los datos al completo se encuentran en la sección B.2.4.

4.2.5. Sin Cargar - GUI - No GPU

La comparación entre Ignition Gazebo y Webots en modo GUI sin GPU y sin cargar la batería revela diferencias notables en eficiencia y consumo de recursos. Ignition muestra un uso de CPU significativamente mayor (49.27 % vs 6.66 % en Webots) con frecuencias de procesador un 36 % más altas (2.20 GHz vs 1.62 GHz), evidenciando su mayor demanda computacional en condiciones de energía limitada.

En memoria RAM, Ignition utiliza un 42.6 % más de memoria activa (3.98 GB vs 2.79 GB) y un 5.9 % más de memoria caché, manteniendo una carga media del sistema (load_avg_5) 7.3 veces superior (4.94 vs 0.68). Estos valores confirman el mayor impacto de Ignition en el sistema.

El rendimiento de simulación (RTF) presenta una diferencia dramática: Webots alcanza un RTF medio de 0.931 (cercano a tiempo real), mientras Ignition solo logra 0.353, con una eficiencia 2.6 veces menor. Esta brecha se acentúa en modo batería, donde Webots demuestra mejor capacidad de adaptación a restricciones energéticas.

El consumo energético es un 34.4 % mayor en Ignition (39.31 W vs 29.25 W), con temperaturas de CPU consistentemente más elevadas (51.55°C vs 48.53°C). Estos resultados demuestran que Webots ofrece un rendimiento notablemente más eficiente en escenarios móviles sin GPU, optimizando mejor los recursos limitados de batería. Los datos al completo se encuentran en la sección B.2.5.

4.2.6. Sin Cargar - GUI - GPU

La comparación entre Ignition, Isaac Sim y Webots en modo sin carga, con GUI y utilizando GPU, revela patrones de rendimiento y eficiencia marcadamente diferentes. Ignition muestra el mayor uso de CPU (47.96 % de media) frente a Isaac Sim (19.32 %) y Webots (6.17 %), con frecuencias de CPU consistentemente más altas (2.55 GHz vs 1.97 GHz y 1.60 GHz respectivamente), indicando una mayor demanda computacional.

En cuanto a utilización de GPU, Isaac Sim presenta el comportamiento más extremo, con utilización media del 72.46 % pero alcanzando frecuentemente el 100 %, junto con un consumo de memoria GPU significativamente mayor (3.12 GB vs 460

MB en Ignition y 478 MB en Webots). Esto contrasta con su bajo RTF medio (0.120), sugiriendo ineficiencias. Webots demuestra la mejor eficiencia energética en GPU, con solo 6.26 W de consumo frente a 14.13 W de Ignition y 26.84 W de Isaac Sim.

El análisis de memoria RAM revela que Isaac Sim utiliza casi el doble de memoria activa (6.67 GB) comparado con Ignition (4.17 GB) y Webots (3.05 GB), mientras que Webots mantiene el uso más bajo y estable en todos los parámetros de memoria. En rendimiento de simulación (RTF), Webots lidera claramente con 0.929 (cerca de tiempo real), seguido por Ignition (0.500) e Isaac Sim (0.120), mostrando una diferencia del 86 % entre el mejor y peor desempeño.

El consumo energético total refleja estas diferencias: Isaac Sim tiene el mayor consumo medio (58.56 W), seguido por Ignition (53.05 W) y Webots (31.49 W), con Webots siendo un 46 % más eficiente que Ignition y un 82 % más que Isaac Sim. Las temperaturas de CPU coinciden con este patrón, siendo Isaac Sim el más caliente (58.19°C) y Webots el más frío (48.60°C).

Estos resultados demuestran que Webots ofrece el mejor equilibrio entre rendimiento y eficiencia en esta configuración, mientras Isaac Sim, a pesar de su alta utilización de recursos, muestra el peor rendimiento en RTF. Ignition se posiciona como una opción intermedia, con mayor consumo que Webots pero mejor RTF que Isaac Sim. Los datos al completo se encuentran en la sección B.2.6.

4.2.7. Sin Cargar - No GUI - No GPU

La comparación entre Ignition y Webots en modo sin carga, sin GUI y sin GPU revela diferencias significativas en eficiencia y rendimiento. Ignition muestra un uso de CPU notablemente superior (29.29 % de media) frente al 5.60 % de Webots, con frecuencias de procesador un 40 % más altas (2.24 GHz vs 1.60 GHz), evidenciando su mayor demanda computacional incluso en esta configuración mínima.

En cuanto a memoria RAM, Ignition utiliza un 25 % más de memoria activa (3.52 GB vs 2.82 GB) y mantiene patrones de uso consistentemente más altos en todos los parámetros, aunque Webots muestra una ligera ventaja en memoria caché (3.31 GB vs 3.04 GB). La carga media del sistema (load_avg_5) es 5.9 veces mayor en Ignition (3.63 vs 0.62), confirmando su mayor impacto global en el sistema incluso sin interfaz gráfica.

El rendimiento de simulación (RTF) muestra una diferencia crucial: Webots alcanza un RTF medio de 0.928 (próximo a tiempo real), mientras Ignition se mantiene

en 0.436, con solo el 47 % de la eficiencia temporal de Webots. Esta brecha del 53 % en rendimiento demuestra la mejor optimización de Webots para operación en modo headless.

El consumo energético es un 27 % mayor en Ignition (34.32 W vs 26.99 W), con temperaturas de CPU consistentemente más elevadas (50.27°C vs 45.66°C). Los datos de potencia del paquete x86 (x86_pk) corroboran este patrón, mostrando valores medios más altos para Ignition (50.45W vs 46.02W).

Estos resultados demuestran que Webots ofrece un rendimiento significativamente más eficiente en configuraciones sin GUI ni GPU, con menor consumo energético (diferencia de 7.33W) y mejor RTF, mientras Ignition requiere más recursos para un rendimiento inferior. Los datos al completo se encuentran en la sección B.2.7.

4.2.8. Sin Cargar - No GUI - GPU

La comparación entre Ignition, Isaac Sim y Webots en modo sin carga, sin GUI pero con GPU activa, revela patrones de rendimiento marcadamente diferentes. Ignition muestra un uso de CPU significativamente mayor (32.81 % de media) frente a Isaac Sim (19.48 %) y Webots (5.79 %), con frecuencias de procesador un 25 % más altas que Isaac Sim (2.57 GHz vs 2.06 GHz) y un 60 % superiores a Webots (1.60 GHz), evidenciando su mayor demanda computacional incluso sin interfaz gráfica.

En utilización de GPU, Isaac Sim presenta el comportamiento más extremo con un 88.07 % de uso medio frente al 31.01 % de Ignition y 18.68 % de Webots, acompañado de un consumo de memoria GPU 18 veces mayor que Ignition (3.79 GB vs 206 MB) y 5.3 veces superior a Webots (714 MB). Sin embargo, esta intensa utilización no se traduce en mejor RTF, siendo Webots el más eficiente con 0.929 (cerca de tiempo real) frente a 0.588 de Ignition y 0.151 de Isaac Sim, mostrando una diferencia del 84 % entre el mejor y peor desempeño.

El análisis de memoria RAM revela que Isaac Sim utiliza casi el doble de memoria activa (7.01 GB) comparado con Ignition (3.65 GB) y Webots (3.07 GB), mientras Webots mantiene el uso más bajo y estable en todos los parámetros. El consumo energético refleja estas diferencias: Isaac Sim tiene el mayor consumo medio (63.64 W), seguido por Ignition (45.63 W) y Webots (29.19 W), siendo Webots un 54 % más eficiente que Ignition y un 118 % más que Isaac Sim.

Estos resultados demuestran que Webots ofrece el mejor equilibrio entre rendimiento y eficiencia en esta configuración, logrando el mayor RTF con el menor consumo de recursos. Ignition presenta un rendimiento intermedio, mientras Isaac

Sim, a pesar de su alta utilización de GPU, muestra el peor desempeño en RTF con el mayor consumo energético. Los datos al completo se encuentran en la sección B.2.8.

4.3. Análisis de resultados

Una vez expuestos los datos obtenidos primero se presentará un análisis comparativo de los tres simuladores evaluados (Ignition Gazebo, Webots e Isaac Sim) por separado, incluyendo gráficas representativas y conclusiones individuales para cada uno. Después se presentará un análisis similar pero comparando los 3 simuladores, y al igual que antes, incluyendo gráficas representativas y conclusiones de la comparación.

4.3.1. Análisis Individual por Simulador

Ignition Gazebo

■ Impacto de la carga de batería:

- El Real Time Factor es claramente superior cuando el portátil está cargando, lo que demuestra una simulación más fluida y eficiente en ese estado.

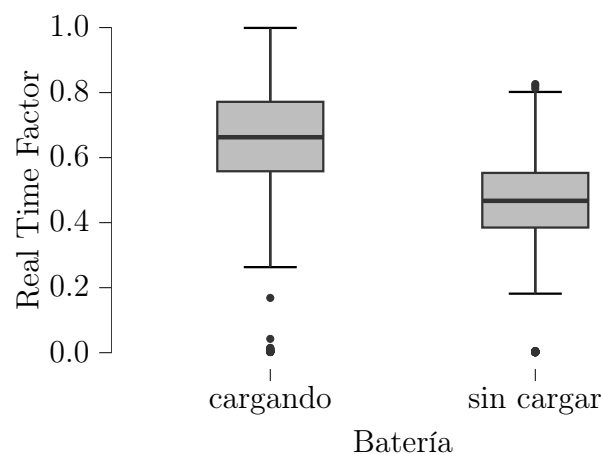


Figura 4.1: Ignition Gazebo - Comparación del RTF dependiendo batería

- Aunque el uso de memoria GPU es similar, la potencia consumida y la temperatura son más elevadas al estar cargando, reflejando un mayor aprovechamiento del hardware gráfico.
- La frecuencia de reloj es significativamente mayor cuando el ordenador está cargando, lo que indica mejor rendimiento en este estado.

■ Efecto de la GUI:

- Incrementa significativamente el uso de CPU (promedio del 45.6 % frente al 29.6 %) y la memoria activa (4066 MB frente a 3602 MB), lo que implica una mayor carga sobre el sistema.

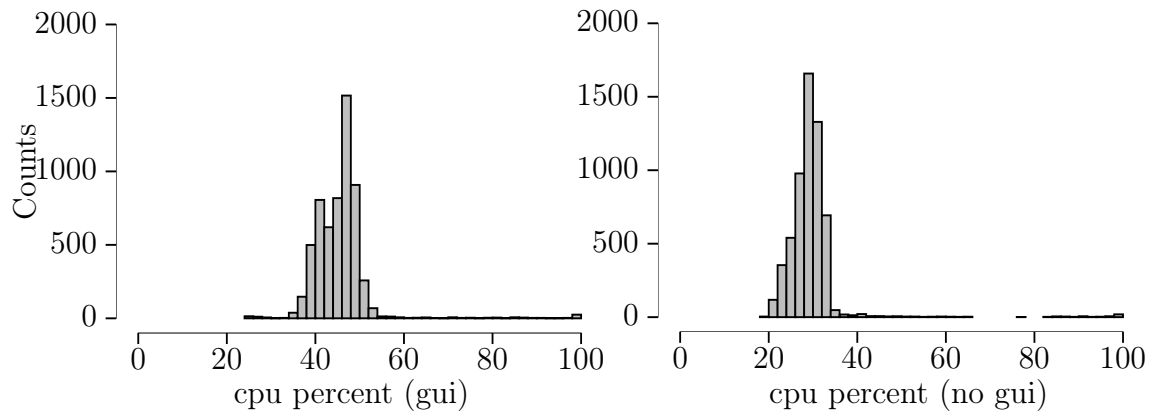


Figura 4.2: Ignition Gazebo - Comparación del uso de CPU con/sin GUI

- Duplica el uso medio de memoria GPU (235 MB con GUI frente a 109 MB sin GUI), junto con un aumento notable en el consumo energético y la temperatura de la tarjeta gráfica.
- Reduce el Real Time Factor (RTF) medio de 0.620 a 0.514, lo que implica simulaciones más lentas, además de un mayor consumo energético total.

■ Uso de GPU:

- El uso de GPU reduce ligeramente la carga de CPU, aumentando su frecuencia media de 2.93 GHz a 3.08 GHz y el uso medio total de CPU del 36.77 % al 38.36 %.
- La utilización de GPU incrementa considerablemente el uso de memoria y potencia gráfica, pasando de 15 MB a 330 MB de memoria usada en la GPU y de 2.58 W a 12.57 W de consumo eléctrico.

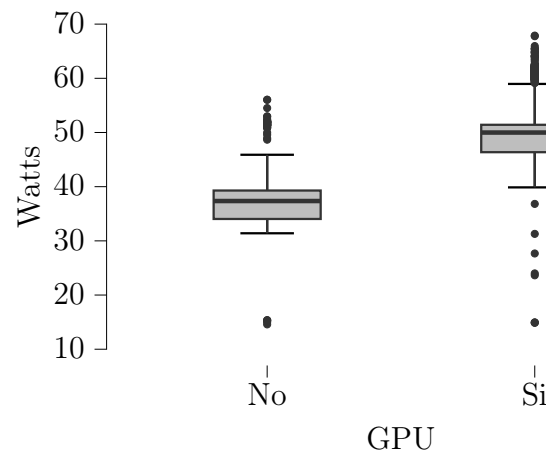


Figura 4.3: Ignition Gazebo - Comparación del gasto de batería con/sin GPU

- El rendimiento de simulación mejora significativamente con GPU, con un incremento del Real Time Factor (RTF) medio de 0.495 a 0.639.

Conclusión final: Ignition Gazebo muestra alta sensibilidad al estado de carga y al uso de interfaz gráfica (GUI). La utilización de la GPU proporciona mejoras significativas en rendimiento, aunque a costa de un mayor consumo energético.

Webots

■ Impacto de la carga de batería:

- Cuando el ordenador está sin cargar, el uso medio de CPU aumenta un 25,6 % (de 4,906 % a 6,162 %) y la frecuencia media disminuye un 21,2 % (de 2,032 MHz a 1,602 MHz), lo que indica una limitación de rendimiento en modo batería.

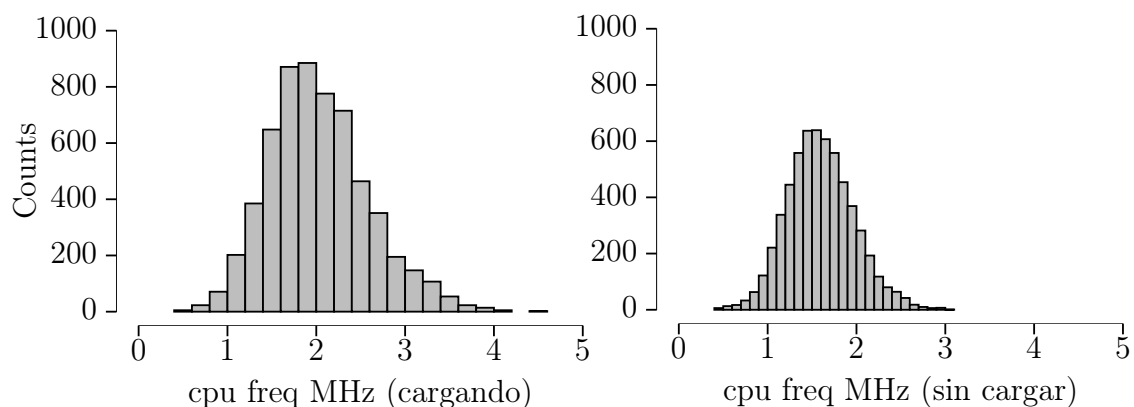


Figura 4.4: Webots - Comparación de la frecuencia dependiendo batería

- A pesar de que la memoria usada por la GPU es prácticamente igual, el consumo medio de energía cae de 4,805 W a 3,176 W sin cargar (-33,9 %).
- El Real Time Factor (RTF) se mantiene prácticamente constante entre ambos estados.

■ Efecto de la GUI:

- Incrementa el uso medio de CPU de un 5.26 % a un 5.80 %, y la frecuencia media de CPU sube ligeramente de 1806 MHz a 1830 MHz.
- La memoria GPU usada disminuye al activar la GUI (de 363 MB a 246 MB en media), pero el consumo energético y la temperatura de la GPU aumentan, pasando de 3.32 W y 41.3 °C a 4.67 W y 43 °C respectivamente.

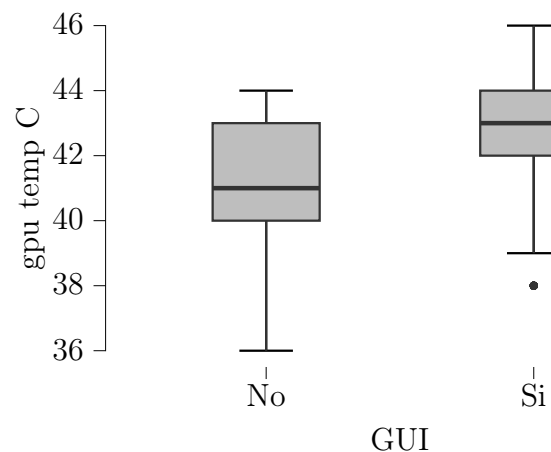


Figura 4.5: Webots - Comparación del uso de CPU con/sin GUI

- El factor de tiempo real (RTF) apenas varía (0.928 sin GUI vs. 0.932 con GUI), pero la batería consume más energía con GUI, pasando de una media de 27.1 W a 31 W, lo que muestra un aumento significativo del consumo energético al usar la interfaz gráfica.

■ Uso de GPU:

- El uso promedio de CPU es ligeramente menor cuando se usa GPU (5.464 % vs. 5.600 %), y la frecuencia media de la CPU baja un poco (1.795 MHz vs. 1.841 MHz), lo que indica un pequeño alivio en la carga del procesador al activar la GPU.
- Activar la GPU implica un aumento significativo en el uso de memoria GPU (media 594.7 MB vs. 15 MB sin GPU) y consumo energético (5.23 W vs. 2.77 W).

- El factor de tiempo real (RTF) apenas varía con GPU (media 0.927 vs. 0.933 sin GPU), pero el consumo de batería aumenta al usar GPU, pasando de 28.1 W a 30.3 W en promedio.

Conclusión final: Webots muestra mayor estabilidad frente a cambios en configuración, particularmente en RTF. La GPU no proporciona beneficios significativos en rendimiento.

Isaac Sim

■ Impacto de la carga de batería:

- El uso medio de CPU es un 11.3 % mayor sin cargar (19.43 % vs. 17.46 %), pero la frecuencia media del procesador disminuye un 18.9 % (2.01 GHz vs. 2.48 GHz).
- La potencia media consumida por la GPU se reduce un 45.1 % en modo batería (29.43 W vs. 53.60 W) junto con una caída de temperatura de 11.2 °C (52.04 °C vs. 63.28 °C), mientras que la memoria usada se mantiene estable cerca de 4.2 GB, mostrando un fuerte ajuste para conservar energía.

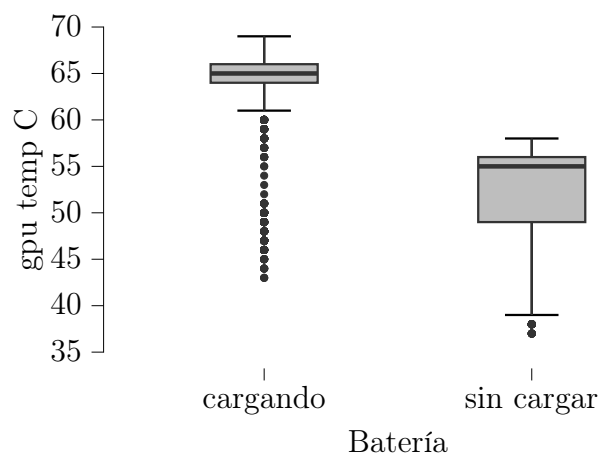


Figura 4.6: Isaac Sim - Comparación de la temperatura de GPU dependiendo de la batería

- La memoria RAM activa se reduce un 4.7 % sin cargar (6.82 GB vs. 7.15 GB), y el factor de tiempo real (RTF) medio baja un 19.4 % en batería (0.137 vs. 0.170).

■ Efecto de la GUI:

- El uso medio de CPU es similar con GUI (18.2 %) y sin GUI (18.6 %), aunque el modo de CPU usuario es ligeramente mayor sin GUI (15.2 % vs. 14.7 %).
- El consumo medio de memoria GPU es menor con GUI (3485 MB vs 3800 MB sin GUI), y la potencia promedio también es menor con GUI (40.4 W vs 42.8 W), pero esto se debe a que la inicialización de la GUI tarda mucho y mientras carga no consume.
- El factor de tiempo real (RTF) medio es ligeramente menor con GUI (0.148) que sin GUI (0.161).

Conclusión final: Isaac Sim muestra el mayor consumo de recursos y es más sensible a limitaciones energéticas. Requiere hardware potente para máximo rendimiento.

Tabla resumen

Métrica	Ignition Gazebo	Webots	Isaac Sim
Sensibilidad a carga	Alta	Baja	Moderada
Impacto GUI en RTF	-17.1 %	+0.4 %	-8.1 %
Memoria GPU típica	100–400 MB	300–700 MB	>4000 MB
Consumo energético	Medio	Bajo	Alto
Estabilidad RTF	Variable	Muy alta	Moderada

Tabla 4.1: Comparación de métricas clave entre simuladores.

4.3.2. Análisis Comparativo entre Simuladores

A continuación se mostrará una lista con las 10 conclusiones más importantes a las que se ha llegado:

- **Eficiencia de CPU:** Webots utiliza consistentemente menos CPU (mediana de 4.67–6.17 %) que Ignition (25.6–49 %) e Isaac Sim (13.6–19.94 %) en todas las configuraciones.

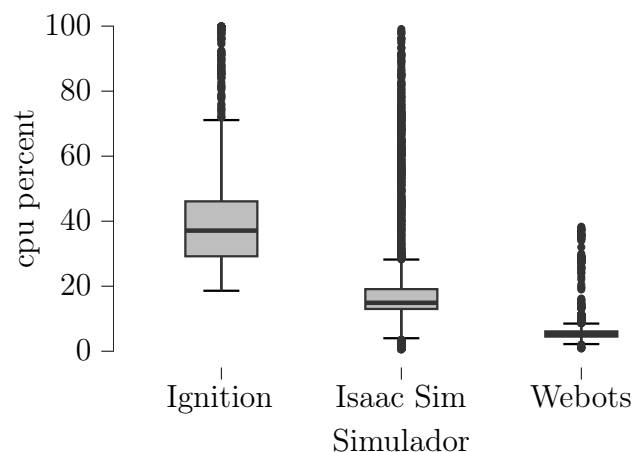


Figura 4.7: Comparación de simuladores - CPU Percent

- **Uso de GPU:** Isaac Sim demanda casi máxima utilización de GPU (mediana de 72–100 %) con alto consumo de energía (26.84–53.76 W), mientras que Webots mantiene un uso moderado de GPU (18–34 % mediana) a menor consumo (4.75–7.11 W).

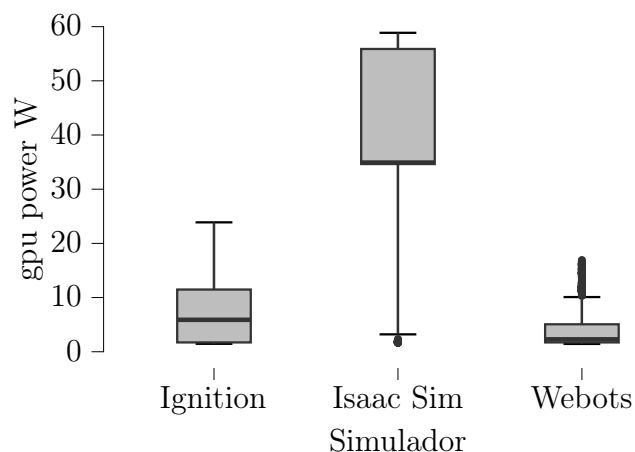


Figura 4.8: Comparación de simuladores - GPU Power

- **Requisitos de RAM:** Isaac Sim requiere considerablemente más RAM activa (6.67–7.21 GB) que Ignition (3.52–4.17 GB) y Webots (2.79–3.08 GB), con hasta 2.6 veces mayor uso en escenarios con GPU activada.

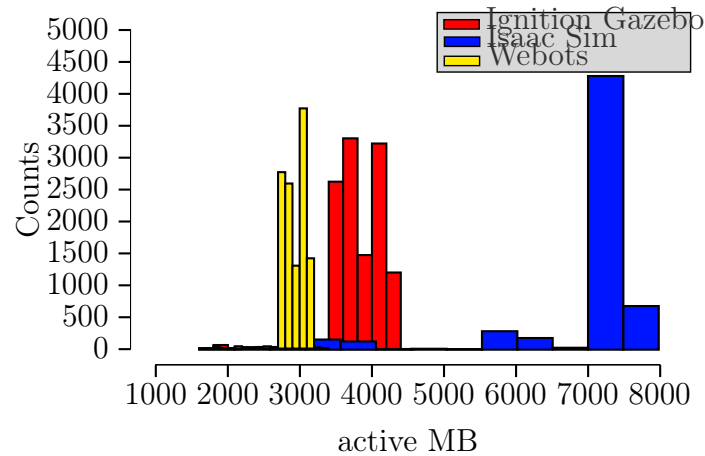


Figura 4.9: Comparación de simuladores - RAM Active MB

- **Rendimiento RTF:** Webots logra mejores factores de tiempo real (media 0.926–0.944), superando a Ignition (0.436–0.812) e Isaac Sim (0.120–0.171) en eficiencia de simulación entre un 44 y 86 %.

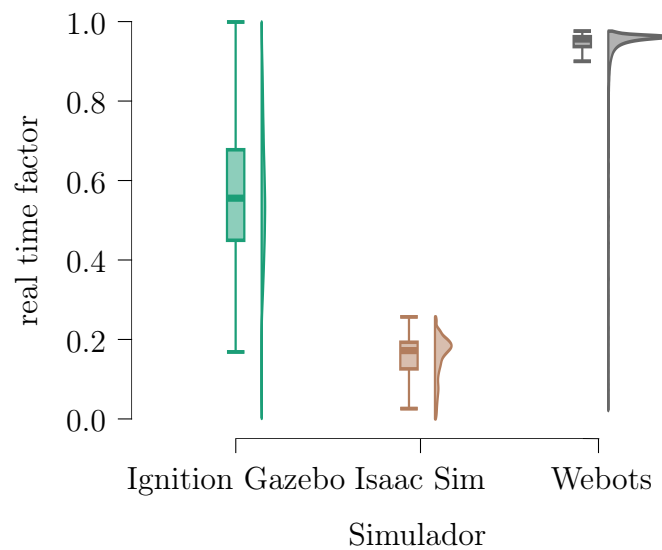


Figura 4.10: Comparación de simuladores - RTF

- **Consumo energético:** Webots consume entre 27 y 54 % menos energía (26.99–31.49 W) que Ignition (34.32–53.05 W) y entre 82 y 118 % menos que Isaac Sim (58.56–63.64 W) en pruebas en modo sin cargar.

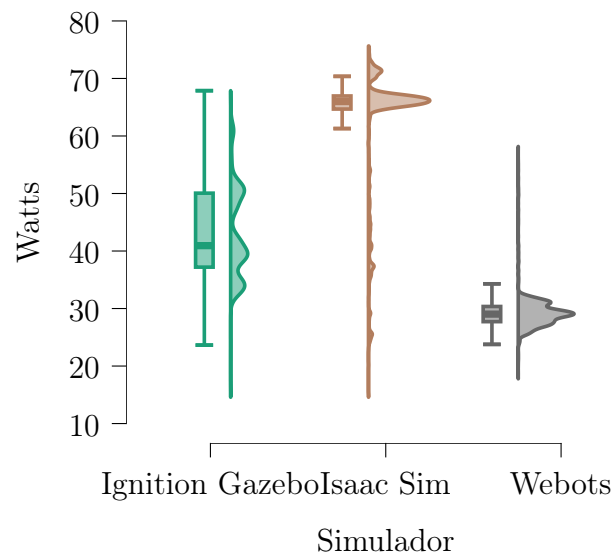


Figura 4.11: Comparación de simuladores - Batería

- **Impacto térmico:** Ignition e Isaac Sim elevan la temperatura de CPU entre 3 y 10 °C (45.66–58.19 °C) en comparación con Webots (42.18–48.60 °C), indicando mayor estrés térmico en el hardware.
- **Impacto de la GUI:** La eliminación de la GUI reduce el uso de CPU entre 30 y 40 % para Ignition (por ejemplo, de 42.58 % a 26.27 % sin GUI ni GPU) y solo entre 10 y 15 % para Webots, mostrando mejor eficiencia base.

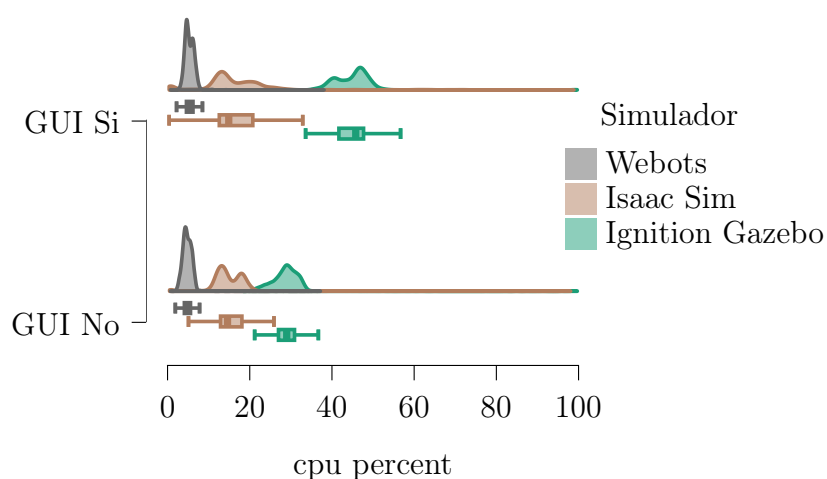


Figura 4.12: Comparación de simuladores - Con/Sin GUI - CPU Percent

- **Beneficio de aceleración GPU:** Ignition obtiene una mejora del RTF entre 20 y 25 % (de 0.546 a 0.654) con GPU, mientras que Webots prácticamente no se ve afectado.
- **Optimización de memoria:** Webots mantiene un uso consistente de memoria cache ($\approx 3.25\text{ GB}$) en todas las configuraciones, mientras que Ignition e Isaac Sim varían entre 15 y 20 % según GPU o GUI activadas.
- **Eficiencia de batería:** En escenarios móviles, Webots prolonga el tiempo operativo con un consumo promedio 7.33 W menor que Ignition y 36.65 W menos que Isaac Sim, lo que se traduce en una duración de batería entre 27 y 118 % superior.

4.4. Explicación detallada de Webots

4.4.1. Porqué Webots

En las comparaciones anteriores se ha visto como el simulador de Webots es el menos limitante respecto a los recursos computacionales necesarios para su funcionamiento. Esto muestra Webots es la mejor alternativa para usar respecto a Ignition Gazebo, especialmente en entornos exteriores, debido a su eficiencia en el uso de recursos computacionales. Aunque los cálculos realizados se basan en un TurtleBot en interiores, se ha observado que Webots requiere significativamente menos CPU (4.67–6.17 %) en comparación con Gazebo (5.6–49 %) e Isaac Sim (15–45 %). Esta menor carga computacional lo convierte en una opción más ligera y escalable, lo que resulta especialmente ventajoso al simular entornos más complejos como los exteriores además de no necesitar un ordenador con grandes capacidades.

A continuación se mostrarán más razones por las que Webots es buena alternativa, desde una explicación más detallada de Webots, la funcionalidad básica que ofrece, los mundos y robots exteriores disponibles y su compatibilidad con ROS 2 gracias a su paquete `webots_ros2`.

4.4.2. Webots: Descripción detallada

Webots es un simulador 3D de robots móviles de código abierto, desarrollado por Cyberbotics Ltd. desde 1998. Su diseño se orienta principalmente a la investigación, docencia y desarrollo de aplicaciones robóticas, y desde 2019 se distribuye bajo licencia Apache 2.0, lo que permite su uso tanto en entornos académicos como industriales sin restricciones comerciales [28,29]. Webots ha ganado popularidad

por ofrecer un entorno completo que integra modelado 3D, simulación física realista, sensores y actuadores preconfigurados, así como herramientas para programación, visualización y exportación de datos. Su objetivo principal es permitir a los usuarios diseñar, simular, probar y validar comportamientos de robots en condiciones cercanas al mundo real.

Una de las mayores fortalezas de Webots es su enfoque multiplataforma (Windows, Linux y macOS) y su capacidad para ejecutarse tanto en local como en la nube mediante Webots Cloud, lo que permite simular robots sin necesidad de una estación potente, utilizando recursos de computación remota [19]. Además, Webots utiliza el motor de física ODE (Open Dynamics Engine) para simular dinámicas complejas como colisiones, fricción, gravedad y articulaciones [22, 28].

A nivel estructural, los mundos en Webots se definen mediante ficheros .wbt basados en un lenguaje jerárquico llamado VRML (Virtual Reality Modeling Language), con una sintaxis clara y editable tanto desde la interfaz gráfica como desde código. También permite importar modelos en formato URDF o CAD (como STL, OBJ), facilitando la transición desde herramientas externas de diseño [20].

Cuenta con una gran comunidad activa en GitHub, con miles de commits y una extensa documentación, refuerza su carácter como herramienta madura y en desarrollo constante [25].

4.4.3. Funcionalidad básica de Webots

Webots proporciona un entorno integrado que cubre el ciclo completo de simulación robótica, desde el diseño del modelo físico hasta la ejecución de algoritmos de control, análisis de resultados y visualización. A continuación, se resumen sus principales funcionalidades básicas:

1. **Modelado de robots y entornos:** Los usuarios pueden crear o importar robots utilizando nodos jerárquicos (como Robot, Solid, Joint, Shape, etc.) con configuraciones de geometría, materiales, articulaciones, sensores y actuadores. La interfaz incluye un editor visual y también se puede modificar directamente el fichero .wbt. Los modelos incluyen ejemplos de robots móviles, brazos manipuladores, drones, vehículos autónomos y humanoides [22].
2. **Simulación física:** Webots emplea el motor ODE para simular la dinámica de cuerpos rígidos con soporte de gravedad, fricción, masas, colisiones, articulaciones, muelles y motores. Esto permite emular con precisión interacciones

físicas del entorno, lo que resulta útil tanto para tareas de navegación como manipulación [22].

3. **Sensores y actuadores:** Dispone de una amplia gama de sensores ya integrados: cámaras, LIDAR, GPS, IMU, brújula, sonar, micrófono, tacto, entre otros. También ofrece actuadores como ruedas, motores de articulación, luces, servos, etc. Todo ello se puede controlar directamente mediante programación [18,22].
4. **Programación:** El control del robot puede implementarse en Python, C, C++, Java, MATLAB o ROS 2. El entorno incluye un compilador y depurador integrados. También es posible ejecutar los controladores de forma externa, conectándose al simulador mediante sockets TCP/IP. En particular, Webots ofrece integración con ROS 2 mediante el paquete `webots_ros2`, lo que permite utilizar nodos, servicios y tópicos nativos de ROS 2 [26,90].
5. **Herramientas adicionales:**
 - **Supervisor API:** permite modificar la simulación en tiempo real, cambiar posiciones, crear/eliminar nodos o capturar datos para análisis [22].
 - **Visualización de trayectorias y sensores:** Webots permite renderizar los valores de sensores (como rayos del LIDAR o imágenes de cámara) directamente en el entorno.
 - **Exportación de datos:** se pueden registrar logs, trayectorias y valores de sensores en ficheros CSV o JSON para análisis posterior.
6. **Simulación en la nube:** Gracias a Webots Cloud, se pueden subir simulaciones al servidor de Cyberbotics, donde se ejecutan de forma remota, lo que elimina la necesidad de hardware potente local y permite colaboración multiusuario [19].

4.4.4. Robots y mundos exteriores disponibles

Webots pone a disposición de los usuarios una extensa librería de modelos PROTO, accesible desde su plataforma en la nube, Webots Cloud. Esta colección abarca tanto robots como sensores, escenarios exteriores, objetos interactivos y estructuras complejas, todos diseñados para integrarse fácilmente en cualquier simulación. Los archivos PROTO son plantillas reutilizables de objetos que encapsulan geometría, comportamiento, sensores y controladores, permitiendo construir mundos detallados de forma rápida y eficiente [19].

Robots disponibles Los modelos de robots están organizados por tipo y aplicación. Algunos de los más destacados incluyen:

■ **Robots móviles diferenciales y omnidireccionales:**

- E-puck Robot, iRobot Create3, Pioneer, Khepera, TurtleBot3 Burger y Summit (entre otros).
- Algunos modelos cuentan con LIDAR, cámara, IMU y motores configurables.

■ **Robots cuadrúpedos y caminantes:**

- Spot Robot (inspirado en Boston Dynamics), Mantis o Aibo.
- Utilizados para simulaciones avanzadas de locomoción y control dinámico.

■ **Drones y vehículos aéreos:**

- Crazyflie, Mavic2Pro, Blimp.
- Simulan vuelo estable, navegación aérea y captura de imágenes.

■ **Robots humanoides:**

- NAO, RobotisOp, Atlas y Khr3hv.
- Aptos para estudios de locomoción bípeda, visión artificial y HRI (interacción humano-robot).

■ **Brazos manipuladores y robots industriales:**

- UR5e, UR10e, UR3e, Panda, Scara, Puma o KUKA.
- Todos incluyen cinemática avanzada, sensores de fuerza, grippers y configuración ROS2-ready.

■ **Vehículos terrestres y autónomos:**

- Tesla Model 3, Toyota Prius, BMW X5, Tractor, Mercedes Sprinter.
- Pueden emplearse junto a sensores como cámaras estéreo, LIDAR o GPS para desarrollo de conducción autónoma.

■ **Otros robots destacados:**

- Tiago, YouBot (KUKA).
- Utilizados en tareas de manipulación móvil y aplicaciones de logística o interacción humano-robot.

Todos estos modelos PROTO están completamente parametrizados, lo que permite modificar aspectos como la masa, tamaño, velocidad de motores o número de sensores sin necesidad de rediseñarlos desde cero.

Mundos exteriores y elementos estructurales Webots también incluye una gran variedad de entornos y objetos de simulación exteriores, que permiten crear mundos realistas para tareas como navegación autónoma, exploración, SLAM o manipulación compleja.

■ **Entornos urbanos y carreteras:**

- City, Highway, Roundabout, Village, StreetFurniture.
- Diseñados para pruebas de vehículos autónomos en condiciones realistas.

■ **Zonas industriales e interiores:**

- Warehouse Environment, Factory Floor, Conveyor Belt, Industrial Scaffold.
- Adecuados para simular logística, picking, y colaboración entre robots.

■ **Escenarios naturales y exteriores:**

- Forest Environment, Mountain Terrain, Mars Landscape, Cliff.
- Permiten explorar movilidad sobre terreno irregular, exploración planetaria y visión por computador.

■ **Edificios y estructuras:**

- Multi Floor Building, Office Environment, Elevator, Stairs, Bridge.
- Útiles para tareas de planificación, movilidad en interiores y navegación compleja.

■ **Objetos interactivos y mobiliario:**

- Table, Chair, Door, Box, Trash Can, Cupboard, Liftable Box.
- Pueden integrarse en simulaciones de manipulación o interacción social.

Todos estos entornos están listos para integrarse con los robots disponibles y pueden combinarse libremente para crear simulaciones personalizadas. Además, muchos de ellos están preparados para su uso con ROS 2, facilitando pruebas de percepción, navegación y control de alto nivel [19].

4.4.5. Compatibilidad con ROS 2

Una de las características más potentes de Webots es su integración nativa con ROS 2, lo que permite a los desarrolladores trabajar con robots simulados utilizando las herramientas, mensajes y nodos gracias al paquete `webots_ros2`, mantenido activamente por Cyberbotics, en el cual los usuarios pueden implementar, simular y probar algoritmos robóticos de manera realista y flexible, con una integración directa en el ecosistema ROS 2 [21, 26].

Estructura del paquete `webots_ros2` El paquete `webots_ros2` incluye nodos específicos para diferentes funcionalidades robóticas y dispositivos del simulador. Está construido utilizando las herramientas estándar de ROS 2, como `ament`, `colcon`, `rclcpp` y `rclpy`. Cada robot o dispositivo tiene su propia interfaz ROS 2 modularizada, facilitando la escalabilidad y personalización del entorno de simulación [26].

Entre sus principales componentes se incluyen:

- **`webots_ros2_driver`**: nodo que actúa como puente entre un robot Webots y ROS 2.
- **`webots_ros2_importer`**: sirve para importar robots desde Webots a ROS 2.
- **`webots_ros2_control`**: permite usar controladores ROS 2 estándar para controlar los actuadores del robot.
- **`webots_ros2_msgs`**: definición de mensajes personalizados si son necesarios.
- Interfaces para robots populares como TurtleBot3, e-puck2, TIAGo, Spot y Tesla Model 3 [12, 13, 21].

Dispositivos soportados en ROS 2 Webots ofrece compatibilidad ROS 2 con una amplia variedad de dispositivos y sensores utilizados habitualmente en robótica, todos accesibles mediante topics, servicios y parámetros estándar de ROS 2. Algunos ejemplos destacados incluyen [14]:

- **Sensores de percepción**: cámara RGB, cámara estéreo, cámara de profundidad (`DepthCamera`), LIDAR (1D y 3D), GPS, brújula (`Compass`), IMU, acelerómetro y giroscopio.
- **Sensores de proximidad**: `DistanceSensor`, `TouchSensor`.
- **Actuadores**: motores de posición, velocidad o torque; servos; luces LED.

- **Sensores hápticos:** ForceSensor, PositionSensor.
- **Controladores adicionales:** receptores, emisores, emisores de sonido, etc.

Prácticamente todos estos dispositivos contarán con una interfaz ROS 2 definida por tópicos (`/camera/image_raw`, `/scan`, `/cmd_vel`, etc.), servicios y parámetros que siguen las convenciones estándar de ROS 2 gracias a `webots_ros2_driver`, permitiendo portabilidad entre simulación y hardware real [14].

Nodos supervisor y monitorización de simulación Una de las extensiones avanzadas que ofrece Webots es la integración del nodo Supervisor en ROS 2, que permite acceder y modificar dinámicamente el mundo simulado desde código ROS 2. El Supervisor tiene autoridad para [17]:

- Añadir o eliminar objetos y robots en tiempo de ejecución.
- Leer y escribir campos de cualquier nodo del mundo simulado.
- Teletransportar o rotar robots.
- Obtener datos de colisión, contacto o geometría.

Gracias al nodo `webots_ros2_supervisor`, este control puede hacerse desde scripts ROS 2 en Python o C++, lo que abre la puerta a simulaciones reactivas, entornos dinámicos o autoevaluaciones en experimentos robóticos [17].

Ejemplos de integración con robots reales y simulados El paquete `webots_ros2` incluye una gran cantidad de ejemplos que ilustran cómo trabajar con robots populares directamente desde ROS 2. Algunos de los más representativos incluyen [11–13, 16, 53]:

- **TurtleBot3 Burger:** navegación autónoma, teleoperación, sensores LIDAR y cámara RGB-D.
- **E-puck2:** control básico y sensores para iniciarse con ROS 2.
- **TIAGo:** manipulación de objetos, movimiento con brazos articulados y navegación.
- **Tesla Model 3:** simulación de conducción autónoma con sensores complejos.
- **Spot (Boston Dynamics):** integración de locomoción cuadrúpeda y navegación con mapas.

Todos los ejemplos se acompañan de archivos de lanzamiento (.launch.py) compatibles con ROS 2 Humble y posteriores, además de mundos .wbt preconfigurados para comenzar la simulación de inmediato.

Ventajas frente a otros simuladores Webots se diferencia de otros simuladores por ofrecer [21, 26, 98]:

- Integración lista para usar con ROS 2 sin necesidad de plugins externos.
- Simulación realista de sensores con ruido configurable.
- Interfaz gráfica interactiva incluso con ROS en ejecución.
- Documentación clara, ejemplos prácticos y soporte activo en GitHub.

Además, su compatibilidad total con robots reales mediante tópicos estándar facilita una transición fluida del entorno virtual al hardware físico.

4.4.6. Webots con ROS 2 y NAV2

Una vez vista la compatibilidad de Webots con ROS 2, se explicará en más detalle la manera en la que Webots se comunica con ROS 2 y también como funciona con una de las integraciones más importante en robótica: el sistema de navegación autónoma NAV2 (Navigation2). NAV2 proporciona un conjunto de nodos y herramientas para tareas de navegación, planificación de rutas, evasión de obstáculos, localización y mapeo en entornos desconocidos o dinámicos que, gracias al paquete webots_ros2, es posible de ejecutar dentro de Webots utilizando robots simulados como TurtleBot3 o TIAGo [13, 26, 70].

Pipeline Webots – ROS 2 – NAV2 (diagramas y explicación)

Explicación del pipeline entre Webots y ROS 2 El paquete webots_ros2 permite controlar robots simulados en Webots mediante nodos ROS 2 lo que hace que se conecte el mundo simulado en Webots con ROS 2 de forma que se pueden usar nodos ROS 2 para leer sensores, publicar comandos y recibir datos del robot simulado. Webots ejecuta su simulación física y gráfica, mientras que ROS 2 maneja la lógica de control [26, 90]. A continuación se presentan los paquetes mas importantes de webots_ros2 [21, 89]:

- **webots_ros2_driver:** Este paquete es el núcleo de la integración entre Webots y ROS 2. Sus funciones principales son:

- Crear y ejecutar un nodo ROS 2 por cada robot en la simulación.
- Estos nodos conectan los dispositivos de los robots (motores, sensores, cámaras...) con topics de ROS 2.
- Traduce entre la API de Webots (C++) y mensajes ROS 2.
- Carga un archivo .urdf y lo usa para saber qué sensores y actuadores deben tener nodos ROS 2.

Para ejemplificar su funcionamiento se mostrará un ejemplo de uso: Se tiene un robot con una cámara como sensor, el `webots_ros2_driver` lee la información de la cámara ofrecida por Webots, transforma esta información a un formato compatible con ROS2 y por último publica las imágenes en el tópico indicado, por ejemplo `/robot/camera/image_raw` como mensajes `sensor_msgs/msg/Image`.

- **`webots_ros2_importer`**: Esta herramienta sirve de ayuda para importar automáticamente robots desde Webots a ROS 2. Sus funciones principales son:

- Convertir un robot definido en Webots (.wbt, .proto) a un archivo URDF compatible con ROS 2.
- Extrae:
 - Geometría y articulaciones del robot.
 - Información de sensores y actuadores.
 - Elementos necesarios para `webots_ros2_driver`.

Esto es de gran utilidad y clave ya que Webots usa su propio formato de robot, mientras que ROS 2 usa URDF por lo que con `webots_ros2_importer` se podrán usar los robots de Webots en entornos ROS 2 sin tener que crear el URDF manualmente [88].

- **`webots_ros2_control`**: Este paquete proporciona integración con el sistema `ros2_control`, que permite usar controladores ROS 2 estándar para controlar los actuadores del robot. Sus funciones principales son:

- Conectar Webots a la infraestructura de `ros2_control`:
 - Controladores de posición, velocidad, esfuerzo.
 - Joint state broadcaster, joint trajectory controller, etc.

- Permite usar `ros2_control` para enviar comandos y recibir estados como si fuera un robot real.

Estos 3 paquetes hacen que un flujo de trabajo típico sea el siguiente:

1. Diseñas el robot en Webots (.proto o .wbt).
2. Ejecutas Webots con el mundo cargado.
3. `webots_ros2_importer` genera un URDF.
4. Webots lanza el plugin `webots_ros2_driver`, que crea nodos ROS 2.
5. Si usas controladores ROS 2, configuras `webots_ros2_control` para habilitar `ros2_control`.
6. Interactúas con el robot desde ROS 2 (RViz, teleop, topics, etc.) [70,98].

Diagrama del pipeline entre Webots y ROS 2 Para que este pipeline sea más sencillo de entender se ha creado el siguiente diagrama que muestra las relaciones y comportamientos expuestos anteriormente.

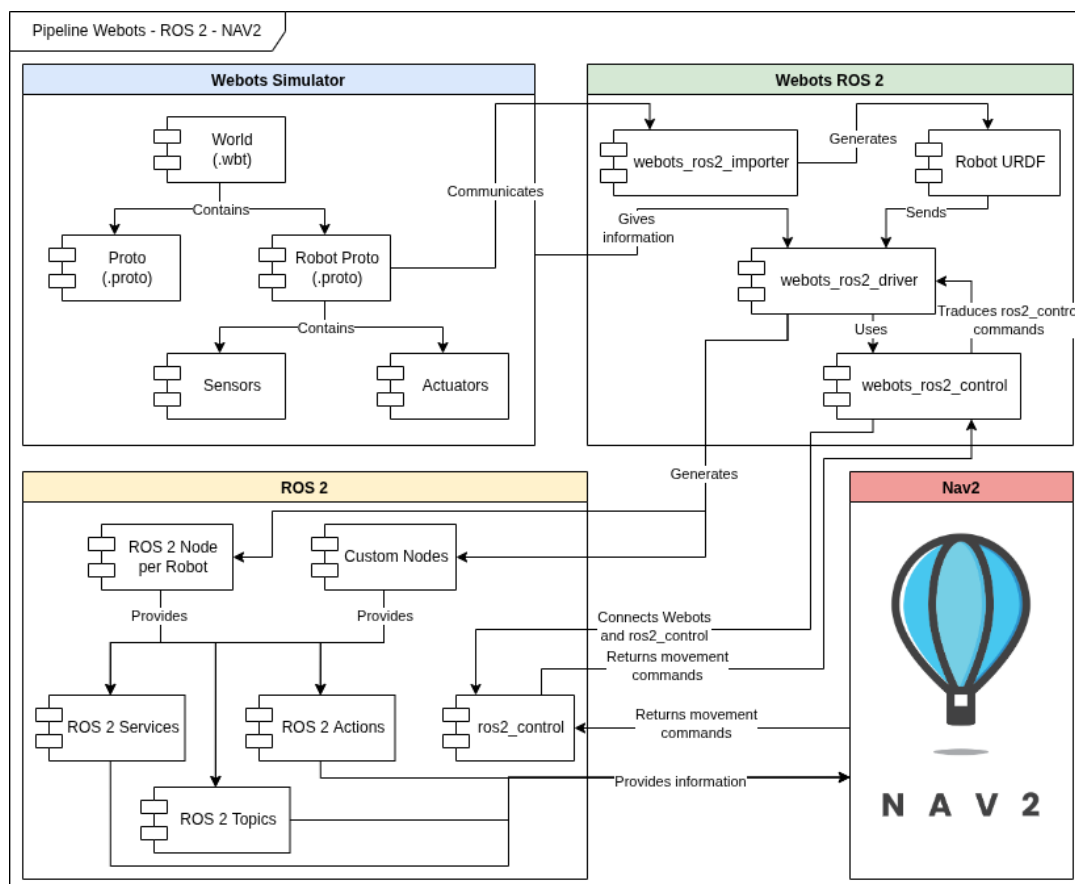


Figura 4.13: Diagrama del pipeline entre Webots y ROS 2

4.5. Ejemplo en Webots ROS 2 y Nav2

Un vez visto que los resultados dan como la mejor alternativa de Ignition Gazebo a Webots se ha desarrollado un ejemplo desde cero que servirá para:

- Mostrar el flujo completo de desarrollo en Webots con ROS 2 y Nav2
- Demostrar los resultados obtenidos en un mundo de exteriores

El desarrollo completo del ejemplo puede consultarse en el repositorio de GitHub del TFM, accesible en el siguiente enlace: https://github.com/JorgeCasatejada02/TFM/tree/main/webots_ros2_example.

En dicho repositorio se documenta paso a paso el proceso seguido, comenzando con la creación de un mundo de ejemplo y finalizando con una demostración en un entorno exterior más complejo. Este segundo escenario tiene como objetivo comprobar el buen rendimiento de Webots incluso en condiciones más exigentes.

A continuación se muestra una captura representativa del sistema en funcionamiento, en la que se incluyen las métricas clave recogidas durante la simulación, tales como el uso de CPU y GPU, así como el Real-Time Factor (RTF):

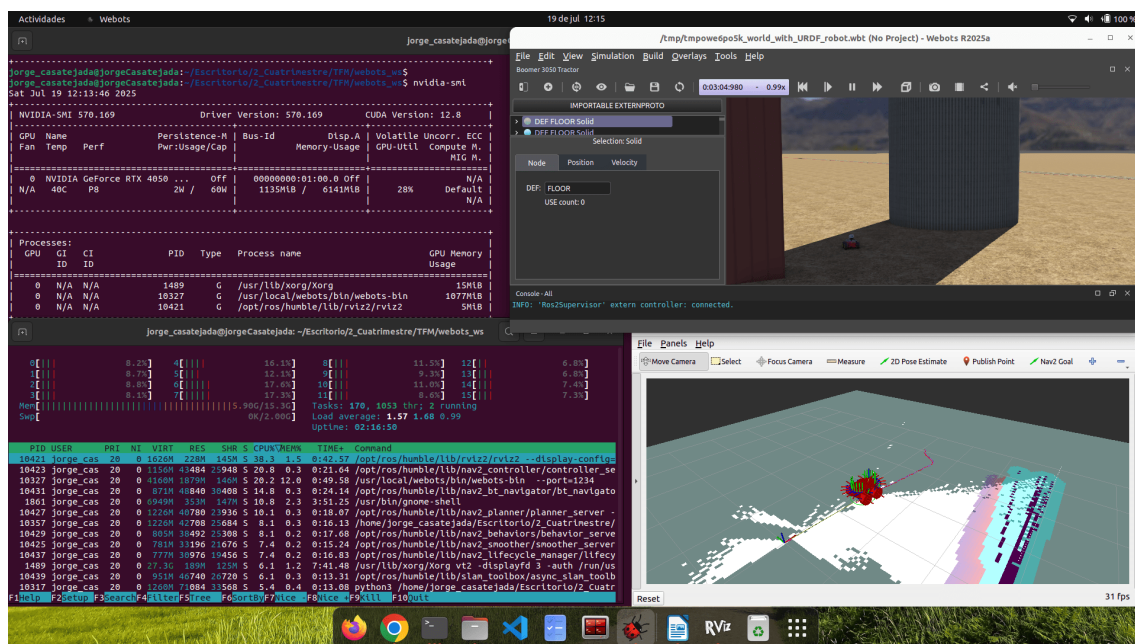


Figura 4.14: Captura del ejemplo en Webots mostrando el uso de CPU, GPU y el RTF durante la navegación en un entorno exterior.

Como muestra la Figura 4.14, Webots mantiene un rendimiento óptimo incluso en entornos complejos, con un consumo eficiente de recursos y un RTF estable que garantiza la correcta ejecución de la simulación en tiempo real.

Capítulo 5

Conclusión

A lo largo de este TFM se ha realizado un análisis comparativo de diferentes simuladores de robótica utilizados en la comunidad de ROS 2, evaluando su rendimiento, consumo de recursos y facilidad de uso en escenarios de exteriores. El trabajo incluyó una revisión del estado del arte, la implementación práctica de ejemplos y la monitorización de métricas de CPU, GPU, memoria, batería y factor de tiempo real (RTF), con el fin de obtener una visión completa sobre la viabilidad de cada herramienta. Todo esto con el objetivo de responder la cuestión de investigación planteada:

Identificar entornos de simulación compatibles con ROS 2 que presenten un menor consumo de recursos computacionales que Gazebo para escenarios al aire libre.

A lo largo de este trabajo hemos identificado y analizado los simuladores más representativos (Gazebo, Webots e Isaac Sim), evaluando sus ventajas, limitaciones y costes computacionales. Como resultado de este análisis podemos afirmar que:

El simulador Webots es con diferencia, sobre Gazebo e Isaac Sim, el simulador que menos uso de recursos computacionales hace, lo que le permite ser usado en ordenadores con hardware limitado en términos de CPU, GPU y memoria. Además de esto Webots, a través de su paquete de ROS 2, permite el desarrollo de proyectos en ROS 2 haciendo que la simulación de robots en exteriores bajo ROS 2 no presente limitaciones en cuanto a los recursos computacionales necesarios para lanzar simulaciones complejas y cambiantes propios de la robótica de exteriores.

Aportaciones realizadas

Las aportaciones realizadas, de manera general, han sido orientadas a resolver los objetivos de este trabajo:

O_1 Comparar el consumo de recursos computacionales entre varios simuladores de robótica de libre acceso y disponibilidad inmediata (off-the-shelf) (Ignition Gazebo, Webots e Isaac Sim)

- Análisis individual de cada simulador (Ignition Gazebo, Webots e Isaac Sim) en distintos escenarios. Estos escenarios son todas las combinaciones posibles entre tener el ordenador cargando o no y usar o no GUI y GPU.
- Comparación entre los simuladores Ignition Gazebo, Webots e Isaac Sim bajo distintos escenarios comunes. Al igual que antes los escenarios serán son todas las combinaciones posibles entre tener el ordenador cargando o no y usar o no GUI y GPU.
- Análisis comparativo global de los simuladores en diversas condiciones de simulación.
- Recopilación y publicación de todos los resultados obtenidos en las simulaciones, los cuales se han subido al repositorio en formato CSV y JASP para su análisis y reutilización.
- Comparación de los resultados obtenidos para visualizar y conocer las principales diferencias entre simuladores.

O_2 Analizar una de las alternativas a Ignition Gazebo más utilizadas (Webots)

- Análisis en profundidad del simulador Webots y todas las opciones que ofrece respecto a ROS 2.
- Realización de un inventario completo de Webots que incluye información sobre versiones, sensores, actuadores, robots, mundos y nodos.

O_3 Desarrollar un ejemplo de robótica de exteriores en ROS 2 y Webots.

- Mostrar el flujo completo de desarrollo en Webots con ROS 2 y Nav2.
- Guía de desarrollo de un ejemplo en Webots ROS 2 y Nav 2.
- Demostrar los resultados obtenidos en un mundo de exteriores.

O_4 Preparar un conjunto de tutoriales de instalación de varios simuladores de robótica de libre acceso y disponibilidad inmediata para favorecer la "ciencia abierta".

- Realizar un tutorial para cada simulador (Gazebo Classic, Ignition Gazebo, Webots e Isaac Sim).
- Subir los tutoriales a GitHub para que estén disponibles de forma abierta.

Lecciones Aprendidas

Problemas en los tutoriales de instalación

Durante la instalación de los simuladores y sus ejemplos se han encontrado varios problemas:

- Problemas de librerías y paquetes: Este problema se refiere a la necesidad de instalación de paquetes o librerías extras no documentados en los tutoriales de instalación oficiales.
- Problema Nvidia driver Isaac Sim: debido a un error hubo que cambiar el driver de Nvidia a la versión 570 y la versión de CUDA toolkit a la 12.8 para su correcto funcionamiento.

Todos estos problemas se encuentran solucionados en los tutoriales de instalación para que así sean evitados en un futuro.

Problema con el Real Time Factor

El problema del Real Time Factor (RTF) se divide en 2 problemas, una para el simulador de Webots y otra para el de Isaac Sim, ya que a diferencia de Ignition Gazebo estos no cuentan con un topic o similar el cual publique el factor de tiempo real de manera interna y automática.

Webots

En el caso de Webots el problema es que este solo expone el RTF a través de su interfaz, pero no es posible extraer este valor para ser leído y guardado de manera automática. Para solucionarlo se desarrolló un nodo de ROS 2 que lee del topic `'/clock'` y calcula el RTF. Una vez calculado es publicado en un topic de ROS 2 `'/rtf'`.

Isaac Sim

En el caso de Isaac Sim el problema es distinto ya que si que cuenta con una manera en la que publica el RTF, pero esta no se puede automatizar ya que solo es posible de activar a través de su interfaz, que crea un nodo de ROS 2 en el que publica esta información. Para solucionarlo se recurrió al topic anteriormente mencionado creado para el calculo del RTF en Webots.

Problemas durante el desarrollo del ejemplo con ROS 2 y Nav2 en Webots

Durante el desarrollo del ejemplo, al no existir tutoriales previos sobre la integración de Webots con ROS 2 y solo disponer de ejemplos sin explicar, surgieron varios problemas. Inicialmente, se intentó utilizar el modelo de tractor disponible en formato Proto, lo que generó complicaciones en la conversión a URDF.

Problemas con la conversión a URDF

El URDF generado automáticamente por Webots presentaba los siguientes inconvenientes:

- Error persistente relacionado con el 'slot 0'
- Intentos de solución:
 1. Añadir manualmente el slot 0 al URDF (sin efecto)
 2. Modificar un nuevo URDF obtenido mediante exportación desde el simulador
 - Eliminar el slot 0 no producía cambios
 - Añadir un link padre para slot 0 generaba conflictos de jerarquía
 - Las modificaciones eran ignoradas por Webots
 3. Pruebas con el prototipo NGJunction (sin éxito)
 4. Verificación mediante Robot Window: las ruedas delanteras solo giraban lateralmente pero no avanzaban/retrocedían

Como resultado, aunque era posible publicar en `cmd_vel`, solo las ruedas traseras respondían adecuadamente, lo que impedía un movimiento correcto del tractor, ya que este solo se movía hacia delante y a tirones.

Diagnóstico del problema

Se buscaron las causas que provocaban este problema y mediante una investigación se encontró que:

- Las articulaciones de las ruedas delanteras usaban `Hinge2Joint`
- Webots no soporta la conversión de `Hinge2Joint` a URDF (según documentación oficial)

Hay que tener en cuenta que previamente ya se habían configurado todos los parámetros de Nav2 y ROS 2 Control para navegación Ackermann, por lo que este trabajo realizado no sirvió para el desarrollo definitivo del proyecto.

Solución implementada

Ante estas limitaciones, se optó por:

1. Buscar modelos alternativos de tractor en URDF (sin éxito)
2. Intentar convertir otros formatos a URDF usando `urdf2webots`
3. Intentar modificar el `Hinge2Joint` en el vehículo Ackermann por otro tipo de articulación soportada
4. Finalmente, cambiar al robot Pioneer 3AT (tracción diferencial y para exteriores)

Ajustes finales con Pioneer 3AT

El único problema restante fue la configuración del LIDAR, solucionado mediante:

- Corrección del nombre del LIDAR en el URDF
- Cambio de `world name` a `scan`
- Actualización de `frame name` y `topicName` a `scan`

Con estas modificaciones, se logró la integración exitosa del Pioneer 3AT en Webots con ROS 2 y Nav2.

Opiniones personales

Comparativa de simuladores robóticos

En mi experiencia trabajando con diferentes entornos de simulación, he observado que:

- **Profundidad funcional:** Los simuladores actuales ofrecen capacidades muy avanzadas que van más allá del uso típico, con múltiples opciones y configuraciones disponibles.
- **Eficiencia de Webots:** Destaca notablemente por su bajo consumo de recursos en comparación con alternativas como Ignition Gazebo. Esta ventaja es particularmente relevante cuando:
 - Se trabaja con hardware limitado
 - Se prioriza la fluidez sobre gráficos ultra-realistas

Experiencia personal con requerimientos hardware

- **Caso Ignition Gazebo:** Al inicio del curso, me vi obligado a adquirir un portátil más potente específicamente para poder utilizar este simulador sin problemas de rendimiento.
- **Ventaja de Webots:** Su optimización lo convierte en una solución ideal para:
 - Equipos con especificaciones modestas
 - Entornos educativos con recursos limitados
 - Desarrollo rápido sin sacrificar productividad

Conclusión

Webots representa una alternativa sumamente valiosa para el desarrollo robótico, especialmente cuando:

- Los recursos computacionales son limitados
- Se busca un equilibrio entre funcionalidad y rendimiento
- Se requiere accesibilidad para una amplia gama de configuraciones hardware

Trabajos futuros

La investigación realizada en este TFM y la identificación de Webots como una alternativa eficiente a Gazebo abren varias líneas de trabajo futuro. Estas se centran principalmente en mejorar las herramientas de integración entre simuladores y el ecosistema ROS 2, así como en la optimización del rendimiento en escenarios complejos.

Mejoras en la conversión de modelos

Uno de los desafíos identificados durante el desarrollo del ejemplo práctico fue la conversión de modelos de robot a un formato compatible con Webots. A la hora de realizar la conversión de modelos es necesario fijarse en dos puntos principalmente: la corrección de las articulaciones y un sistema de pruebas robusto para validar el proceso. A continuación, se detallan las propuestas para estas dos áreas.

- **Corrección de articulaciones:**

- Investigar soluciones alternativas para la conversión de joints problemáticos (especialmente Hinge2Joint de Ackermann)
- Solucionar el problema desde el propio código que genera estos URDF y dar soporte a este tipo de articulaciones aún no soportadas

- **Pruebas con urdf2webots:**

- Evaluar la conversión de modelos Ackermann existentes a formato Webots
- Verificar la compatibilidad con los controladores ROS 2

Optimización de simuladores

Analizar en profundidad las opciones de configuración y optimización gráfica disponibles en cada simulador con el objetivo de minimizar el uso de recursos computacionales y mejorar el rendimiento en entornos de simulación complejos. En particular, se propone:

- Desactivar sombras dinámicas o proyectadas si no son necesarias para los experimentos, ya que suelen tener un impacto significativo en el rendimiento gráfico.

- Reducir la calidad de renderizado, por ejemplo, disminuyendo la resolución de texturas, desactivando reflejos o simplificando materiales y geometrías innecesarias.
- Utilizar modos de renderizado simplificados o alternativos que ofrezcan un buen equilibrio entre fidelidad visual y eficiencia computacional (como renderizado en modo wireframe o sin efectos posprocesado).
- Ajustar el campo de visión, distancia de dibujo y tasa de actualización del renderizado para evitar procesamiento gráfico innecesario.
- Deshabilitar la simulación de efectos ambientales costosos, como niebla, luces múltiples o iluminación global, cuando no sean relevantes para el objetivo del experimento.

Agradecimientos

A mi familia y amigos por el apoyo y ayuda durante el transcurso del proyecto y el máster.

A mis tutores, Fran y Miguel, por guiarme tanto en este proyecto como en el curso académico.

Lista de referencias

- [1] arXiv. Simulación robótica y benchmarking. <https://arxiv.org/abs/2008.04627>, 2020. Estudio académico.
- [2] arXiv. Comparativa de simuladores. <https://arxiv.org/pdf/2204.06433>, 2022. Estudio académico.
- [3] Asepeyo. Bases y tipos de cotización a la seguridad social. <https://www.asepeyo.es/duda/bases-y-tipos-de-cotizacion/>, 2025. Bases y tipos de cotización 2025.
- [4] Colin Ian King. Powerstat — Herramienta para medir el consumo energético. <https://github.com/ColinIanKing/powerstat>, 2025. Repositorio oficial de Powerstat.
- [5] Comisión Europea. Propuesta de Reglamento del Parlamento Europeo y del Consejo relativo a la inteligencia artificial (Ley de Inteligencia Artificial) y por el que se modifican determinados actos legislativos de la Unión. <https://eur-lex.europa.eu/legal-content/ES/ALL/?uri=CELEX:52021PC0206>, 2021. COM/2021/206 final - 2021/0106(COD).
- [6] Coppelia Robotics. Coppeliasim - robot simulator. <https://www.coppeliarobotics.com/>. Sitio oficial de CoppeliaSim.
- [7] Coppelia Robotics. Coppeliasim ros interface. <https://github.com/CoppeliaRobotics/simROS>. Repositorio oficial ROS para CoppeliaSim.
- [8] Coppelia Robotics. Coppeliasim ros2 interface. <https://github.com/CoppeliaRobotics/simROS2>. Repositorio oficial ROS 2 para CoppeliaSim.
- [9] Coppelia Robotics. Ros2 interface tutorial. <https://manual.coppeliarobotics.com/en/ros2Tutorial.htm>. Tutorial oficial de integración ROS 2 en CoppeliaSim.

-
- [10] Cortes Generales. Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>, 2018. Publicado en el Boletín Oficial del Estado núm. 294, de 6 de diciembre de 2018, páginas 119728 a 119773.
 - [11] Cyberbotics. Example - tesla model 3. https://github.com/cyberbotics/webots_ros2/wiki/Example-Tesla-Model-3. Tutorial oficial.
 - [12] Cyberbotics. Example - tiago. https://github.com/cyberbotics/webots_ros2/wiki/Example-TIAGo. Tutorial oficial.
 - [13] Cyberbotics. Example - turtlebot3 burger. https://github.com/cyberbotics/webots_ros2/wiki/Example-TurtleBot3-Burger. Tutorial oficial.
 - [14] Cyberbotics. References - devices. https://github.com/cyberbotics/webots_ros2/wiki/References-Devices. Documentación técnica.
 - [15] Cyberbotics. Starting webots - command line arguments. <https://cyberbotics.com/doc/guide/starting-webots#command-line-arguments>. Documentación oficial de argumentos CLI para Webots.
 - [16] Cyberbotics. Tutorial - e-puck for ros2 beginners. https://github.com/cyberbotics/webots_ros2/wiki/Tutorial-E-puck-for-ROS2-Beginners. Tutorial oficial.
 - [17] Cyberbotics. Tutorial - ros2 supervisor node. https://github.com/cyberbotics/webots_ros2/wiki/Tutorial-Ros2Supervisor-Node. Tutorial oficial.
 - [18] Cyberbotics. Webots automobile simulation. <https://cyberbotics.com/doc/automobile/index>. Documentación especializada.
 - [19] Cyberbotics. Webots cloud - remote simulation platform. <https://webots.cloud/>. Plataforma oficial.
 - [20] Cyberbotics. Webots reference manual. <https://cyberbotics.com/doc/reference/index>. Documentación técnica.
 - [21] Cyberbotics. Webots ros2 wiki - overview. https://github.com/cyberbotics/webots_ros2/wiki. Documentación comunitaria.

- [22] Cyberbotics. Webots user guide. <https://cyberbotics.com/doc/guide/index>. Documentación oficial.
- [23] Cyberbotics. Webotslauncher node reference. https://github.com/cyberbotics/webots_ros2/wiki/References-Nodes#WebotsLauncher. Documentación del nodo WebotsLauncher para ROS 2.
- [24] Cyberbotics. Worldinfo reference. <https://cyberbotics.com/doc/reference/worldinfo>. Documentación oficial del nodo WorldInfo en Webots.
- [25] Cyberbotics Ltd. Webots open source robot simulator. <https://github.com/cyberbotics/webots>. Repositorio open-source de Webots.
- [26] Cyberbotics Ltd. Webots ros 2 interface. https://github.com/cyberbotics/webots_ros2. Repositorio oficial de la integración ROS 2 para Webots.
- [27] Cyberbotics Ltd. Webots Discord Community — Canal de soporte técnico. <https://cyberbotics.com/doc/discord/news>, 2025. Información sobre el canal de Discord para soporte de Webots.
- [28] Cyberbotics Ltd. Webots R2025a — Página oficial. <https://cyberbotics.com/>, 2025. Página oficial de instalación e información.
- [29] Cyberbotics Ltd. Webots — Última versión R2025a. <https://en.wikipedia.org/wiki/Webots>, 2025. Descripción general y características del simulador.
- [30] Ministerio de Hacienda. Real decreto 1098/2001, de 12 de octubre, por el que se aprueba el reglamento general de la ley de contratos de las administraciones públicas. <https://www.boe.es/buscar/act.php?id=B0E-A-2001-19995>, 2001. Beneficio industrial.
- [31] Universidad de León. Presupuesto 2025. <https://www.unileon.es/files/2025-01/presupuesto2025nw.pdf>, 2025. Presupuestos Universidad de León 2025.
- [32] Discord Inc. Discord — Plataforma de comunicación. <https://en.wikipedia.org/wiki/Discord>, 2025. Definición y funcionalidades de Discord (voz, texto, vídeo).
- [33] Indeed España. Indeed. <https://es.indeed.com/>, 2025. Página de empleo en España.

- [34] Indeed España. Salarios en España. https://es.indeed.com/career/ingeniero-inform%C3%A1tico/salaries?from=top_sb, 2025. Salario de Ingeniero Informático.
- [35] Indeed España. Salarios en España. <https://es.indeed.com/career/scrum-master/salaries>, 2025. Salario de Scrum Master.
- [36] Europa Press. Robodk celebra una década de desarrollo en robótica. <https://www.europapress.es/comunicados/empresas-00908/noticia-comunicado-robodk-celebra-decada-desarrollo-robotica-20250425125610.html>. Comunicado de prensa de Europa Press.
- [37] Jarosław Foksa. Boxy svg – scalable vector graphics editor. <https://boxy-svg.com/>. sitio oficial del proyecto.
- [38] Formant. Best robot simulators. <https://formant.io/blog/best-robot-simulators/>. Artículo técnico.
- [39] Gazebo Team. Gazebo 6 api documentation. <https://gazebo-sim.org/api/gazebo/6/>. Documentación oficial de la API de Gazebo.
- [40] Gazebo Team. Gazebo sim repository. <https://github.com/gazebo-sim/gz-sim>. Repositorio oficial de Gazebo Sim.
- [41] Gazebo Team. Home – gazebo sim. <https://gazebo-sim.org/home>. Sitio oficial de Gazebo.
- [42] Gazebo Team. Ros 2 overview. https://classic.gazebo-sim.org/tutorials?tut=ros2_overview. Tutorial oficial ROS 2 para Gazebo Classic.
- [43] Giampaolo Rodola. psutil — Python system and process utilities. <https://psutil.readthedocs.io/>, 2025. Documentación oficial de psutil para monitorización de sistema y procesos.
- [44] Git SCM. Git — Distributed version control system. <https://git-scm.com/>, 2025. Sitio oficial de Git.
- [45] GitHub, Inc. GitHub — Platform for collaborative development. <https://github.com/features>, 2025. Descripción oficial de las características y uso de GitHub.
- [46] Google. Gmail — Servicio de correo electrónico. <https://workspace.google.com/intl/es/gmail/>, 2025. Sitio oficial de Gmail.

- [47] Google. Google Drive — Almacenamiento en la nube. <https://workspace.google.com/intl/es/products/drive/>, 2025. Descripción de la interfaz web y funcionalidades.
- [48] Google. Google Meet — Plataforma de videoconferencia. https://es.wikipedia.org/wiki/Google_Meet, 2025. Sitio oficial de Google Meet.
- [49] JASP Team. JASP — Software Estadístico. <https://jasp-stats.org/>, 2025. Sitio oficial del software JASP.
- [50] Jefatura del Estado. Ley 1/1996, de 12 de abril, de Propiedad Intelectual. <https://www.boe.es/buscar/act.php?id=BOE-A-1996-8930>, 1996. Publicada en el BOE núm. 97, de 22 de abril de 1996, páginas 14369 a 14377.
- [51] JGraph Ltd. Draw.io — Editor de diagramas en línea. <https://www.drawio.com/>, 2025. Herramienta gratuita para diagramas de flujo, UML, redes y más.
- [52] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2149–2154. IEEE, 2004.
- [53] MASKOR. webots_ros2_spot. https://github.com/MASKOR/webots_ros2_spot. Repositorio comunitario.
- [54] MathWorks Inc. Coppeliasim product details. https://www.mathworks.com/products/connections/product_detail/coppeliasim.html. Detalles del producto CoppeliaSim en MathWorks.
- [55] Microsoft Corporation. Visual Studio Code — Editor de código fuente. <https://code.visualstudio.com/>, 2025. Sitio oficial de descarga y documentación.
- [56] NVIDIA Corporation. Installing ros support. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/installation/install_ros.html. Guía oficial de instalación de soporte ROS en Isaac Sim.
- [57] NVIDIA Corporation. Isaac real time factor node. <https://docs.isaacsim.omniverse.nvidia.com/4.5.0/py/source/extensions/isaacsim.core.nodes/docs/ogn/OgnIsaacRealTimeFactor.html>. Documentación del nodo RTF en Isaac Sim.
- [58] NVIDIA Corporation. Isaac sim documentation (v4.5.0). <https://docs.isaacsim.omniverse.nvidia.com/4.5.0/index.html>. Página principal de la documentación de Isaac Sim.

-
- [59] NVIDIA Corporation. Isaac sim glossary. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/reference_material/reference_glossary.html#real-time-render-mode. Glosario oficial de términos de Isaac Sim.
- [60] NVIDIA Corporation. Isaac sim overview. <https://developer.nvidia.com/isaac/sim>. Página oficial de Isaac Sim de NVIDIA.
- [61] NVIDIA Corporation. Isaacsim-ros_workspaces. https://github.com/isaac-sim/IsaacSim-ros_workspaces. Repositorios con ejemplos y workspaces para Isaac Sim ROS.
- [62] NVIDIA Corporation. Ros 2 real time factor tutorial. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/ros2_tutorials/tutorial_ros2_rtf.html. Tutorial oficial de RTF con ROS 2 en Isaac Sim.
- [63] NVIDIA Corporation. Standalone python scripting. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/python_scripting/manual_standalone_python.html. Documentación oficial de scripts Python autónomos en Isaac Sim.
- [64] NVIDIA Corporation. Isaac Sim ROS 2 Navigation Tutorial — Nova Carter. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/ros2_tutorials/tutorial_ros2_navigation.html, 2025. Tutorial oficial de navegación con Nova Carter y Nav 2.
- [65] NVIDIA Corporation. NVIDIA Isaac Sim — ROS 2. https://docs.isaacsim.omniverse.nvidia.com/4.5.0/ros2_tutorials/ros2_landing_page.html, 2025. Documentación oficial Isaac Sim ROS 2.
- [66] NVIDIA Developer Forums. How faster does isaac sim can run than real-time? <https://forums.developer.nvidia.com/t/how-faster-does-isaac-sim-can-run-than-real-time/218288>. Discusión en foros de NVIDIA sobre RTF.
- [67] NVIDIA Developer Forums. Real time factor in isaac sim. <https://forums.developer.nvidia.com/t/real-time-factor-in-issac-sim/225784/5>. Discusión técnica sobre RTF en Isaac Sim.

- [68] NVIDIA Isaac ROS Team. Getting started with isaac sim. https://nvidia-isaac-ros.github.io/getting_started/isaac_sim/index.html. Documentación para empezar con Isaac Sim y ROS.
- [69] OneBrowsing. Comparativa simuladores. <https://onebrowsing.com/compare/coppeliastm-vs-gazebo-vs-nvidia-isaac-sim/>. Análisis comparativo.
- [70] Open Navigation Working Group. Navigation 2 — ROS 2 Navigation Stack. <https://docs.nav2.org/>, 2025. Documentación oficial del proyecto Nav2.
- [71] Open Robotics. Gazebo classic physics parameters. https://classic.gazebosim.org/tutorials?tut=physics_params. Tutorial oficial de parámetros físicos en Gazebo Classic.
- [72] Open Robotics. Ignition gazebo gui documentation. <https://gazebosim.org/docs/fortress/gui/>. Documentación oficial de la GUI en Ignition Gazebo.
- [73] Open Robotics. Ignition gazebo sdf worlds. https://gazebosim.org/docs/fortress/sdf_worlds/. Documentación sobre mundos SDF en Ignition Gazebo.
- [74] Open Robotics. ROS 2 — Humble Hawksbill (LTS). <https://docs.ros.org/en/humble/>, 2025. Documentación oficial de ROS 2 Humble.
- [75] Open Robotics. RViz — Visualizador 3D para ROS 2. <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-Main.html>, 2025. Documentación oficial de RViz para ROS 2 Humble.
- [76] Open Robotics. TurtleBot 4 Simulator — Integración con Ignition Gazebo. https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html, 2025. Ejemplo de uso con navegación Nav2 en Ignition Gazebo.
- [77] Open Robotics & Clearpath Robotics. TurtleBot 4 — Plataforma educativa con ROS 2. <https://clearpathrobotics.com/turtlebot-4/>, 2025. Descripción general de TurtleBot 4 – web oficial.
- [78] Open Source Robotics Foundation. Ignition Gazebo Fortress — Versión 6.17.0. <https://gazebosim.org/api/gazebo/6/install.html>, 2025. Documentación oficial de instalación y uso.

- [79] Overleaf. Overleaf — Editor LaTeX en línea. <https://www.overleaf.com/learn/>, 2025. Plataforma en línea para la edición colaborativa de documentos LaTeX.
- [80] Python Software Foundation. Python — Lenguaje de programación. <https://www.python.org/>, 2025. Sitio web oficial del lenguaje Python.
- [81] QVIRO. Webots review. <https://qviro.com/product/cyberbotics-ltd/webots-cyberbotics/reviews>. Análisis independiente.
- [82] Reddit r/robotics. Gazebo vs webots. https://www.reddit.com/r/robotics/comments/zqzdyt/gazebo_vs_webots/?tl=es-es. Discusión comunitaria.
- [83] RoboDK Team. Planes y precios. <https://robodk.com/es/pricing#products-tab>. Información sobre licencias y versiones de RoboDK.
- [84] RoboDK Team. Software de simulación de robots y programación offline. <https://robodk.com/es/>. Sitio oficial de RoboDK.
- [85] Open Robotics. Gazebo classic. <https://classic.gazebosim.org/>, 2025. Documentación oficial de Gazebo Classic.
- [86] Open Robotics. Turtlebot 3 simulator — integración con gazebo classic. <https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>, 2025. Tutorial oficial de instalación y simulación de TurtleBot 3.
- [87] ROBOTIS & Open Source Robotics Foundation. TurtleBot 3 — Plataforma modular para ROS. <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>, 2025. Documentación oficial de TurtleBot 3.
- [88] ROS 2 Documentation. Setting up simulation with webots - advanced. <https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Webots/Setting-Up-Simulation-Webots-Advanced.html>. Guía oficial.
- [89] ROS 2 Documentation. Setting up simulation with webots - basic. <https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Webots/Setting-Up-Simulation-Webots-Basic.html#create-the-my-robot-urdf-file>. Guía oficial.
- [90] ROS 2 Documentation. Simulation with webots. <https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Webots/Simulation-Webots.html>. Documentación oficial ROS 2.

- [91] ROS 2 Documentation Team. Simulation with gazebo. <https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Simulation-Gazebo.html>. Documentación ROS 2 para simulación con Gazebo.
- [92] ROS Discourse. Why is gazebo very famous in the ros community? <https://discourse.ros.org/t/why-is-gazebo-very-famous-in-the-ros-community-what-about-webots/42459/3>. Foro de discusión.
- [93] Ken Schwaber and Jeff Sutherland. The scrum guide – the definitive guide to scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>, 2020. Versión oficial del Scrum Guide.
- [94] Scrum.org. What is scrum? — scrum.org learning module. <https://www.scrum.org/resources/what-scrum-module>, 2023. Módulo de introducción oficial a Scrum.
- [95] Segway Robotics & NVIDIA Corporation. Nova Carter — Plataforma AMR Segway/NVIDIA. <https://robotics.segway.com/nova-carter/>, 2025. Descripción oficial de la plataforma y características.
- [96] Semaiq. Simuladores de robots 2024. <https://semaiq.com/los-mejores-sofwarees-para-la-simulacion-de-robots-en-2024/>, 2024. Artículo comparativo.
- [97] SIA Webby. Compress png – free online png compression tool. <https://compresspng.com/>. Compresión de imágenes online.
- [98] The Construct. How to use webots with ros 2. <https://www.theconstruct.ai/how-to-use-webots-with-ros2-ros2-qa-235/>. Tutorial especializado.
- [99] The Construct. Low simulation performance / low real time factor in gazebo. <https://get-help.theconstruct.ai/t/low-simulation-performance-low-real-time-factor/30179>, 2025. Discusión.
- [100] The Document Foundation. LibreOffice — Suite Ofimática Libre. <https://www.libreoffice.org/>, 2025. Sitio oficial de LibreOffice.
- [101] Unión Europea. Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo de 27 de abril de 2016 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de

- estos datos (RGPD). <https://www.boe.es/doue/2016/119/L00001-00088.pdf>, 2016. Diario Oficial de la Unión Europea L 119, 4.5.2016, p. 1-88.
- [102] Wikipedia. Boxy svg. https://en.wikipedia.org/wiki/Boxy_SVG. versión 4.70.0 lanzada el 22 de mayo de 2025.
- [103] Wikipedia contributors. Gazebo (simulator). [https://en.wikipedia.org/wiki/Gazebo_\(simulator\)](https://en.wikipedia.org/wiki/Gazebo_(simulator)). Artículo de Wikipedia sobre Gazebo.
- [104] Wikipedia contributors. Diagrams.net (Draw.io) — Wikipedia. <https://en.wikipedia.org/wiki/Diagrams.net>, 2025. Información sobre licencia, historia y funcionalidades.
- [105] Wikipedia contributors. Git — Wikipedia. <https://en.wikipedia.org/wiki/Git>, 2025. Información general sobre Git.
- [106] Wikipedia contributors. GitHub — Wikipedia. <https://en.wikipedia.org/wiki/GitHub>, 2025. Información general y estadísticas sobre GitHub.
- [107] Wikipedia contributors. Google Chrome — Navegador web. https://en.wikipedia.org/wiki/Google_Chrome, 2025. Información sobre versión y características.

Anexo A

Control de versiones

En el marco del desarrollo de este Trabajo de Fin de Máster (TFM), se ha empleado GitHub como servicio de control de versiones para gestionar eficientemente el código generado. GitHub ha proporcionado una plataforma robusta que facilita la colaboración, el seguimiento de cambios y la gestión de versiones de manera integral.

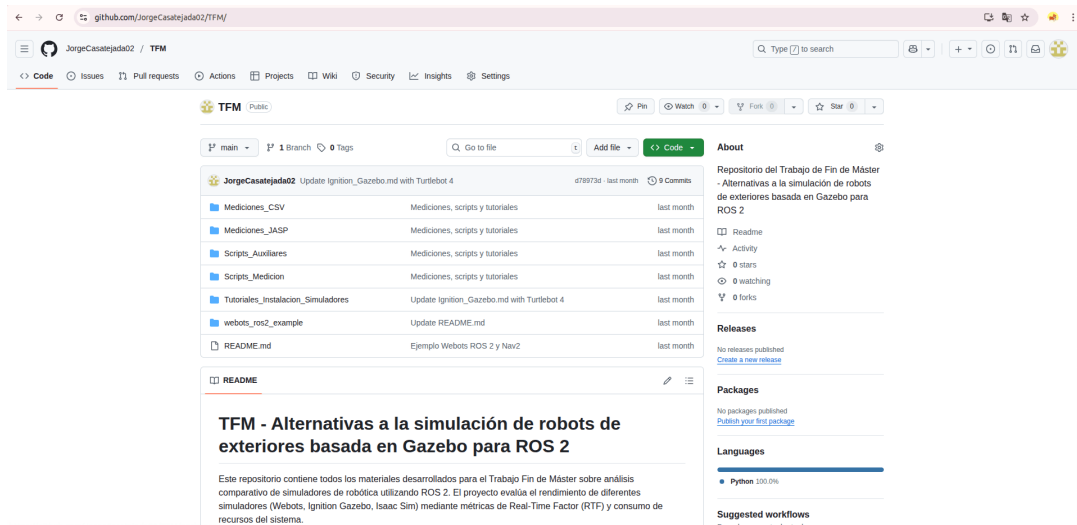


Figura A.1: Repositorio Github

Anexo B

Datos Experimentales Completos

B.1. Análisis Individual de Simuladores

B.1.1. Ignition Gazebo

Datos Ignition Gazebo respecto a estados de batería

Se exponen los datos de CPU, GPU, RAM y RTF respecto a la diferencia de usar Ignition Gazebo con el ordenador enchufado a la corriente cargando y sin estar enchufado a la corriente.

CPU

Tabla B.1: Ignition Gazebo - Estado batería - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	5888	5821	5888	5821	5888	5821
Mode	29.100	47.200	22.100	35.800	4.420	3.050
Median	36.300	44.100	28.100	33.400	3.830	2.350
Mean	35.414	39.747	27.030	29.980	3.626	2.386
Std. Deviation	9.347	11.022	6.685	7.621	0.697	0.438
Range	81.100	76.700	44.500	45.900	4.030	2.580
Minimum	18.600	23.200	9.000	11.500	0.400	0.490
Maximum	99.700	99.900	53.500	57.400	4.430	3.070
25th percentile	28.300	30.600	21.500	23.000	3.400	2.120
50th percentile	36.300	44.100	28.100	33.400	3.830	2.350

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
75th percentile	42.100	47.700	32.800	36.700	4.110	2.750
	load_avg_5		total_tasks		running_tasks	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	5888	5821	5888	5821	5888	5821
Mode	2.350	3.910	435.000	390.000	0.000	1.000
Median	3.660	4.090	435.000	395.000	1.000	1.000
Mean	3.994	4.353	434.143	406.629	1.223	1.385
Std. Deviation	2.167	1.967	9.073	20.906	1.315	1.396
Range	9.620	10.250	86.000	92.000	16.000	25.000
Minimum	0.160	0.110	367.000	361.000	0.000	0.000
Maximum	9.780	10.360	453.000	453.000	16.000	25.000
25th percentile	2.400	2.900	431.000	391.000	0.000	0.000
50th percentile	3.660	4.090	435.000	395.000	1.000	1.000
75th percentile	5.370	5.750	439.000	432.000	2.000	2.000

GPU

Tabla B.2: Ignition Gazebo - Estado batería - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	5833	5764	5833	5764	5833	5764
Mode	15.000	15.000	1.660	1.750	51.000	44.000
Median	15.000	15.000	6.190	2.235	51.000	45.000
Mean	171.883	173.581	8.353	6.788	51.607	45.554
Std. Deviation	181.527	182.712	30.150	5.815	2.023	3.006
Range	459.000	457.000	588.550	21.300	18.000	15.000
Minimum	15.000	15.000	1.460	1.650	40.000	38.000
Maximum	474.000	472.000	590.010	22.950	58.000	53.000
25th percentile	15.000	15.000	1.660	1.780	51.000	43.000
50th percentile	15.000	15.000	6.190	2.235	51.000	45.000
75th percentile	206.000	206.000	11.360	11.643	52.000	48.000

RAM

Tabla B.3: Ignition Gazebo - Estado batería - RAM

	active_MB		buffers_MB		cached_MB	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	6012	6029	6012	6029	6012	6029
Mode	3563.930	3536.540	285.500	285.380	2751.620	2742.590
Median	3728.605	3690.950	284.550	285.770	2782.375	2850.190
Mean	3845.245	3823.133	279.335	285.834	2984.579	3001.590
Std. Deviation	300.361	342.562	8.579	1.753	277.833	292.084
Range	2708.110	2568.710	44.660	27.290	1534.130	1543.250
Minimum	1719.740	1716.030	242.610	261.830	2376.400	2381.460
Maximum	4427.850	4284.740	287.270	289.120	3910.530	3924.710
25th percentile	3642.860	3560.820	268.450	285.300	2744.360	2741.540
50th percentile	3728.605	3690.950	284.550	285.770	2782.375	2850.190
75th percentile	4044.158	4059.270	285.520	286.500	3077.720	3063.470

RTF

Tabla B.4: Ignition Gazebo - Estado batería - RTF

	real_time_factor		iterations	
	cargando	sin_cargar	cargando	sin_cargar
Valid	5911	5867	5911	5867
Mode	0.640	0.451	2.000	2.000
Median	0.663	0.467	31362.000	20967.000
Mean	0.663	0.470	32102.504	21970.205
Std. Deviation	0.147	0.117	19575.479	13884.546
Range	0.997	0.825	85224.000	56624.000
Minimum	0.002	0.001	2.000	2.000
Maximum	0.999	0.826	85226.000	56626.000
25th percentile	0.558	0.385	15536.500	10320.500
50th percentile	0.663	0.467	31362.000	20967.000
75th percentile	0.772	0.553	47072.500	32076.000

Datos Ignition Gazebo respecto a GUI

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Ignition Gazebo con GUI o sin ella.

CPU**Tabla B.5:** Ignition Gazebo - Uso GUI - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	No	Sí	No	Sí	No	Sí
Valid	5874	5835	5874	5835	5874	5835
Mode	29.600	46.900	22.100	35.800	3.050	2.170
Median	29.200	46.000	22.200	35.400	2.970	2.850
Mean	29.632	45.557	22.122	34.913	3.051	2.969
Std. Deviation	7.047	6.443	3.467	3.637	0.918	0.776
Range	81.300	75.500	46.100	48.400	4.030	4.030
Minimum	18.600	24.400	10.000	9.000	0.400	0.400
Maximum	99.900	99.900	56.100	57.400	4.430	4.430
25th percentile	27.200	41.850	20.500	32.600	2.310	2.250
50th percentile	29.200	46.000	22.200	35.400	2.970	2.850
75th percentile	31.100	47.900	23.500	36.900	4.010	3.760

	load_avg_5		total_tasks		running_tasks	
	No	Sí	No	Sí	No	Sí
Valid	5874	5835	5874	5835	5874	5835
Mode	4.030	6.130	435.000	440.000	0.000	1.000
Median	3.170	5.360	430.000	433.000	1.000	1.000
Mean	3.138	5.214	417.853	423.093	1.057	1.551
Std. Deviation	1.237	2.226	20.680	21.322	1.181	1.476
Range	6.240	10.230	80.000	92.000	18.000	25.000
Minimum	0.110	0.130	364.000	361.000	0.000	0.000
Maximum	6.350	10.360	444.000	453.000	18.000	25.000
25th percentile	2.350	3.540	391.000	397.000	0.000	1.000
50th percentile	3.170	5.360	430.000	433.000	1.000	1.000
75th percentile	4.070	6.870	435.000	440.000	2.000	2.000

GPU

Tabla B.6: Ignition Gazebo - Uso GUI - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	No	Sí	No	Sí	No	Sí
Valid	5797	5800	5797	5800	5797	5800
Mode	15.000	15.000	1.670	1.660	51.000	51.000
Median	15.000	15.000	5.750	10.780	49.000	50.000
Mean	109.544	235.878	6.373	8.777	47.684	49.513
Std. Deviation	95.318	221.939	22.126	21.373	3.515	4.171
Range	191.000	459.000	588.480	588.550	15.000	20.000
Minimum	15.000	15.000	1.530	1.460	38.000	38.000
Maximum	206.000	474.000	590.010	590.010	53.000	58.000
25th percentile	15.000	15.000	1.710	1.730	44.000	45.000
50th percentile	15.000	15.000	5.750	10.780	49.000	50.000
75th percentile	206.000	461.000	10.360	11.960	51.000	53.000

RAM**Tabla B.7:** Ignition Gazebo - Uso GUI - RAM

	active_MB		buffers_MB		cached_MB	
	No	Sí	No	Sí	No	Sí
Valid	6023	6018	6023	6018	6023	6018
Mode	3536.540	3962.140	285.680	285.480	2751.620	2732.690
Median	3642.860	4057.475	285.050	285.655	2801.500	2850.480
Mean	3602.466	4066.073	282.107	283.071	2895.421	3090.853
Std. Deviation	186.500	256.116	7.130	6.811	161.254	342.923
Range	2154.950	2711.820	46.150	45.660	978.820	1548.310
Minimum	1729.070	1716.030	242.970	242.610	2396.570	2376.400
Maximum	3884.020	4427.850	289.120	288.270	3375.390	3924.710
25th percentile	3559.660	3990.882	283.975	284.870	2742.110	2741.980
50th percentile	3642.860	4057.475	285.050	285.655	2801.500	2850.480
75th percentile	3680.830	4179.045	285.640	286.540	3055.370	3452.785

RTF

Tabla B.8: Ignition Gazebo - Uso GUI - RTF

	real_time_factor		iterations	
	No	Sí	No	Sí
Valid	5890	5888	5890	5888
Mode	0.579	0.451	2.000	2.000
Median	0.610	0.509	28048.000	22859.000
Mean	0.620	0.514	29739.231	24370.418
Std. Deviation	0.168	0.142	19098.960	15774.243
Range	0.998	0.990	85224.000	65895.000
Minimum	0.001	0.001	2.000	2.000
Maximum	0.999	0.991	85226.000	65897.000
25th percentile	0.499	0.414	13889.500	11268.500
50th percentile	0.610	0.509	28048.000	22859.000
75th percentile	0.745	0.612	43007.500	35448.000

Batería

Tabla B.9: Ignition Gazebo - Uso GUI - Batería

	Watts		TCPU		x86_pk	
	No	Sí	No	Sí	No	Sí
Valid	3000	3000	3000	3000	3000	3000
Mode	34.040	39.250	51.050	52.050	51.000	52.000
Median	40.530	46.830	52.050	53.050	53.000	53.000
Mean	39.972	46.176	53.362	54.754	53.554	55.009
Std. Deviation	6.648	7.743	3.721	3.980	3.970	4.196
Range	48.500	53.010	21.000	21.000	34.000	29.000
Minimum	14.580	14.870	45.050	44.050	44.000	45.000
Maximum	63.080	67.880	66.050	65.050	78.000	74.000
25th percentile	34.040	39.250	50.050	52.050	51.000	52.000
50th percentile	40.530	46.830	52.050	53.050	53.000	53.000
75th percentile	46.813	51.143	57.050	59.050	57.000	59.000

Datos Ignition Gazebo respecto a GPU

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Ignition Gazebo con GPU o sin ella.

CPU

Tabla B.10: Ignition Gazebo - Uso GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	No	Sí	No	Sí	No	Sí
Valid	5851	5858	5851	5858	5851	5858
Mode	28.800	31.300	21.300	23.200	2.170	3.050
Median	36.200	38.300	28.300	29.300	2.750	3.000
Mean	36.775	38.361	28.045	28.948	2.934	3.085
Std. Deviation	11.393	9.326	8.179	6.306	0.939	0.745
Range	81.300	79.100	47.400	47.200	4.030	3.700
Minimum	18.600	20.800	9.000	10.200	0.400	0.730
Maximum	99.900	99.900	56.400	57.400	4.430	4.430
25th percentile	27.250	30.800	20.600	23.200	2.170	2.510
50th percentile	36.200	38.300	28.300	29.300	2.750	3.000
75th percentile	46.100	46.100	35.800	35.200	3.870	3.770
	load_avg_5		total_tasks		running_tasks	
	No	Sí	No	Sí	No	Sí
Valid	5851	5858	5851	5858	5851	5858
Mode	4.030	3.910	390.000	435.000	1.000	1.000
Median	3.850	4.060	428.000	435.000	1.000	1.000
Mean	4.162	4.183	417.269	423.657	1.326	1.281
Std. Deviation	2.066	2.089	20.409	21.424	1.360	1.357
Range	9.660	10.250	81.000	92.000	13.000	25.000
Minimum	0.120	0.110	364.000	361.000	0.000	0.000
Maximum	9.780	10.360	445.000	453.000	13.000	25.000
25th percentile	2.640	2.600	393.000	397.000	0.000	0.000
50th percentile	3.850	4.060	428.000	435.000	1.000	1.000
75th percentile	5.550	5.600	435.000	440.000	2.000	2.000

GPU

Tabla B.11: Ignition Gazebo - Uso GPU - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	No	Sí	No	Sí	No	Sí
Valid	5799	5798	5799	5798	5799	5798
Mode	15.000	206.000	1.660	11.270	51.000	51.000
Median	15.000	206.000	1.720	11.440	46.000	51.000
Mean	15.000	330.481	2.579	12.572	46.897	50.301
Std. Deviation	0.000	128.697	20.457	21.928	3.805	3.339
Range	0.000	459.000	588.480	588.550	15.000	20.000
Minimum	15.000	15.000	1.530	1.460	38.000	38.000
Maximum	15.000	474.000	590.010	590.010	53.000	58.000
25th percentile	15.000	206.000	1.660	10.100	44.000	48.000
50th percentile	15.000	206.000	1.720	11.440	46.000	51.000
75th percentile	15.000	461.000	1.790	12.460	51.000	53.000

RAM

Tabla B.12: Ignition Gazebo - Uso GPU - RAM

	used_MB		buffers_MB		cached_MB	
	No	Sí	No	Sí	No	Sí
Valid	6021	6020	6021	6020	6021	6020
Mode	3264.790	3793.700	285.380	285.550	3047.120	2732.690
Median	3364.300	3866.205	285.290	285.600	3077.340	2742.290
Mean	3431.012	4155.591	282.328	282.850	3233.380	2752.774
Std. Deviation	260.464	456.623	6.970	6.999	212.154	46.236
Range	2464.960	3253.120	44.250	46.510	1548.310	578.630
Minimum	1597.230	1572.640	243.020	242.610	2376.400	2381.250
Maximum	4062.190	4825.760	287.270	289.120	3924.710	2959.880
25th percentile	3261.070	3805.727	284.170	284.590	3054.870	2733.820
50th percentile	3364.300	3866.205	285.290	285.600	3077.340	2742.290
75th percentile	3650.980	4575.448	285.700	286.390	3452.770	2762.240

RTF

Tabla B.13: Ignition Gazebo - Uso GPU - RTF

	real_time_factor		iterations	
	No	Sí	No	Sí
Valid	5882	5896	5882	5896
Mode	0.338	0.513	2.000	2.000
Median	0.480	0.627	21767.000	29574.000
Mean	0.495	0.639	23437.545	30664.426
Std. Deviation	0.142	0.153	15350.760	19134.053
Range	0.962	0.998	65051.000	85224.000
Minimum	0.001	0.001	2.000	2.000
Maximum	0.963	0.999	65053.000	85226.000
25th percentile	0.389	0.529	10827.250	14707.500
50th percentile	0.480	0.627	21767.000	29574.000
75th percentile	0.588	0.748	33646.750	44592.750

Batería

Tabla B.14: Ignition Gazebo - Uso GPU - Batería

	Watts		TCPU		x86_pk	
	No	Sí	No	Sí	No	Sí
Valid	3000	3000	3000	3000	3000	3000
Mode	39.250	50.470	51.050	58.050	52.000	57.000
Median	37.350	49.990	51.050	58.050	51.000	58.000
Mean	36.812	49.336	50.908	57.208	51.092	57.472
Std. Deviation	3.355	5.804	1.858	2.710	2.216	3.026
Range	41.460	53.010	21.000	20.000	30.000	32.000
Minimum	14.580	14.870	45.050	44.050	44.000	46.000
Maximum	56.040	67.880	66.050	64.050	74.000	78.000
25th percentile	34.040	46.362	50.050	56.050	50.000	56.000
50th percentile	37.350	49.990	51.050	58.050	51.000	58.000
75th percentile	39.290	51.420	52.050	59.050	52.000	60.000

B.1.2. Webots

Datos Webots respecto a estados de batería

Se exponen los datos de CPU, GPU, RAM y RTF respecto a la diferencia de usar Webots con el ordenador enchufado a la corriente cargando y sin estar cargando.

CPU

Tabla B.15: Webots - Estado batería - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	5943	5901	5943	5901	5943	5901
Mode	4.700	6.200	3.700	4.900	2.060	1.480
Median	4.700	6.100	3.600	4.800	1.980	1.580
Mean	4.906	6.162	3.702	4.843	2.032	1.602
Std. Deviation	2.818	2.544	1.307	1.392	0.560	0.374
Range	37.400	37.900	17.700	23.600	3.930	2.640
Minimum	0.900	0.900	0.400	0.300	0.490	0.410
Maximum	38.300	38.800	18.100	23.900	4.420	3.050
25th percentile	4.200	5.500	3.200	4.300	1.640	1.340
50th percentile	4.700	6.100	3.600	4.800	1.980	1.580
75th percentile	5.000	6.600	4.000	5.300	2.370	1.840
	load_avg_5		total_tasks		running_tasks	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	5943	5901	5943	5901	5943	5901
Mode	0.630	0.640	388.000	345.000	0.000	0.000
Median	0.630	0.610	389.000	347.000	0.000	0.000
Mean	0.641	0.621	387.441	361.668	0.462	0.532
Std. Deviation	0.238	0.271	5.960	21.896	0.755	0.780
Range	1.180	1.300	56.000	64.000	7.000	7.000
Minimum	0.140	0.080	343.000	336.000	0.000	0.000
Maximum	1.320	1.380	399.000	400.000	7.000	7.000
25th percentile	0.460	0.420	387.000	345.000	0.000	0.000
50th percentile	0.630	0.610	389.000	347.000	0.000	0.000
75th percentile	0.810	0.780	391.000	389.000	1.000	1.000

GPU

Tabla B.16: Webots - Estado batería - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	Cargando	Sin cargar	Cargando	Sin cargar	Cargando	Sin cargar
Valid	5846	5767	5846	5767	5846	5767
Mode	15.000	15.000	1.470	1.800	43.000	41.000
Median	15.000	15.000	2.290	2.160	43.000	41.000
Mean	303.782	305.184	4.805	3.176	43.003	41.293
Std. Deviation	302.509	302.617	31.701	2.020	1.413	1.511
Range	701.000	701.000	588.570	14.300	10.000	7.000
Minimum	15.000	15.000	1.440	1.650	36.000	37.000
Maximum	716.000	716.000	590.010	15.950	46.000	44.000
25th percentile	15.000	15.000	1.500	1.830	42.000	40.000
50th percentile	15.000	15.000	2.290	2.160	43.000	41.000
75th percentile	679.000	679.000	5.070	5.170	44.000	43.000

RAM**Tabla B.17:** Webots - Estado batería - RAM

	active_MB		buffers_MB		cached_MB	
	Cargando	Sin cargar	Cargando	Sin cargar	Cargando	Sin cargar
Valid	6011	6022	6011	6022	6011	6022
Mode	2725.000	3110.730	283.210	285.520	2669.530	2772.050
Median	2942.420	2911.250	283.190	285.080	3161.020	2878.125
Mean	2913.632	2929.360	284.820	284.846	2982.557	3017.483
Std. Deviation	164.056	171.681	67.479	1.671	282.881	273.159
Range	1524.020	1440.010	788.050	25.870	1421.120	1430.070
Minimum	1785.610	1716.810	242.460	261.800	2385.660	2388.870
Maximum	3309.630	3156.820	1030.510	287.670	3806.780	3818.940
25th percentile	2795.260	2810.095	268.640	284.030	2701.565	2740.840
50th percentile	2942.420	2911.250	283.190	285.080	3161.020	2878.125
75th percentile	3033.950	3072.320	284.500	285.750	3263.540	3292.195

RTF

Tabla B.18: Webots - Estado batería - RTF

	real_time_factor		iterations	
	Sí	No	Sí	No
Valid	5914	5914	5914	5914
Mode	0.961	0.963	10.000	6872.000
Median	0.956	0.956	7209.500	7203.000
Mean	0.931	0.929	7208.342	7199.278
Std. Deviation	0.088	0.088	4167.237	4160.974
Range	0.955	0.940	14535.000	14407.000
Minimum	0.021	0.027	4.000	5.000
Maximum	0.976	0.967	14539.000	14412.000
25th percentile	0.938	0.937	3601.500	3600.000
50th percentile	0.956	0.956	7209.500	7203.000
75th percentile	0.963	0.962	10817.750	10798.500

Datos Webots respecto a GUI

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Webots con GUI o sin ella.

CPU

Tabla B.19: Webots - Uso GUI - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	No	Sí	No	Sí	No	Sí
Valid	5927	5917	5927	5917	5927	5917
Mode	4.400	4.700	3.700	3.700	1.640	1.680
Median	5.000	5.400	4.000	4.300	1.740	1.760
Mean	5.262	5.802	4.083	4.459	1.806	1.830
Std. Deviation	2.717	2.771	1.441	1.466	0.520	0.525
Range	36.600	37.900	22.700	23.600	3.560	4.010
Minimum	0.900	0.900	0.400	0.300	0.470	0.410
Maximum	37.500	38.800	23.100	23.900	4.030	4.420
25th percentile	4.300	4.800	3.300	3.700	1.445	1.470
50th percentile	5.000	5.400	4.000	4.300	1.740	1.760
75th percentile	5.800	6.400	4.700	5.000	2.090	2.120

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	No	Sí	No	Sí	No	Sí
	load_avg_5		total_tasks		running_tasks	
	No	Sí	No	Sí	No	Sí
Valid	5927	5917	5927	5917	5927	5917
Mode	0.620	0.660	389.000	345.000	0.000	0.000
Median	0.610	0.660	384.000	388.000	0.000	0.000
Mean	0.598	0.664	373.891	375.311	0.441	0.552
Std. Deviation	0.227	0.277	20.463	20.636	0.741	0.791
Range	1.090	1.300	64.000	56.000	7.000	7.000
Minimum	0.140	0.080	336.000	343.000	0.000	0.000
Maximum	1.230	1.380	400.000	399.000	7.000	7.000
25th percentile	0.430	0.470	347.000	346.000	0.000	0.000
50th percentile	0.610	0.660	384.000	388.000	0.000	0.000
75th percentile	0.750	0.850	390.000	391.000	1.000	1.000

GPU

Tabla B.20: Webots - Uso GUI - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	No	Sí	No	Sí	No	Sí
Valid	5817	5796	5817	5796	5817	5796
Mode	15.000	15.000	1.470	1.580	41.000	43.000
Median	15.000	15.000	2.290	4.950	41.000	43.000
Mean	362.996	245.749	3.323	4.672	41.280	43.031
Std. Deviation	349.323	232.521	24.399	20.510	1.458	1.443
Range	701.000	685.000	588.570	588.550	8.000	8.000
Minimum	15.000	15.000	1.440	1.460	36.000	38.000
Maximum	716.000	700.000	590.010	590.010	44.000	46.000
25th percentile	15.000	15.000	1.770	1.710	40.000	42.000
50th percentile	15.000	15.000	2.290	4.950	41.000	43.000
75th percentile	714.000	475.000	2.490	6.000	43.000	44.000

RAM

Tabla B.21: Webots - Uso GUI - RAM

	active_MB		buffers_MB		cached_MB	
	No	Sí	No	Sí	No	Sí
Valid	6022	6011	6022	6011	6022	6011
Mode	2725.000	3113.180	283.750	285.520	2669.530	2737.320
Median	2942.340	2911.480	284.150	284.110	2832.605	3160.390
Mean	2915.136	2927.882	281.904	287.767	2996.110	3003.969
Std. Deviation	163.394	172.449	6.517	67.055	290.763	265.811
Range	1447.810	1537.020	44.740	788.050	1340.720	1384.370
Minimum	1716.810	1772.610	242.930	242.460	2385.660	2434.570
Maximum	3164.620	3309.630	287.670	1030.510	3726.380	3818.940
25th percentile	2811.800	2786.880	283.120	283.140	2694.840	2731.770
50th percentile	2942.340	2911.480	284.150	284.110	2832.605	3160.390
75th percentile	3023.363	3072.850	285.440	285.340	3299.088	3261.635

RTF

Tabla B.22: Webots - Uso GUI - RTF

	real_time_factor		iterations	
	No	Sí	No	Sí
Valid	5914	5914	5914	5914
Mode	0.963	0.964	4049.000	10.000
Median	0.954	0.958	7189.000	7220.500
Mean	0.928	0.932	7190.741	7216.880
Std. Deviation	0.089	0.088	4156.211	4171.952
Range	0.945	0.941	14392.000	14531.000
Minimum	0.021	0.035	4.000	8.000
Maximum	0.966	0.976	14396.000	14539.000
25th percentile	0.936	0.939	3594.250	3606.750
50th percentile	0.954	0.958	7189.000	7220.500
75th percentile	0.961	0.964	10787.000	10828.500

Batería

Tabla B.23: Webots - Uso GUI - Batería

	Watts		TCPU		x86_pk	
	No	Sí	No	Sí	No	Sí
Valid	3000	3000	3000	3000	3000	3000
Mode	27.140	30.990	46.050	49.050	46.000	49.000
Median	27.770	30.150	46.050	49.050	46.000	49.000
Mean	28.091	30.367	43.918	48.566	46.554	48.847
Std. Deviation	2.732	2.402	6.560	1.818	3.451	2.540
Range	33.750	40.400	43.000	27.000	38.000	31.000
Minimum	21.130	17.780	25.050	42.050	41.000	42.000
Maximum	54.880	58.180	68.050	69.050	79.000	73.000
25th percentile	26.910	29.100	45.050	48.050	45.000	47.000
50th percentile	27.770	30.150	46.050	49.050	46.000	49.000
75th percentile	28.930	31.200	47.050	49.050	47.000	50.000

Datos Webots respecto a GPU

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Webots con GPU o sin ella.

CPU

Tabla B.24: Webots - Uso GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	No	Si	No	Si	No	Si
Valid	5923	5921	5923	5921	5923	5921
Mode	5.000	4.700	3.900	3.700	1.670	1.560
Median	5.200	5.200	4.200	4.000	1.770	1.730
Mean	5.600	5.464	4.361	4.180	1.841	1.795
Std. Deviation	2.789	2.724	1.496	1.429	0.530	0.514
Range	37.900	37.700	23.600	20.900	4.010	3.940
Minimum	0.900	0.900	0.300	0.400	0.410	0.470
Maximum	38.800	38.600	23.900	21.300	4.420	4.410
25th percentile	4.600	4.500	3.600	3.400	1.470	1.450
50th percentile	5.200	5.200	4.200	4.000	1.770	1.730
75th percentile	6.200	6.000	5.000	4.800	2.140	2.070

	load_avg_5		total_tasks		running_tasks	
	No	Si	No	Si	No	Si
Valid	5923	5921	5923	5921	5923	5921
Mode	0.630	0.770	389.000	393.000	0.000	0.000
Median	0.630	0.630	382.000	388.000	0.000	0.000
Mean	0.640	0.621	373.019	376.182	0.489	0.504
Std. Deviation	0.260	0.250	20.045	20.947	0.768	0.769
Range	1.300	1.200	63.000	57.000	7.000	6.000
Minimum	0.080	0.120	336.000	343.000	0.000	0.000
Maximum	1.380	1.320	399.000	400.000	7.000	6.000
25th percentile	0.450	0.440	346.000	348.000	0.000	0.000
50th percentile	0.630	0.630	382.000	388.000	0.000	0.000
75th percentile	0.840	0.770	389.000	392.000	1.000	1.000

GPU

Tabla B.25: Webots - Uso GPU - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	No	Sí	No	Sí	No	Sí
Valid	5814	5799	5814	5799	5814	5799
Mode	15.000	714.000	1.470	2.370	43.000	43.000
Median	15.000	679.000	1.720	5.010	42.000	43.000
Mean	15.000	594.706	2.770	5.226	41.654	42.655
Std. Deviation	0.000	122.682	23.160	21.857	1.586	1.651
Range	0.000	701.000	588.570	588.460	9.000	8.000
Minimum	15.000	15.000	1.440	1.550	36.000	38.000
Maximum	15.000	716.000	590.010	590.010	45.000	46.000
25th percentile	15.000	475.000	1.500	2.460	41.000	41.000
50th percentile	15.000	679.000	1.720	5.010	42.000	43.000
75th percentile	15.000	714.000	1.840	6.010	43.000	44.000

RAM

Tabla B.26: Webots - Uso GPU - RAM

	active_MB		buffers_MB		cached_MB	
	No	Sí	No	Sí	No	Sí
Valid	6017	6016	6017	6016	6017	6016
Mode	2725.000	3113.180	282.980	286.180	3252.540	2669.530
Median	2803.300	3065.750	283.930	284.500	3271.110	2730.225
Mean	2794.220	3048.808	281.410	288.256	3267.136	2732.892
Std. Deviation	97.438	120.863	6.372	66.995	87.390	69.805
Range	1284.770	1592.820	43.160	788.050	1433.280	781.180
Minimum	1772.610	1716.810	242.930	242.460	2385.660	2388.870
Maximum	3057.380	3309.630	286.090	1030.510	3818.940	3170.050
25th percentile	2768.980	3022.677	282.950	283.290	3251.800	2694.348
50th percentile	2803.300	3065.750	283.930	284.500	3271.110	2730.225
75th percentile	2824.320	3099.270	284.670	285.880	3299.610	2761.090

RTF**Tabla B.27:** Webots - Uso GPU - RTF

	real_time_factor		iterations	
	No	Sí	No	Sí
Valid	5918	5910	5918	5910
Mode	0.963	0.961	4437.000	3123.000
Median	0.957	0.954	7213.000	7189.000
Mean	0.933	0.927	7211.652	7195.958
Std. Deviation	0.085	0.092	4167.534	4160.662
Range	0.955	0.944	14535.000	14487.000
Minimum	0.021	0.027	4.000	5.000
Maximum	0.976	0.971	14539.000	14492.000
25th percentile	0.940	0.934	3606.750	3588.250
50th percentile	0.957	0.954	7213.000	7189.000
75th percentile	0.963	0.961	10819.500	10796.250

Batería

Tabla B.28: Webots - Uso GPU - Batería

	Watts		TCPU		x86_pk	
	No	Sí	No	Sí	No	Sí
Valid	3000	3000	3000	3000	3000	3000
Mode	27.140	30.990	46.050	47.050	49.000	49.000
Median	28.100	30.140	47.050	47.050	47.000	48.000
Mean	28.117	30.341	47.097	45.387	47.401	48.000
Std. Deviation	2.025	3.041	2.313	7.095	3.145	3.304
Range	30.660	32.910	26.000	44.000	38.000	37.000
Minimum	17.780	25.270	41.050	25.050	41.000	41.000
Maximum	48.440	58.180	67.050	69.050	79.000	78.000
25th percentile	26.990	28.850	46.050	46.050	45.000	46.000
50th percentile	28.100	30.140	47.050	47.050	47.000	48.000
75th percentile	29.160	31.220	49.050	49.050	49.000	49.000

B.1.3. Isaac Sim

Datos Isaac Sim respecto a estados de batería

Se exponen los datos de CPU, GPU, RAM y RTF respecto a la diferencia de usar Isaac Sim con el ordenador enchufado a la corriente cargando y sin estar cargando.

CPU

Tabla B.29: Isaac Sim - Estado Batería - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	2960	2942	2960	2942	2960	2942
Mode	13.200	0.900	11.100	0.400	4.420	3.050
Median	13.800	16.800	11.300	13.500	2.280	1.980
Mean	17.461	19.432	14.359	15.537	2.483	2.013
Std. Deviation	11.837	14.382	10.067	11.781	0.842	0.543
Range	98.400	97.700	85.700	81.100	3.760	2.530
Minimum	0.700	0.600	0.000	0.200	0.670	0.590
Maximum	99.100	98.300	85.700	81.300	4.430	3.120
25th percentile	12.600	14.000	10.400	11.000	1.860	1.650
50th percentile	13.800	16.800	11.300	13.500	2.280	1.980

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
75th percentile	18.300	20.000	14.900	16.100	3.030	2.370
	load_avg_5		total_tasks		running_tasks	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	2960	2942	2960	2942	2960	2942
Mode	1.350	1.900	401.000	355.000	0.000	0.000
Median	1.760	1.680	399.000	356.000	0.000	0.000
Mean	1.623	1.564	395.657	368.600	0.640	0.609
Std. Deviation	0.583	0.675	11.224	20.344	0.800	0.746
Range	2.660	3.030	55.000	55.000	6.000	5.000
Minimum	0.140	0.080	352.000	352.000	0.000	0.000
Maximum	2.800	3.110	407.000	407.000	6.000	5.000
25th percentile	1.280	1.160	394.000	354.000	0.000	0.000
50th percentile	1.760	1.680	399.000	356.000	0.000	0.000
75th percentile	2.020	2.040	401.000	393.000	1.000	1.000

GPU

Tabla B.30: Isaac Sim - Estado Batería - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	2926	2890	2926	2890	2926	2890
Mode	4200.000	4212.000	55.560	34.780	66.000	56.000
Median	4200.000	4200.000	55.875	34.710	65.000	55.000
Mean	3840.593	3441.866	53.603	29.428	63.281	52.043
Std. Deviation	1105.710	1489.735	34.042	10.521	5.470	5.959
Range	4229.000	3931.000	578.620	33.460	26.000	21.000
Minimum	15.000	281.000	11.390	1.670	43.000	37.000
Maximum	4244.000	4212.000	590.010	35.130	69.000	58.000
25th percentile	4200.000	4168.000	55.170	34.370	64.000	49.000
50th percentile	4200.000	4200.000	55.875	34.710	65.000	55.000
75th percentile	4212.000	4212.000	56.890	34.830	66.000	56.000

RAM

Tabla B.31: Isaac Sim - Estado Batería - RAM

	active_MB		buffers_MB		cached_MB	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	3000	3010	3000	3010	3000	3010
Mode	7396.520	3493.130	349.790	351.730	5619.810	5663.700
Median	7426.895	7364.690	349.800	351.635	5623.920	5590.995
Mean	7158.439	6821.576	345.433	350.343	5466.055	5250.940
Std. Deviation	960.986	1260.152	10.574	6.921	570.983	748.136
Range	5890.870	5873.580	106.650	85.950	3348.740	3365.780
Minimum	1694.450	1663.690	248.430	267.210	2389.050	2362.450
Maximum	7585.320	7537.270	355.080	353.160	5737.790	5728.230
25th percentile	7359.767	7262.740	334.340	350.610	5615.210	5564.595
50th percentile	7426.895	7364.690	349.800	351.635	5623.920	5590.995
75th percentile	7488.560	7419.703	350.950	352.110	5687.460	5656.615

RTF**Tabla B.32:** Isaac Sim - Estado Batería - RTF

	real_time_factor		iterations	
	cargando	sin_cargar	cargando	sin_cargar
Valid	2690	2351	2690	2351
Mode	0.194	0.178	610.000	1.000
Median	0.182	0.157	1843.500	1457.000
Mean	0.170	0.137	1851.818	1568.096
Std. Deviation	0.050	0.057	1113.529	1015.938
Range	0.257	0.211	4301.000	3682.000
Minimum	0.000	0.000	1.000	1.000
Maximum	0.257	0.211	4302.000	3683.000
25th percentile	0.151	0.090	879.500	666.000
50th percentile	0.182	0.157	1843.500	1457.000
75th percentile	0.201	0.182	2788.000	2468.000

Datos Isaac Sim respecto a GUI

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Isaac Sim con GUI o sin ella.

CPU**Tabla B.33:** Isaac Sim - Uso GUI - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	2950	2952	2950	2952	2950	2952
Mode	13.300	0.900	11.100	0.400	4.420	3.050
Median	14.800	15.100	11.900	12.000	2.120	2.100
Mean	18.244	18.644	14.706	15.186	2.272	2.226
Std. Deviation	12.984	13.418	10.742	11.188	0.733	0.759
Range	97.600	98.500	82.600	85.700	3.830	3.840
Minimum	0.700	0.600	0.000	0.000	0.600	0.590
Maximum	98.300	99.100	82.600	85.700	4.430	4.430
25th percentile	13.100	12.800	10.700	10.400	1.750	1.720
50th percentile	14.800	15.100	11.900	12.000	2.120	2.100
75th percentile	18.300	21.000	14.600	17.800	2.670	2.620
	load_avg_5		total_tasks		running_tasks	
	cargando	sin_cargar	cargando	sin_cargar	cargando	sin_cargar
Valid	2950	2952	2950	2952	2950	2952
Mode	1.350	1.900	353.000	355.000	0.000	0.000
Median	1.720	1.720	393.000	394.000	0.000	0.000
Mean	1.591	1.596	381.034	383.305	0.641	0.608
Std. Deviation	0.595	0.665	20.928	21.553	0.756	0.791
Range	2.610	2.990	54.000	55.000	6.000	6.000
Minimum	0.080	0.120	352.000	352.000	0.000	0.000
Maximum	2.690	3.110	406.000	407.000	6.000	6.000
25th percentile	1.240	1.260	355.000	357.000	0.000	0.000
50th percentile	1.720	1.720	393.000	394.000	0.000	0.000
75th percentile	2.060	1.970	398.000	401.000	1.000	1.000

GPU

Tabla B.34: Isaac Sim - Uso GUI - GPU

	gpu_mem_used_MB		gpu_power_W		gpu_temp_C	
	No	Sí	No	Sí	No	Sí
Valid	2908	2908	2908	2908	2908	2908
Mode	4212.000	4200.000	34.820	34.740	66.000	65.000
Median	4212.000	4200.000	34.920	34.890	57.000	56.000
Mean	3800.188	3484.738	42.821	40.359	58.560	56.833
Std. Deviation	1162.694	1453.829	27.009	28.911	7.214	8.661
Range	4229.000	4201.000	578.600	588.340	30.000	31.000
Minimum	15.000	15.000	11.410	1.670	39.000	37.000
Maximum	4244.000	4216.000	590.010	590.010	69.000	68.000
25th percentile	4212.000	4168.000	34.710	34.520	55.000	52.000
50th percentile	4212.000	4200.000	34.920	34.890	57.000	56.000
75th percentile	4212.000	4200.000	56.030	55.800	66.000	65.000

RAM

Tabla B.35: Isaac Sim - Uso GUI - RAM

	active_MB		buffers_MB		cached_MB	
	No	Sí	No	Sí	No	Sí
Valid	3005	3005	3005	3005	3005	3005
Mode	7394.340	3493.130	350.980	349.790	5663.700	5619.810
Median	7364.580	7446.460	350.990	350.030	5672.160	5591.770
Mean	7046.866	6932.588	348.327	347.458	5456.985	5259.652
Std. Deviation	1028.884	1226.462	9.348	9.159	617.503	713.076
Range	5846.860	5921.630	106.550	103.900	3349.740	3375.340
Minimum	1681.110	1663.690	248.530	248.430	2376.460	2362.450
Maximum	7527.970	7585.320	355.080	352.330	5726.200	5737.790
25th percentile	7309.470	7351.420	350.070	349.500	5651.900	5566.150
50th percentile	7364.580	7446.460	350.990	350.030	5672.160	5591.770
75th percentile	7405.250	7497.300	351.950	351.660	5695.300	5617.000

RTF

Tabla B.36: Isaac Sim - Uso GUI - RTF

	real_time_factor		iterations	
	No	Sí	No	Sí
Valid	2640	2401	2640	2401
Mode	0.189	0.194	1.000	140.000
Median	0.176	0.166	1779.500	1532.000
Mean	0.161	0.148	1791.778	1640.021
Std. Deviation	0.053	0.058	1097.915	1050.924
Range	0.257	0.225	4301.000	4013.000
Minimum	0.000	0.000	1.000	1.000
Maximum	0.257	0.225	4302.000	4014.000
25th percentile	0.136	0.111	826.000	723.000
50th percentile	0.176	0.166	1779.500	1532.000
75th percentile	0.192	0.194	2712.000	2542.000

Batería

Tabla B.37: Isaac Sim - Uso GUI - Batería

	Watts		TCPU		x86_pk	
	No	Sí	No	Sí	No	Sí
Valid	1500	1500	1500	1500	1500	1500
Mode	66.590	65.550	62.050	61.050	61.000	59.000
Median	66.080	65.835	61.050	60.050	60.000	59.000
Mean	63.640	58.561	59.610	58.188	59.765	58.160
Std. Deviation	7.508	15.316	4.324	7.329	5.230	7.653
Range	61.010	60.710	21.000	33.000	32.000	39.000
Minimum	14.580	14.940	44.050	40.050	44.000	39.000
Maximum	75.590	75.650	65.050	73.050	76.000	78.000
25th percentile	65.320	44.640	58.050	52.050	57.000	52.000
50th percentile	66.080	65.835	61.050	60.050	60.000	59.000
75th percentile	66.720	69.905	62.050	62.050	63.000	64.000

B.2. Comparativa entre Simuladores

B.2.1. Cargando - GUI - No GPU

Se exponen los datos de CPU, RAM y RTF respecto a la diferencia de usar Gazebo, Webots e Isaac Sim cargando, con GUI y sin GPU.

CPU

Tabla B.38: Comparación Simuladores Cargando - GUI - No GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1470	1484	1470	1484	1470	1484
Mode	44.700	4.900	34.300	3.900	3.870	2.250
Median	42.400	4.900	33.200	3.900	3.870	2.100
Mean	42.584	5.266	33.161	4.007	3.683	2.132
Std. Deviation	5.643	2.819	3.375	1.268	0.627	0.555
Range	75.100	37.400	44.500	16.700	4.030	3.930
Minimum	24.400	0.900	9.000	0.500	0.400	0.490
Maximum	99.500	38.300	53.500	17.200	4.430	4.420
25th percentile	39.900	4.700	31.400	3.600	3.660	1.730
50th percentile	42.400	4.900	33.200	3.900	3.870	2.100
75th percentile	44.700	5.200	35.000	4.200	4.000	2.462
	load_avg_5		total_tasks		running_tasks	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1470	1484	1470	1484	1470	1484
Mode	3.920	1.000	437.000	388.000	1.000	0.000
Median	5.780	0.660	436.000	388.000	1.000	0.000
Mean	5.534	0.682	432.370	386.917	1.488	0.507
Std. Deviation	2.422	0.234	10.589	3.233	1.440	0.772
Range	9.590	1.010	74.000	46.000	11.000	7.000
Minimum	0.190	0.160	371.000	348.000	0.000	0.000
Maximum	9.780	1.170	445.000	394.000	11.000	7.000
25th percentile	3.690	0.530	432.000	387.000	0.000	0.000
50th percentile	5.780	0.660	436.000	388.000	1.000	0.000
75th percentile	7.780	0.840	438.000	389.000	2.000	1.000

RAM

Tabla B.39: Comparación Simuladores Cargando - GUI - No GPU - RAM

	active_MB		buffers_MB		cached_MB	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1503	1500	1503	1500	1503	1500
Mode	3927.340	2804.540	285.300	284.640	3345.020	3258.440
Median	3967.310	2801.570	285.370	284.020	3399.580	3265.155
Mean	3957.681	2791.779	280.321	278.756	3394.528	3261.767
Std. Deviation	198.490	72.587	8.437	7.776	114.123	73.519
Range	2471.100	1129.960	44.180	41.840	1534.130	1350.490
Minimum	1719.740	1854.790	243.090	243.050	2376.400	2456.290
Maximum	4190.840	2984.750	287.270	284.890	3910.530	3806.780
25th percentile	3956.810	2792.508	269.100	268.860	3354.605	3257.975
50th percentile	3967.310	2801.570	285.370	284.020	3399.580	3265.155
75th percentile	4012.045	2806.858	285.530	284.570	3442.555	3271.123

RTF

Tabla B.40: Comparación Simuladores Cargando - GUI - No GPU - RTF

	real_time_factor	
	Ignition	Webots
Valid	1478	1481
Mode	0.503	0.970
Median	0.537	0.965
Mean	0.546	0.944
Std. Deviation	0.101	0.078
Range	0.817	0.916
Minimum	0.002	0.060
Maximum	0.819	0.976
25th percentile	0.479	0.951
50th percentile	0.537	0.965
75th percentile	0.605	0.971

B.2.2. Cargando - GUI - GPU

Se exponen los datos de CPU, GPU, RAM y RTF respecto a la diferencia de usar Gazebo, Webots e Isaac Sim cargando, con GUI y con GPU.

CPU

Tabla B.41: Comparación Simuladores Cargando - GUI - GPU - CPU

	cpu_percent			cpu_freq_MHz		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1467	1480	1485	1467	1480	1485
Mode	40.700	13.200	4.400	3.870	4.420	1.560
Median	41.700	14.000	4.600	3.600	2.280	1.900
Mean	42.657	17.940	4.906	3.431	2.484	1.972
Std. Deviation	5.408	11.739	2.734	0.567	0.832	0.556
Range	74.900	98.200	33.300	3.200	3.700	3.900
Minimum	24.700	0.900	2.400	1.230	0.730	0.510
Maximum	99.600	99.100	35.700	4.430	4.430	4.410
25th percentile	40.300	12.700	4.200	3.050	1.880	1.590
50th percentile	41.700	14.000	4.600	3.600	2.280	1.900
75th percentile	44.000	19.300	5.000	3.830	3.010	2.300
	load_avg_5			running_tasks		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1467	1480	1485	1467	1480	1485
Mode	2.380	1.850	0.680	1.000	0.000	0.000
Median	4.930	1.840	0.680	1.000	0.000	0.000
Mean	4.795	1.727	0.695	1.404	0.651	0.507
Std. Deviation	2.050	0.585	0.256	1.414	0.832	0.770
Range	7.980	2.660	1.160	16.000	6.000	6.000
Minimum	0.200	0.140	0.160	0.000	0.000	0.000
Maximum	8.180	2.800	1.320	16.000	6.000	6.000
25th percentile	3.200	1.410	0.490	0.000	0.000	0.000
50th percentile	4.930	1.840	0.680	1.000	0.000	0.000
75th percentile	6.260	2.050	0.850	2.000	1.000	1.000

GPU

Tabla B.42: Comparación Simuladores Cargando - GUI - GPU - GPU

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1462	1463	1458	1462	1463	1458
Mode	39.000	100.000	34.000	457.000	4200.000	475.000
Median	39.000	100.000	34.000	457.000	4200.000	475.000
Mean	37.880	90.694	33.021	452.975	3857.081	476.169
Std. Deviation	6.472	27.943	6.983	44.615	1064.272	45.048
Range	74.000	100.000	53.000	459.000	4201.000	685.000
Minimum	0.000	0.000	0.000	15.000	15.000	15.000
Maximum	74.000	100.000	53.000	474.000	4216.000	700.000
25th percentile	36.000	99.000	31.000	457.000	4200.000	475.000
50th percentile	39.000	100.000	34.000	457.000	4200.000	475.000
75th percentile	40.000	100.000	36.000	461.000	4200.000	475.000
	gpu_power_W			gpu_temp_C		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1462	1463	1458	1462	1463	1458
Mode	11.270	55.560	6.050	54.000	65.000	45.000
Median	11.930	55.790	5.630	54.000	65.000	45.000
Mean	15.533	53.764	7.112	54.382	63.164	44.674
Std. Deviation	26.489	33.868	26.528	1.510	5.356	0.512
Range	588.550	578.620	588.460	13.000	25.000	4.000
Minimum	1.460	11.390	1.550	45.000	43.000	42.000
Maximum	590.010	590.010	590.010	58.000	68.000	46.000
25th percentile	11.310	55.130	5.410	53.000	64.000	44.000
50th percentile	11.930	55.790	5.630	54.000	65.000	45.000
75th percentile	14.953	56.635	6.027	55.000	66.000	45.000

RAM

Tabla B.43: Comparación Simuladores Cargando - GUI - GPU - RAM

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1502	1500	1501	1502	1500	1501

Mode	32.900	50.600	22.100	4140.580	7396.520	3113.180
Median	33.000	50.600	21.900	4181.815	7486.510	3098.400
Mean	32.882	49.010	21.815	4171.588	7213.668	3083.625
Std. Deviation	1.758	5.669	0.682	225.979	934.071	106.122
Range	21.600	37.100	10.700	2644.890	5869.710	1458.870
Minimum	13.700	15.500	13.700	1782.960	1715.610	1850.760
Maximum	35.300	52.600	24.400	4427.850	7585.320	3309.630
25th percentile	32.800	50.100	21.500	4155.443	7395.285	3084.870
50th percentile	33.000	50.600	21.900	4181.815	7486.510	3098.400
75th percentile	33.500	50.800	22.100	4248.602	7512.152	3105.850
buffers_MB			cached_MB			
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1502	1500	1501	1502	1500	1501
Mode	284.790	349.790	283.210	2732.690	5619.810	2737.320
Median	284.825	349.690	283.190	2740.690	5616.930	2734.590
Mean	279.793	344.786	302.739	2744.449	5443.993	2744.024
Std. Deviation	8.511	10.273	132.765	35.169	549.838	79.631
Range	44.240	101.920	788.050	467.150	3348.740	694.460
Minimum	242.610	248.430	242.460	2381.250	2389.050	2475.590
Maximum	286.850	350.350	1030.510	2848.400	5737.790	3170.050
25th percentile	269.360	333.660	266.890	2732.710	5610.780	2724.360
50th percentile	284.825	349.690	283.190	2740.690	5616.930	2734.590
75th percentile	286.360	349.960	283.450	2765.245	5622.325	2737.320

RTF

Tabla B.44: Comparación Simuladores Cargando - GUI - GPU - RTF

real_time_factor			
	Ignition	Isaac Sim	Webots
Valid	1479	1355	1477
Mode	0.640	0.194	0.964
Median	0.670	0.185	0.955
Mean	0.654	0.170	0.926
Std. Deviation	0.115	0.046	0.095

	real_time_factor		
	Ignition	Isaac Sim	Webots
Range	0.989	0.224	0.936
Minimum	0.002	0.001	0.035
Maximum	0.991	0.225	0.971
25th percentile	0.594	0.154	0.934
50th percentile	0.670	0.185	0.955
75th percentile	0.734	0.198	0.963

B.2.3. Cargando - No GUI - No GPU

Se exponen los datos de CPU, RAM y RTF respecto a la diferencia de usar Gazebo, Webots e Isaac Sim cargando, sin GUI y sin GPU.

CPU

Tabla B.45: Comparación Simuladores Cargando - No GUI - No GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1477	1488	1477	1488	1477	1488
Mode	26.900	4.400	18.500	3.400	4.420	2.060
Median	25.600	4.400	19.500	3.500	3.920	1.980
Mean	26.269	4.668	19.786	3.564	3.600	2.019
Std. Deviation	6.406	2.886	3.276	1.314	0.875	0.562
Range	81.000	36.500	40.400	17.700	4.020	3.390
Minimum	18.600	0.900	10.000	0.400	0.400	0.640
Maximum	99.600	37.400	50.400	18.100	4.420	4.030
25th percentile	23.700	3.900	18.000	3.000	3.200	1.640
50th percentile	25.600	4.400	19.500	3.500	3.920	1.980
75th percentile	27.500	4.900	21.100	3.900	4.260	2.340
	load_avg_5		total_tasks		running_tasks	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1477	1488	1477	1488	1477	1488
Mode	2.770	0.620	432.000	382.000	0.000	0.000
Median	2.640	0.610	432.000	382.000	1.000	0.000

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Mean	2.595	0.590	429.232	383.347	0.970	0.419
Std. Deviation	1.006	0.221	7.635	5.604	1.136	0.756
Range	4.360	0.780	74.000	49.000	8.000	6.000
Minimum	0.190	0.160	367.000	343.000	0.000	0.000
Maximum	4.550	0.940	441.000	392.000	8.000	6.000
25th percentile	1.970	0.400	428.000	381.000	0.000	0.000
50th percentile	2.640	0.610	432.000	382.000	1.000	0.000
75th percentile	3.250	0.820	433.000	389.000	2.000	1.000

RAM

Tabla B.46: Comparación Simuladores Cargando - No GUI - No GPU - RAM

	active_MB		buffers_MB		cached_MB	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1504	1505	1504	1505	1504	1505
Mode	3563.930	2725.000	284.610	267.040	3064.320	3207.500
Median	3572.160	2744.510	283.250	282.950	3064.550	3256.450
Mean	3571.374	2781.943	278.012	278.128	3056.365	3248.522
Std. Deviation	152.279	109.132	8.709	7.872	66.155	89.382
Range	1973.140	1271.770	41.870	42.430	953.890	1340.720
Minimum	1824.160	1785.610	243.020	242.930	2421.500	2385.660
Maximum	3797.300	3057.380	284.890	285.360	3375.390	3726.380
25th percentile	3566.217	2725.810	266.700	267.230	3057.145	3201.070
50th percentile	3572.160	2744.510	283.250	282.950	3064.550	3256.450
75th percentile	3592.742	2807.000	284.550	283.210	3068.613	3286.360

RTF

Tabla B.47: Comparación Simuladores Cargando - No GUI - No GPU - RTF

	real_time_factor	
	Ignition	Webots
Valid	1476	1480
Mode	0.640	0.963

	real_time_factor	
	Ignition	Webots
Median	0.634	0.955
Mean	0.642	0.930
Std. Deviation	0.112	0.085
Range	0.961	0.944
Minimum	0.002	0.021
Maximum	0.963	0.965
25th percentile	0.569	0.938
50th percentile	0.634	0.955
75th percentile	0.715	0.961

B.2.4. Cargando - No GUI - GPU

Se exponen los datos de CPU, GPU, RAM y RTF respecto a la diferencia de usar Gazebo, Webots e Isaac Sim cargando, sin GUI y con GPU.

CPU

Tabla B.48: Comparación Simuladores Cargando - No GUI - GPU - CPU

	cpu_percent			cpu_freq_MHz		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1474	1480	1486	1474	1480	1486
Mode	29.600	13.300	4.700	4.250	4.420	1.850
Median	29.500	13.600	4.500	4.050	2.290	1.960
Mean	30.220	16.983	4.785	3.791	2.482	2.007
Std. Deviation	5.724	11.920	2.799	0.628	0.851	0.555
Range	78.900	95.500	34.200	3.700	3.760	3.490
Minimum	20.800	0.700	2.200	0.730	0.670	0.540
Maximum	99.700	96.200	36.400	4.430	4.430	4.030
25th percentile	28.700	12.500	3.900	3.580	1.848	1.620
50th percentile	29.500	13.600	4.500	4.050	2.290	1.960
75th percentile	30.400	17.700	5.000	4.210	3.075	2.350
	load_avg_5			running_tasks		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1474	1480	1486	1474	1480	1486

	cpu_percent			cpu_freq_MHz		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Mode	2.890	1.350	0.620	0.000	0.000	0.000
Median	3.250	1.520	0.620	1.000	0.000	0.000
Mean	3.066	1.520	0.597	1.033	0.629	0.413
Std. Deviation	1.372	0.563	0.220	1.166	0.767	0.716
Range	5.700	2.470	1.010	14.000	6.000	5.000
Minimum	0.160	0.220	0.140	0.000	0.000	0.000
Maximum	5.860	2.690	1.150	14.000	6.000	5.000
25th percentile	2.090	1.210	0.430	0.000	0.000	0.000
50th percentile	3.250	1.520	0.620	1.000	0.000	0.000
75th percentile	4.190	1.970	0.740	2.000	1.000	1.000

GPU

Tabla B.49: Comparación Simuladores Cargando - No GUI - GPU - GPU

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1455	1463	1463	1455	1463	1463
Mode	33.000	100.000	17.000	206.000	4212.000	714.000
Median	33.000	100.000	18.000	206.000	4212.000	714.000
Mean	33.836	89.085	18.212	203.852	3824.104	709.352
Std. Deviation	7.553	30.292	3.612	18.456	1145.777	52.486
Range	90.000	100.000	24.000	191.000	4229.000	701.000
Minimum	0.000	0.000	0.000	15.000	15.000	15.000
Maximum	90.000	100.000	24.000	206.000	4244.000	716.000
25th percentile	30.000	99.000	17.000	206.000	4212.000	714.000
50th percentile	33.000	100.000	18.000	206.000	4212.000	714.000
75th percentile	37.000	100.000	20.000	206.000	4244.000	714.000

	gpu_power_W			gpu_temp_C		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1455	1463	1463	1455	1463	1463
Mode	6.260	55.670	2.370	51.000	66.000	43.000
Median	7.440	56.010	2.350	51.000	66.000	43.000
Mean	11.173	53.441	4.751	51.199	63.397	42.675

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Std. Deviation	34.126	34.227	34.342	0.813	5.581	0.518
Range	584.260	578.600	587.780	8.000	24.000	4.000
Minimum	5.750	11.410	2.230	45.000	45.000	40.000
Maximum	590.010	590.010	590.010	53.000	69.000	44.000
25th percentile	6.300	55.215	2.320	51.000	64.000	42.000
50th percentile	7.440	56.010	2.350	51.000	66.000	43.000
75th percentile	11.310	57.020	2.380	52.000	66.000	43.000

RAM

Tabla B.50: Comparación Simuladores Cargando - No GUI - GPU - RAM

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1503	1500	1505	1503	1500	1505
Mode	28.200	49.800	23.600	3675.700	7369.160	2995.820
Median	28.300	49.900	23.600	3696.460	7375.125	3007.430
Mean	28.148	48.183	23.511	3680.734	7103.210	2997.227
Std. Deviation	1.101	6.047	0.729	161.478	984.384	105.647
Range	13.700	36.400	8.500	2052.830	5833.520	1369.660
Minimum	16.300	15.400	16.200	1831.190	1694.450	1794.960
Maximum	30.000	51.800	24.700	3884.020	7527.970	3164.620
25th percentile	28.200	49.700	23.500	3682.510	7343.708	2997.480
50th percentile	28.300	49.900	23.600	3696.460	7375.125	3007.430
75th percentile	28.400	50.200	23.700	3710.155	7441.517	3022.790

	buffers_MB			cached_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1503	1500	1505	1503	1500	1505
Mode	285.680	351.020	283.250	2751.620	5683.300	2669.530
Median	284.320	350.950	283.560	2746.750	5687.080	2678.550
Mean	279.214	346.080	279.686	2742.767	5488.118	2676.207
Std. Deviation	8.494	10.831	8.072	27.847	590.732	28.882
Range	42.910	106.550	43.430	399.480	3302.380	383.770
Minimum	242.970	248.530	242.950	2431.550	2409.840	2401.320

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Maximum	285.880	355.080	286.380	2831.030	5712.220	2785.090
25th percentile	268.430	334.360	269.460	2736.300	5679.890	2669.530
50th percentile	284.320	350.950	283.560	2746.750	5687.080	2678.550
75th percentile	285.600	351.220	285.880	2751.620	5705.825	2694.550

RTF

Tabla B.51: Comparación Simuladores Cargando - No GUI - GPU - RTF

	real_time_factor		
	Ignition	Isaac Sim	Webots
Valid	1478	1335	1476
Mode	0.798	0.189	0.960
Median	0.828	0.179	0.952
Mean	0.812	0.171	0.923
Std. Deviation	0.117	0.053	0.094
Range	0.997	0.257	0.925
Minimum	0.002	0.000	0.040
Maximum	0.999	0.257	0.965
25th percentile	0.765	0.147	0.930
50th percentile	0.828	0.179	0.952
75th percentile	0.881	0.210	0.959

B.2.5. Sin Cargar - GUI - No GPU

Se exponen los datos de CPU, RAM, RTF y batería respecto a la diferencia de usar Gazebo, Webots e Isaac Sim sin el ordenador cargando conectado a la corriente, con GUI y sin GPU.

CPU

Tabla B.52: Comparación Simuladores Sin Cargar - GUI - No GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1439	1468	1439	1468	1439	1468
Mode	49.000	6.600	38.400	5.300	2.170	1.480
Median	48.600	6.600	37.500	5.300	2.170	1.595
Mean	49.271	6.664	37.730	5.313	2.201	1.616
Std. Deviation	5.879	1.744	2.435	1.150	0.271	0.362
Range	61.000	36.500	27.700	23.600	2.230	2.640
Minimum	38.900	1.100	28.700	0.300	0.820	0.410
Maximum	99.900	37.600	56.400	23.900	3.050	3.050
25th percentile	46.900	6.100	36.300	4.900	2.090	1.360
50th percentile	48.600	6.600	37.500	5.300	2.170	1.595
75th percentile	49.900	7.000	38.700	5.700	2.270	1.840
	load_avg_5		total_tasks		running_tasks	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1439	1468	1439	1468	1439	1468
Mode	3.620	0.400	393.000	345.000	1.000	0.000
Median	5.160	0.650	394.000	345.000	1.000	0.000
Mean	4.941	0.675	406.479	360.380	1.762	0.593
Std. Deviation	2.022	0.337	19.853	21.560	1.505	0.790
Range	8.140	1.300	55.000	55.000	10.000	5.000
Minimum	0.130	0.080	389.000	343.000	0.000	0.000
Maximum	8.270	1.380	444.000	398.000	10.000	5.000
25th percentile	3.260	0.390	392.000	345.000	1.000	0.000
50th percentile	5.160	0.650	394.000	345.000	1.000	0.000
75th percentile	6.490	0.950	433.000	387.000	3.000	1.000

RAM

Tabla B.53: Comparación Simuladores Sin Cargar - GUI - No GPU - RAM

	active_MB		buffers_MB		cached_MB	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1501	1500	1501	1500	1501	1500

	active_MB		buffers_MB		cached_MB	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Mode	3956.070	2770.320	286.840	283.710	3449.160	3252.540
Median	4007.940	2776.680	285.500	284.040	3458.560	3252.975
Mean	3982.052	2792.605	285.724	284.379	3449.839	3257.774
Std. Deviation	203.326	91.512	0.965	1.040	109.682	76.787
Range	2238.670	1121.750	9.580	8.660	1436.680	1372.120
Minimum	1840.870	1807.070	277.550	277.430	2488.030	2446.820
Maximum	4079.540	2928.820	287.130	286.090	3924.710	3818.940
25th percentile	3983.540	2770.037	285.120	283.580	3454.460	3242.280
50th percentile	4007.940	2776.680	285.500	284.040	3458.560	3252.975
75th percentile	4036.500	2786.340	286.600	285.392	3464.510	3274.283

RTF

Tabla B.54: Comparación Simuladores Sin Cargar - GUI - No GPU - RTF

	real_time_factor	
	Ignition	Webots
Valid	1461	1478
Mode	0.318	0.964
Median	0.352	0.956
Mean	0.353	0.931
Std. Deviation	0.065	0.085
Range	0.544	0.893
Minimum	0.001	0.073
Maximum	0.545	0.966
25th percentile	0.312	0.938
50th percentile	0.352	0.956
75th percentile	0.395	0.963

Batería

Tabla B.55: Comparación Simuladores Sin Cargar - GUI - No GPU - Batería

	Watts		TCPU		x86_pk	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1500	1500	1500	1500	1500	1500
Mode	39.250	28.960	52.050	49.050	52.000	49.000
Median	39.250	29.120	52.050	49.050	52.000	49.000
Mean	39.307	29.247	51.545	48.533	51.731	48.778
Std. Deviation	1.903	1.674	1.739	1.742	2.113	2.264
Range	37.080	30.520	19.000	22.000	29.000	27.000
Minimum	15.350	17.780	46.050	43.050	45.000	42.000
Maximum	52.430	48.300	65.050	65.050	74.000	69.000
25th percentile	38.700	28.670	51.050	48.050	51.000	47.000
50th percentile	39.250	29.120	52.050	49.050	52.000	49.000
75th percentile	39.682	29.630	53.050	49.050	53.000	50.000

B.2.6. Sin Cargar - GUI - GPU

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Gazebo, Webots e Isaac Sim sin el ordenador cargando conectado a la corriente, con GUI y con GPU.

CPU

Tabla B.56: Comparación Simuladores Sin Cargar - GUI - GPU - CPU

	cpu_percent			cpu_freq_MHz		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1449	1467	1470	1449	1467	1470
Mode	47.200	0.900	6.200	3.050	3.050	1.420
Median	47.200	17.000	6.100	2.580	1.960	1.580
Mean	47.964	19.318	6.170	2.549	1.966	1.596
Std. Deviation	5.577	14.902	1.850	0.339	0.570	0.377
Range	65.200	95.500	34.700	1.650	2.530	2.310
Minimum	34.700	0.600	1.100	1.400	0.590	0.520
Maximum	99.900	96.100	35.800	3.050	3.120	2.830
25th percentile	46.500	13.500	5.600	2.280	1.570	1.340
50th percentile	47.200	17.000	6.100	2.580	1.960	1.580

75th percentile	48.100	22.100	6.500	2.850	2.340	1.840
	load_avg_5			running_tasks		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1449	1467	1470	1449	1467	1470
Mode	3.910	0.190	0.770	1.000	0.000	0.000
Median	5.850	1.620	0.640	1.000	0.000	0.000
Mean	5.620	1.469	0.607	1.557	0.560	0.583
Std. Deviation	2.239	0.711	0.263	1.525	0.736	0.791
Range	10.220	2.970	1.120	25.000	5.000	6.000
Minimum	0.140	0.140	0.120	0.000	0.000	0.000
Maximum	10.360	3.110	1.240	25.000	5.000	6.000
25th percentile	4.020	0.980	0.440	1.000	0.000	0.000
50th percentile	5.850	1.620	0.640	1.000	0.000	0.000
75th percentile	7.150	1.890	0.780	2.000	1.000	1.000

GPU

Tabla B.57: Comparación Simuladores Sin Cargar - GUI - GPU - GPU

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1437	1440	1430	1437	1440	1430
Mode	39.000	100.000	30.000	461.000	4200.000	475.000
Median	37.000	99.000	32.000	461.000	4200.000	475.000
Mean	35.979	72.464	32.462	460.359	3117.533	477.748
Std. Deviation	6.361	42.824	5.530	8.155	1675.635	18.277
Range	54.000	100.000	49.000	201.000	3908.000	198.000
Minimum	18.000	0.000	5.000	271.000	292.000	475.000
Maximum	72.000	100.000	54.000	472.000	4200.000	673.000
25th percentile	32.000	18.000	29.000	461.000	590.000	475.000
50th percentile	37.000	99.000	32.000	461.000	4200.000	475.000
75th percentile	40.000	100.000	35.000	461.000	4200.000	475.000
	gpu_power_W			gpu_temp_C		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1437	1440	1430	1437	1440	1430

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Mode	11.900	34.740	6.300	50.000	56.000	43.000
Median	11.950	34.640	6.240	50.000	54.000	43.000
Mean	14.128	26.835	6.262	48.963	50.465	42.536
Std. Deviation	4.014	12.443	1.001	2.218	6.314	1.178
Range	12.980	33.460	10.890	14.000	21.000	6.000
Minimum	9.970	1.670	4.380	39.000	37.000	38.000
Maximum	22.950	35.130	15.270	53.000	58.000	44.000
25th percentile	11.660	12.250	5.930	48.000	43.000	42.000
50th percentile	11.950	34.640	6.240	50.000	54.000	43.000
75th percentile	16.020	34.780	6.430	50.000	56.000	43.000

RAM

Tabla B.58: Comparación Simuladores Sin Cargar - GUI - GPU - RAM

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1502	1500	1500	1502	1500	1500
Mode	33.300	50.600	22.000	4177.030	3493.130	3077.380
Median	33.200	50.400	22.000	4176.660	7411.380	3064.185
Mean	32.981	45.686	21.940	4168.143	6669.034	3050.910
Std. Deviation	1.950	8.644	0.627	237.109	1378.835	97.964
Range	21.400	38.100	9.900	2534.990	5840.840	1285.940
Minimum	14.100	13.900	14.100	1749.750	1696.430	1811.760
Maximum	35.500	52.000	24.000	4284.740	7537.270	3097.700
25th percentile	33.100	40.900	21.900	4168.477	5905.955	3053.205
50th percentile	33.200	50.400	22.000	4176.660	7411.380	3064.185
75th percentile	33.300	50.700	22.100	4256.405	7467.712	3071.445

	buffers_MB			cached_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1502	1500	1500	1502	1500	1500
Mode	286.590	350.800	285.660	2742.590	3518.110	2727.270
Median	286.510	351.660	285.590	2742.935	5577.830	2730.950
Mean	286.588	350.391	285.350	2778.847	5084.949	2756.004

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Std. Deviation	0.920	5.114	0.860	61.058	790.089	60.044
Range	10.470	68.790	8.580	565.930	3349.160	481.000
Minimum	277.800	283.540	277.780	2393.950	2379.070	2482.120
Maximum	288.270	352.330	286.360	2959.880	5728.230	2963.120
25th percentile	286.180	350.600	285.220	2737.915	4334.032	2725.307
50th percentile	286.510	351.660	285.590	2742.935	5577.830	2730.950
75th percentile	286.790	352.000	285.920	2845.640	5584.670	2738.910

RTF

Tabla B.59: Comparación Simuladores Sin Cargar - GUI - GPU - RTF

	real_time_factor		
	Ignition	Isaac Sim	Webots
Valid	1470	1046	1478
Mode	0.448	0.166	0.963
Median	0.499	0.131	0.956
Mean	0.500	0.120	0.929
Std. Deviation	0.078	0.061	0.091
Range	0.825	0.207	0.922
Minimum	0.001	0.000	0.045
Maximum	0.826	0.207	0.967
25th percentile	0.455	0.066	0.937
50th percentile	0.499	0.131	0.956
75th percentile	0.548	0.173	0.962

Batería

Tabla B.60: Comparación Simuladores Sin Cargar - GUI - GPU - Batería

	Watts			TCPU		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1500	1500	1500	1500	1500	1500
Mode	50.600	65.550	30.990	60.050	61.050	49.050

	Watts			TCPU		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Median	51.110	65.835	31.170	59.050	60.050	49.050
Mean	53.045	58.561	31.488	57.963	58.188	48.599
Std. Deviation	4.679	15.316	2.495	2.838	7.329	1.890
Range	53.010	60.710	29.650	19.000	33.000	27.000
Minimum	14.870	14.940	28.530	44.050	40.050	42.050
Maximum	67.880	75.650	58.180	63.050	73.050	69.050
25th percentile	50.378	44.640	30.720	57.050	52.050	48.050
50th percentile	51.110	65.835	31.170	59.050	60.050	49.050
75th percentile	55.300	69.905	31.640	60.050	62.050	50.050

B.2.7. Sin Cargar - No GUI - No GPU

Se exponen los datos de CPU, RAM, RTF y batería respecto a la diferencia de usar Gazebo, Webots e Isaac Sim sin el ordenador cargando conectado a la corriente, sin GUI y sin GPU.

CPU

Tabla B.61: Comparación Simuladores Sin Cargar - No GUI - No GPU - CPU

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1455	1473	1455	1473	1455	1473
Mode	28.800	5.800	21.300	4.700	3.050	1.310
Median	28.400	5.700	21.400	4.600	2.250	1.580
Mean	29.292	5.602	21.792	4.484	2.237	1.596
Std. Deviation	7.518	1.691	3.381	1.185	0.500	0.375
Range	76.700	33.600	38.800	21.600	2.560	2.530
Minimum	23.200	2.600	17.300	1.500	0.490	0.470
Maximum	99.900	36.200	56.100	23.100	3.050	3.000
25th percentile	27.100	4.800	20.400	3.800	1.940	1.320
50th percentile	28.400	5.700	21.400	4.600	2.250	1.580
75th percentile	29.700	6.200	22.400	5.100	2.610	1.840
	load_avg_5		total_tasks		running_tasks	
	Ignition	Webots	Ignition	Webots	Ignition	Webots

	cpu_percent		cpu_user_percent		cpu_freq_MHz	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1455	1473	1455	1473	1455	1473
Mode	4.030	0.630	390.000	344.000	1.000	0.000
Median	3.820	0.610	390.000	346.000	1.000	0.000
Mean	3.625	0.618	400.825	361.082	1.090	0.423
Std. Deviation	0.886	0.217	18.426	22.472	1.178	0.709
Range	6.230	1.040	53.000	63.000	13.000	7.000
Minimum	0.120	0.150	387.000	336.000	0.000	0.000
Maximum	6.350	1.190	440.000	399.000	13.000	7.000
25th percentile	3.030	0.460	389.000	344.000	0.000	0.000
50th percentile	3.820	0.610	390.000	346.000	1.000	0.000
75th percentile	4.160	0.710	419.000	389.000	2.000	1.000

RAM

Tabla B.62: Comparación Simuladores Sin Cargar - No GUI - No GPU - RAM

	active_MB		buffers_MB		cached_MB	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1503	1502	1503	1502	1503	1502
Mode	3536.540	2823.070	285.680	283.930	3038.320	3291.040
Median	3538.900	2825.460	285.610	284.330	3046.080	3308.215
Mean	3522.823	2817.057	285.390	284.508	3038.434	3305.794
Std. Deviation	154.703	74.006	0.796	0.798	67.992	71.677
Range	1818.260	1010.990	8.660	8.000	935.860	1222.670
Minimum	1760.720	1851.960	277.830	278.000	2408.900	2501.250
Maximum	3578.980	2862.950	286.490	286.000	3344.760	3723.920
25th percentile	3531.415	2817.408	285.230	283.990	3038.575	3300.032
50th percentile	3538.900	2825.460	285.610	284.330	3046.080	3308.215
75th percentile	3551.760	2833.877	285.870	285.108	3053.650	3316.475

RTF

Tabla B.63: Comparación Simuladores Sin Cargar - No GUI - No GPU - RTF

	real_time_factor	
	Ignition	Webots
Valid	1467	1479
Mode	0.419	0.963
Median	0.434	0.955
Mean	0.436	0.928
Std. Deviation	0.076	0.090
Range	0.633	0.938
Minimum	0.002	0.027
Maximum	0.635	0.965
25th percentile	0.389	0.937
50th percentile	0.434	0.955
75th percentile	0.484	0.961

Batería

Tabla B.64: Comparación Simuladores Sin Cargar - No GUI - No GPU - Batería

	Watts		TCPU		x86_pk	
	Ignition	Webots	Ignition	Webots	Ignition	Webots
Valid	1500	1500	1500	1500	1500	1500
Mode	34.040	27.140	51.050	46.050	51.000	45.000
Median	34.040	27.000	51.050	46.050	51.000	45.000
Mean	34.317	26.988	50.271	45.661	50.453	46.023
Std. Deviation	2.536	1.688	1.753	1.883	2.130	3.297
Range	41.460	27.310	21.000	26.000	25.000	38.000
Minimum	14.580	21.130	45.050	41.050	44.000	41.000
Maximum	56.040	48.440	66.050	67.050	69.000	79.000
25th percentile	33.330	26.255	49.050	45.050	49.000	45.000
50th percentile	34.040	27.000	51.050	46.050	51.000	45.000
75th percentile	34.740	27.580	51.050	46.050	52.000	46.000

B.2.8. Sin Cargar - No GUI - GPU

Se exponen los datos de CPU, GPU, RAM, RTF y batería respecto a la diferencia de usar Gazebo, Webots e Isaac Sim sin el ordenador cargando conectado a la corriente, sin GUI y con GPU.

CPU

Tabla B.65: Comparación Simuladores Sin Cargar - No GUI - GPU - CPU

	cpu_percent			cpu_freq_MHz		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1458	1465	1470	1458	1465	1470
Mode	32.200	17.600	6.200	3.050	3.050	1.480
Median	32.000	16.800	5.800	2.665	2.000	1.570
Mean	32.814	19.479	5.790	2.568	2.058	1.602
Std. Deviation	6.845	13.873	1.643	0.445	0.511	0.382
Range	76.100	97.400	33.100	2.020	2.460	2.510
Minimum	23.800	0.900	0.900	1.050	0.600	0.470
Maximum	99.900	98.300	34.000	3.070	3.060	2.980
25th percentile	31.225	14.200	5.200	2.232	1.690	1.340
50th percentile	32.000	16.800	5.800	2.665	2.000	1.570
75th percentile	32.800	18.500	6.300	2.980	2.440	1.840

	load_avg_5			running_tasks		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1458	1465	1470	1458	1465	1470
Mode	2.720	2.040	0.460	1.000	0.000	0.000
Median	3.120	1.770	0.560	1.000	1.000	0.000
Mean	3.295	1.669	0.590	1.134	0.645	0.501
Std. Deviation	1.354	0.613	0.245	1.238	0.732	0.769
Range	5.920	2.500	1.070	18.000	5.000	6.000
Minimum	0.110	0.080	0.160	0.000	0.000	0.000
Maximum	6.030	2.580	1.230	18.000	5.000	6.000
25th percentile	2.400	1.290	0.420	0.000	0.000	0.000
50th percentile	3.120	1.770	0.560	1.000	1.000	0.000
75th percentile	4.250	2.100	0.750	2.000	1.000	1.000

GPU

Tabla B.66: Comparación Simuladores Sin Cargar - No GUI - GPU - GPU

	gpu_util_percent			gpu_mem_used_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1434	1440	1438	1434	1440	1438
Mode	29.000	99.000	18.000	206.000	4212.000	714.000
Median	30.000	99.000	19.000	206.000	4212.000	714.000
Mean	31.013	88.068	18.678	205.449	3788.110	713.970
Std. Deviation	8.189	31.540	2.523	6.876	1163.362	0.737
Range	79.000	100.000	26.000	151.000	3931.000	23.000
Minimum	9.000	0.000	1.000	55.000	281.000	693.000
Maximum	88.000	100.000	27.000	206.000	4212.000	716.000
25th percentile	27.000	99.000	17.000	206.000	4212.000	714.000
50th percentile	30.000	99.000	19.000	206.000	4212.000	714.000
75th percentile	32.000	100.000	20.000	206.000	4212.000	714.000
	gpu_power_W			gpu_temp_C		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1434	1440	1438	1434	1440	1438
Mode	6.390	34.820	2.560	48.000	57.000	41.000
Median	10.560	34.760	2.570	47.000	56.000	41.000
Mean	9.418	32.137	2.701	46.644	53.714	40.732
Std. Deviation	2.948	7.167	1.061	1.841	4.989	1.055
Range	10.380	23.290	10.720	9.000	19.000	5.000
Minimum	4.970	11.790	2.400	40.000	39.000	38.000
Maximum	15.350	35.080	13.120	49.000	58.000	43.000
25th percentile	6.440	34.600	2.530	46.000	53.000	40.000
50th percentile	10.560	34.760	2.570	47.000	56.000	41.000
75th percentile	11.560	34.850	2.610	48.000	57.000	41.000

RAM

Tabla B.67: Comparación Simuladores Sin Cargar - No GUI - GPU - RAM

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1503	1500	1500	1503	1500	1500

	percent			active_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Mode	28.200	50.000	23.500	3669.200	7394.340	3110.730
Median	28.200	49.800	23.500	3666.270	7329.545	3101.120
Mean	28.079	47.805	23.458	3647.263	7008.363	3071.877
Std. Deviation	1.342	6.385	0.758	167.327	1025.777	109.011
Range	15.300	37.600	10.700	1916.680	5726.050	1404.430
Minimum	14.100	13.700	14.000	1788.060	1713.570	1752.390
Maximum	29.400	51.300	24.700	3704.740	7439.620	3156.820
25th percentile	28.100	49.500	23.400	3657.690	7282.850	3087.428
50th percentile	28.200	49.800	23.500	3666.270	7329.545	3101.120
75th percentile	28.300	50.100	23.600	3675.950	7381.592	3110.863
	buffers_MB			cached_MB		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1503	1500	1500	1503	1500	1500
Mode	285.070	350.700	283.750	2739.540	5663.700	2772.050
Median	285.480	351.600	285.625	2739.920	5656.610	2770.830
Mean	285.941	350.841	285.435	2747.145	5436.040	2757.077
Std. Deviation	1.452	5.017	1.263	35.691	618.557	62.497
Range	11.310	69.880	9.780	416.870	3333.160	510.440
Minimum	277.810	283.280	277.890	2439.480	2393.040	2402.410
Maximum	289.120	353.160	287.670	2856.350	5726.200	2912.850
25th percentile	285.140	350.620	284.295	2732.550	5649.032	2761.090
50th percentile	285.480	351.600	285.625	2739.920	5656.610	2770.830
75th percentile	285.790	352.260	286.200	2744.315	5664.325	2780.430

RTF**Tabla B.68:** Comparación Simuladores Sin Cargar - No GUI - GPU - RTF

	<i>real_time_factor</i>		
	Ignition	Isaac Sim	Webots
Valid	1469	1305	1479
Mode	0.535	0.178	0.964
Median	0.594	0.172	0.955
Mean	0.588	0.151	0.929

	<i>real_time_factor</i>		
	Ignition	Isaac Sim	Webots
Std. Deviation	0.093	0.050	0.088
Range	0.819	0.211	0.939
Minimum	0.001	0.000	0.027
Maximum	0.821	0.211	0.966
25th percentile	0.542	0.126	0.937
50th percentile	0.594	0.172	0.955
75th percentile	0.646	0.187	0.962

Batería

Tabla B.69: Comparación Simuladores Sin Cargar - No GUI - GPU - Batería

	Watts			TCPU		
	Ignition	Isaac Sim	Webots	Ignition	Isaac Sim	Webots
Valid	1500	1500	1500	1500	1500	1500
Mode	41.860	66.590	28.720	58.050	62.050	47.050
Median	46.580	66.080	28.860	57.050	61.050	46.050
Mean	45.627	63.640	29.193	56.453	59.610	42.175
Std. Deviation	4.240	7.508	3.106	2.346	4.324	8.745
Range	48.140	61.010	29.610	18.000	21.000	43.000
Minimum	14.940	14.580	25.270	46.050	44.050	25.050
Maximum	63.080	75.590	54.880	64.050	65.050	68.050
25th percentile	41.850	65.320	28.137	55.050	58.050	44.050
50th percentile	46.580	66.080	28.860	57.050	61.050	46.050
75th percentile	48.530	66.720	29.480	58.050	62.050	47.050

Anexo C

Tecnologías utilizadas

Durante el desarrollo de este Trabajo Fin de Máster se ha hecho uso de una amplia variedad de tecnologías tanto de software como de hardware, esenciales para la simulación, el análisis de datos, el control de robots móviles y la documentación. A continuación, se enumeran y describen en detalle.

Simuladores robóticos

Ignition Gazebo (Fortress) Simulador de robots en 3D desarrollado por Open Robotics, que integra un conjunto modular de bibliotecas optimizadas para simulación avanzada. Permite la simulación de sensores, dinámica y control de robots, siendo especialmente útil para evaluar rendimiento y algoritmos de navegación en entornos virtuales realistas [78].

- **Versión:** Gazebo Sim 6.17.0 (Ignition Gazebo Fortress)
- **Uso:** Empleado para medir su uso de recursos computacionales en un ejemplo de navegación autónoma, específicamente de TurtleBot 4 usando ROS 2 y Nav2 [76] y también para realizar un tutorial de instalación tanto del simulador como del ejemplo de Turtlebot 4.

Gazebo Classic Simulador de robots 3D ampliamente utilizado en la comunidad de robótica, especialmente en entornos educativos y de investigación. Su integración con ROS y ROS 2 permite la simulación de robots en entornos realistas, facilitando el desarrollo y prueba de algoritmos de navegación y control. Se encuentra discontinuado desde enero de 2025 [85].

- **Versión:** Gazebo multi-robot simulator, version 11.10.2

- **Uso:** Realizado un tutorial que abarca tanto la instalación del simulador como la configuración del ejemplo de TurtleBot 3 [86].

Webots Simulador 3D libre y de código abierto para robots, con soporte para múltiples plataformas (Windows, Linux, macOS). Desarrollado por Cyberbotics desde 1996 y liberado bajo licencia Apache 2.0 en 2018, ofrece un entorno completo para modelar, simular y controlar robots, incluyendo sensores, actuadores y dinámicas físicas realistas basadas en ODE. Webots también permite programar controladores en C++, Python, ROS y otros lenguajes [29] [28].

- **Versión:** Webots R2025a
- **Uso:** Realizado un tutorial de instalación y configuración del simulador junto con ejemplo de TurtleBot 3, evaluado el uso de recursos computacionales en un escenario de navegación autónoma con ROS 2 y Nav2 y también empleado para el desarrollo de un ejemplo de robótica en exteriores [26].

Isaac Sim Simulador de robótica en 3D basada en NVIDIA Omniverse, que ofrece simulación física acelerada por GPU (usando PhysX y RTX), sensores realistas y puentes integrados con ROS 2. Está enfocado para uso profesional y requiere de hardware potente para alcanzar un rendimiento adecuado [58].

- **Versión:** 4.5.0
- **Uso:** Realizado un tutorial de instalación y configuración del simulador, se ha hecho uso del ejemplo “Nova Carter Navigation” y para medir el uso de recursos computacionales en un escenario de navegación autónoma con ROS 2 y Nav2 [64] [65] .

Robots simulados

TurtleBot (3 y 4) Plataformas móviles de código abierto concebidas para educación e investigación en robótica, altamente integradas con ROS y ROS 2. TurtleBot 3 fue lanzado en 2017 por ROBOTIS y la Open Source Robotics Foundation, destaca por su diseño modular, personalizable y económico, permitiendo experimentación con SLAM, navegación y control en escenarios académicos y de prototipado [87]. Su sucesor, TurtleBot 4, desarrollado por Open Robotics y Clearpath Robotics, ofrece potencia de cómputo mejorada y sensores avanzados (OAK-D, LiDAR, IMU) con ROS 2 y simulación en Ignition

Gazebo, conservando una experiencia accesible para principiantes y robusta para investigadores [77] [76].

- **Versión:** TurtleBot 3 (modelos Burger y Waffle); TurtleBot 4 (Standard y Lite)
- **Uso:** Empleados para probar los simuladores (Gazebo, Webots, Isaac Sim), midiendo rendimiento de navegación autónoma con ROS 2 y Nav2. TurtleBot 3 fue utilizado en tutoriales con Gazebo Classic, mientras que TurtleBot 4 con Ignition Gazebo [86] [76].

Nova Carter Plataforma de robot móvil autónomo (AMR) desarrollada por Segway Robotics en colaboración con NVIDIA, diseñada para investigación e implementación en robótica avanzada. Cuenta con un chasis diferencial RMP Lite 220, potencia de cómputo basada en Jetson AGX Orin y un completo conjunto de sensores (LiDAR 2D/3D, cámaras estéreo y ojo de pez, IMU), lo que permite simulaciones realistas con sincronización precisa y gran capacidad de procesamiento en Isaac Sim con puente ROS 2 y Nav2 [95].

- **Uso:** Empleado para probar Isaac Sim mediante el tutorial “Nova Carter Navigation”, midiendo el rendimiento de navegación autónoma y uso de recursos [64].

Desarrollo

Visual Studio Code Editor de código fuente multiplataforma desarrollado por Microsoft, ampliamente utilizado en desarrollo de software por su ligereza, extensibilidad mediante extensiones y compatibilidad con múltiples lenguajes. Cuenta con integración con terminal, depuración, control de versiones y soporte avanzado para Python, C++, YAML, entre otros, lo que lo convierte en una herramienta ideal para entornos de desarrollo robótico y científico [55].

- **Versión:** 1.101.2
- **Uso:** Utilizado como entorno principal para desarrollar los scripts de medición de uso de CPU, GPU, RAM y energía durante la ejecución de los simuladores. También fue empleado para modificar mapas, ficheros de configuración y launchers ROS 2 de los distintos simuladores, así como para el desarrollo del ejemplo de robótica de exteriores en Webots.

Python Lenguaje de programación de alto nivel, interpretado y de propósito general, ampliamente adoptado en ciencia de datos, inteligencia artificial y robótica por su sintaxis clara, extenso ecosistema de librerías y compatibilidad con múltiples plataformas. Python facilita el desarrollo rápido de scripts, prototipos y sistemas complejos [80].

- **Versión:** 3.10.12
- **Uso:** Utilizado para desarrollar los scripts que miden el uso de CPU, RAM, GPU y energía durante la ejecución de simulaciones, así como para implementar el ejemplo de robótica de exteriores en Webots.

ROS 2 Conjunto de herramientas de middleware y desarrollo de software diseñado para la programación de sistemas robóticos. ROS 2 (Robot Operating System 2) es la evolución del sistema original ROS, con mejoras en comunicación distribuida, soporte en tiempo real, seguridad y compatibilidad con microcontroladores. Es un estándar de facto en investigación y desarrollo en robótica, especialmente en simulación, percepción y navegación autónoma [74].

- **Versión:** Humble Hawksbill
- **Uso:** Utilizado en todos los ejemplos de simulación (Gazebo, Webots, Isaac Sim) para gestionar nodos, sensores y algoritmos de navegación. También se empleó en el desarrollo del ejemplo de robótica de exteriores en Webots.

Nav2 Conjunto de paquetes de navegación para ROS 2 que implementa algoritmos de localización, planificación global y local, control de movimiento y evitación de obstáculos en entornos 2D. Nav2 (Navigation 2) es el sucesor directo del sistema de navegación de ROS 1 y permite implementar soluciones de navegación autónoma en robots móviles, integrándose estrechamente con sensores, mapas y controladores dentro del ecosistema ROS 2 [70].

- **Versión:** 1.1.18
- **Uso:** Utilizado junto con ROS 2 Humble para todos los experimentos de navegación autónoma en los simuladores Gazebo, Webots e Isaac Sim. Se empleó tanto en la medición del consumo de recursos computacionales durante el proceso de navegación como en el desarrollo del ejemplo de robótica de exteriores en Webots.

RViz Herramienta de visualización para el sistema operativo de robots (ROS 2), utilizada para representar datos sensoriales, mapas, trayectorias y estados de robots en entornos simulados o reales. RViz permite la interacción en tiempo real con los datos del robot, facilitando el desarrollo, depuración y análisis de algoritmos de percepción, planificación y control. Es una herramienta esencial en el ecosistema ROS 2 para la visualización de información en aplicaciones de robótica móvil y autónoma [75].

- **Versión:** 11.2.17

- **Uso:** Empleada en conjunto con ROS 2 Humble y Nav2 para visualizar en tiempo real los datos de sensores, mapas y trayectorias durante las simulaciones de navegación autónoma. También se utilizó para desarrollar y depurar el ejemplo de robótica de exteriores en Webots.

JASP Software estadístico de código abierto y multiplataforma, desarrollado por la Universidad de Ámsterdam, que ofrece una interfaz gráfica intuitiva para realizar análisis estadísticos clásicos y bayesianos. JASP permite realizar análisis descriptivos, t-tests, ANOVA, regresiones, análisis de fiabilidad, entre otros, con resultados presentados en formato APA. Su diseño facilita la reproducibilidad y la integración con el Open Science Framework, promoviendo la ciencia abierta [49].

- **Versión:** 0.19.3

- **Uso:** Utilizado para analizar y comparar los datos obtenidos de los simuladores, proporcionando herramientas estadísticas avanzadas para interpretar los resultados y extraer conclusiones significativas.

Git Sistema de control de versiones distribuido de código abierto, diseñado para gestionar proyectos de cualquier tamaño con alta eficiencia. Git permite realizar seguimiento detallado de los cambios, gestionar ramas locales, colaboraciones distribuidas y recuperación de versiones anteriores, siendo una herramienta esencial en el desarrollo de software y proyectos colaborativos [44] [105].

- **Versión:** 2.34.1

- **Uso:** Utilizado para descargar ejemplos de código, gestionar repositorios de simulación y controlar versiones de scripts, configuraciones y documentación del TFM.

GitHub Plataforma de desarrollo colaborativo basada en la web, que utiliza Git para gestionar control de versiones distribuido. Permite almacenar, compartir y colaborar en proyectos de código, gestionar issues, pull requests, wikis, CI/CD y acciones automatizadas, siendo la plataforma más grande del mundo para desarrolladores y proyectos open source y comerciales [45] [106].

- **Versión:** Versión en línea
- **Uso:** Utilizado para investigar ejemplos de simuladores, consultar repositorios y documentación relevante, así como alojar el repositorio propio del TFM y controlar versiones del código.

Documentación y colaboración

Overleaf Plataforma para la edición de documentos LaTeX, que permite escribir, editar y publicar documentos científicos de manera eficiente y accesible desde cualquier dispositivo con conexión a Internet. Overleaf facilita la colaboración en tiempo real, el control de versiones y la integración con diversas herramientas y servicios, convirtiéndolo en una opción popular entre investigadores y académicos para la redacción de tesis, artículos y otros documentos técnicos [79].

- **Versión:** Versión en línea
- **Uso:** Utilizado para la redacción y edición de la memoria del TFM, permitiendo la escritura en LaTeX. Además, facilitó la colaboración con el director del TFM mediante la función de comentarios y seguimiento de cambios, optimizando el proceso de revisión y corrección del documento.

LibreOffice Writer & Calc Conjunto de ofimática de código abierto que ofrece herramientas para procesamiento de texto y hojas de cálculo, respectivamente. Writer es un procesador de texto similar a Microsoft Word, mientras que Calc es una hoja de cálculo comparable a Excel. Ambas aplicaciones permiten la creación, edición y análisis de documentos y datos, siendo ampliamente utilizadas en entornos académicos y profesionales [100].

- **Versión:** 7.3.7.2
- **Uso:** Empleadas en las fases preliminares del proyecto para la organización de datos, creación de tablas y anotaciones. También se utilizaron

para visualizar y analizar los datos obtenidos de los simuladores, facilitando la comparación y presentación de resultados.

Google Drive Servicio de almacenamiento en la nube de Google accesible desde navegador, que permite la gestión, sincronización y respaldo de archivos de forma sencilla y segura. Su interfaz web facilita la organización de documentos, control de versiones y búsqueda avanzada [47].

- **Versión:** Versión en línea
- **Uso:** Utilizado para la gestión y respaldo de archivos del proyecto, incluyendo scripts, documentación del TFM y resultados de simulaciones.

Gmail Servicio de correo electrónico de Google, integración con múltiples plataformas y potentes filtros de correo. Ideal para comunicación académica y profesional [46].

- **Versión:** Versión en línea
- **Uso:** Empleado como principal medio de comunicación y gestión con el tutor del TFM.

Google Meet Plataforma de videoconferencias de Google integrada con Gmail y Calendar, que permite realizar reuniones virtuales, compartir pantalla y colaborar en tiempo real [48].

- **Versión:** Versión en línea
- **Uso:** Utilizada para consultas periódicas con tutores y revisiones del TFM mediante reuniones virtuales.

Google Chrome Navegador web de Google basado en Chromium, conocido por su velocidad, estabilidad y ecosistema de extensiones [107].

- **Versión:** 138.0.7204.92
- **Uso:** Utilizado como navegador principal para consultar documentación en línea, acceder a recursos académicos y gestión del proyecto.

Discord Plataforma de comunicación multiplataforma que facilita el uso de canales de voz, texto y video en tiempo real. Muy popular en comunidades técnicas y de desarrollo, permite la creación de servidores con múltiples canales organizados por temas, lo que facilita el intercambio de información, colaboración y soporte de proyectos [32].

- **Versión:** 0.0.100
- **Uso:** Empleado para la comunicación con el tutor y la obtención de información en canales oficiales de Webots [27].

Draw.io (diagrams.net) Herramienta para hacer diagramas en línea gratuita y de código abierto, desarrollada por JGraph Ltd., que permite crear diagramas de flujo, mapas mentales, UML, esquemas de red, organigramas, planos y más. Funciona directamente desde el navegador, con integración a servicios de almacenamiento en la nube, colaboración en tiempo real y autosave, siendo altamente versátil para documentar arquitectura, procesos y diseños de sistemas [51] [104].

- **Versión:** Versión en línea
- **Uso:** Empleada para realizar diagramas explicativos en la documentación del TFM.

Boxy SVG Editor de gráficos vectoriales (SVG), desarrollado por Jarosław Foksa, que permite crear ilustraciones, logotipos, iconos, diagramas y otros elementos de diseño. Disponible como aplicación web y de escritorio para Windows, macOS, Linux y Chrome OS, ofrece edición on-canvas de geometría, rutas, rellenos y propiedades visuales, inspector de código SVG/CSS, exportación a formatos como PNG, JPG, WebP, PDF y HTML5. Su objetivo es generar un SVG limpio y eficiente, preservando metadatos como IDs, clases y títulos. [37] [102].

- **Versión:** 4.2.1 (Escritorio)
- **Uso:** Empleada para crear y editar diagramas con formato SVG para la documentación.

CompressPNG (compresspng.com) Herramienta en línea gratuita para comprimir imágenes PNG, desarrollada por SIA Webby, que aplica compresión con reducción de paleta de color (p. ej., convertir PNG de 24 bits a 8 bits) de forma automática o manual, sin necesidad de registro ni instalación [97].

- **Versión:** Aplicación web
- **Uso:** Utilizada para reducir el tamaño de imágenes PNG en la documentación del TFM.

Monitorización y recursos del sistema

psutil Biblioteca multiplataforma en Python para obtener información detallada de procesos y utilización del sistema (CPU, memoria, discos, red, sensores). Integra funciones de monitorización, perfiles de rendimiento y control de procesos, equivalentes a herramientas Unix como ps, top, free, iostat, entre otras [43].

- **Versión:** 5.9.0
- **Uso:** Utilizada en los scripts desarrollados para acceder a datos de procesos en tiempo real y monitorizar el rendimiento de los simuladores (CPU, RAM, etc.), permitiendo medir consumo de recursos computacionales durante las pruebas.

Powerstat Herramienta de código abierto para sistemas Linux que permite medir el consumo energético de equipos portátiles mediante la información de la batería ACPI o la interfaz RAPL de Intel. Su salida es similar a la de ‘vmstat’, pero incluye estadísticas de consumo energético, calculando al final de la ejecución el promedio, desviación estándar y valores mínimo/máximo de los datos recopilados [4].

- **Versión:** 0.02.27
- **Uso:** Utilizada para medir el consumo energético y evaluar el rendimiento de los simuladores durante las pruebas, proporcionando datos precisos sobre el consumo en vatios durante las ejecuciones.

Anexo D

Tutoriales

A lo largo del proyecto se han realizado los tutoriales de instalación, junto a un ejemplo de uso con ROS 2 y Nav2 de los simuladores Gazebo Classic Ignition Gazebo, Webots e Isaac Sim. Estos se pueden encontrar en el repositorio del proyecto:

- **Gazebo Classic:** https://github.com/JorgeCasatejada02/TFM/blob/main/Tutoriales_Instalacion_Simuladores/Gazebo_Classic.md
- **Ignition Gazebo:** https://github.com/JorgeCasatejada02/TFM/blob/main/Tutoriales_Instalacion_Simuladores/Ignition_Gazebo.md
- **Webots:** https://github.com/JorgeCasatejada02/TFM/blob/main/Tutoriales_Instalacion_Simuladores/Webots.md
- **Isaac Sim:** https://github.com/JorgeCasatejada02/TFM/blob/main/Tutoriales_Instalacion_Simuladores/IsaacSim.md

Anexo E

Inventario Webots ROS 2

E.1. Versiones OS

Está disponibles para los 3 principales sistemas operativos, Linux, Windows y macOS, pero se especifica que para Windows se necesitará WSL2 y para macOS una máquina virtual.

- Linux
- Windows, WSL2
- macOS, VM

E.2. Versiones ROS 2

En la documentación de ROS 2 se encuentra el manual de instalación para los 3 sistemas operativos mencionados y algunos ejemplos del uso de Webots con ROS 2 para las versiones:

- Humble
- Jazzy
- Rolling
- Kilted
- Iron
- Foxy

Sin embargo en el repositorio de Webots ROS 2, `webots_ros2`, se puede ver como en las actions, solo se hacen pruebas sobre las versiones Humble, Jazzy y Rolling. Además en la versión Rolling actualmente (27/06/2025) los test fallan por lo que no se puede asegurar su correcto funcionamiento.

E.3. Sensores

Los sensores actualmente disponibles son (Sensor Devices):

- Camera
- Compass
- DistanceSensor
- GPS
- Lidar
- LightSensor
- RangeFinder
- Receiver
- IMU

Para poder encontrar el modelo (.proto) de estos sensores podemos visitar la página webots.cloud/proto. Para saber que tipo de sensor es el modelo el cual estamos viendo habrá que fijarse en el apartado Keyword, el cual indicará el tipo de sensor, por ejemplo sensor/lidar.

E.4. Actuadores

Los actuadores actualmente disponibles son (Actuator Devices):

- Emitter
- LED
- Gripper
- Motor

Aunque estos 2 últimos no aparezcan en la documentación se puede verificar que si que están disponibles lanzando un ejemplo, por ejemplo el del turtlebot, y usando el comando:

```
$ ros2 control list_controllers
```

Para ver los motores. O también usar los comandos:

```
$ ros2 node list  
$ ros2 topic list
```

Para ver posibles topics de los gripper o motores. Documentación oficial de actuadores

E.5. Robots

Podemos encontrar los modelos de los robots disponible en la página webots.cloud/proto. Además hay una herramienta para convertir los archivos URDF a Proto `urdf2webots`. En esta encontramos 6 categorías, cada una con sus robots correspondientes que son los siguientes:

E.5.1. Con ruedas

Robot/Wheeled

- Altino
- BoeBot
- Create
- E-puck
- Elisa3
- Fabtino
- FireBird6
- FourWheelsRobot
- Hemisson

- JetBot
- Khepera1
- Khepera2
- Khepera3
- Khepera4
- Koala
- Microbot
- MindstormsRover
- Mir100
- Moose
- Pioneer2
- Pioneer3at
- Pioneer3dx
- Pr2
- Robotino3
- Rosbot
- RosbotXl
- Shrimp
- Sojourner
- SummitXlSteel
- Thymio2
- Tiago
- Tiago++
- TiagoBase
- TiagoLite
- TurtleBot3Burger
- Youbot

E.5.2. Con piernas

Robot/Legged

- AiboErs7
- Atlas
- BioloidDog
- Darwin-op
- Darwin-opHinge2
- GhostDog
- Hoap2
- Khr2hv
- Khr3hv
- Mantis
- Nao
- QRIO
- RobotisOp2
- RobotisOp2Hinge2
- RobotisOp3
- Salamander
- Scout
- Spot

E.5.3. Brazos robóticos

Robot/Arm

- IprHd6m180
- IprHd6m90
- IprHd6ms180
- IprHd6ms90

- Irb4600-40
- LBRiiwa7R800
- Ned
- P-Rob3
- Panda
- Puma560
- ScaraT6
- UR10e
- UR3e
- UR5e

E.5.4. Voladores

Robot/Flying

- Blimp
- Crazyflie
- Mavic2Pro

E.5.5. Otros

Robot/Other

- BB-8
- Pedestrian
- Portal
- TinkerbotsBase
- Yamor

E.6. Vehículos

También existen modelos de vehículos como coches, buses, camiones, tractores, furgonetas o motocicletas. Se encuentran aquí: webots.cloud/proto

De todos estos robots encontramos ejemplos ya implementados en `webots_ros2` de:

- Crazyflie
- E-puck
- Husarionn (Rosbot y RosbotXL)
- Mavic
- Tesla (coche)
- Tiago
- Turtlebot3
- UR3e, UR5e y UR10e

E.7. Mundos

Hay una gran número de mundos disponibles. En su documentación por ejemplo podemos encontrar mundos como:

- City
- City night
- City traffic
- CH Morges
- CH Vens
- Highway
- Highway overtake
- SUMO interface example
- Village
- Village center
- Village winter
- Realistic Village
- apartment.wbt

- break_room.wbt
- complete_apartment.wbt
- factory.wbt
- hall.wbt
- kitchen.wbt
- anaglyph.wbt
- gantry.wbt
- hexapod.wbt
- moon.wbt
- soccer.wbt
- stewart_platform.wbt

Estos pueden ser encontrados en las páginas:

- Automobile Worlds
- Sample Environments
- Sample Demos

También hay que destacar que cuenta con un importador de OpenStreetMap que facilita la creación de nuevos entornos para simulaciones y permite generar mundos de Webots a partir de mapas de OpenStreetMap utilizando un script de importación (OpenStreetMap Importer).

En sus repositorios de github también hay un gran número de mundos, como se puede ver reflejado si abrimos el simulador y navegamos al menú File→Open Sample World el cual nos ofrece una amplia variedad de ejemplos para muchos robots y escenarios distintos.

E.8. Nodos

Webots ROS 2 proporciona varios nodos clave, Webots ROS 2 Nodes, que permiten integrar simulaciones con ROS 2 de forma flexible y parametrizable. A continuación se detallan los principales nodos y sus argumentos.

E.8.1. WebotsLauncher

Este nodo se encarga de lanzar Webots como proceso externo desde ROS 2, utilizando argumentos equivalentes a los que pueden pasarse por línea de comandos (Documentación oficial).

Parámetros principales:

- **gui** (bool): Si es False, añade automáticamente los flags:
 - **-no-rendering**: desactiva la visualización de gráficos.
 - **-stdout**, **-stderr**: redireccionan la salida del simulador.
 - **-minimize**: minimiza la ventana de Webots.
 - Equivalente CLI:

```
$ webots --no-rendering --stdout --stderr --minimize
```

- **mode** (str): Establece el modo de ejecución (realtime, fast or pause) usando **-mode**.
 - Equivalente CLI:

```
$ webots --mode=realtime
```

- **stream** (bool): Si es True, activa el flag **-stream** para permitir el streaming de visualización.
 - Equivalente CLI:

```
$ webots --stream
```

- **port** (int): Puerto de la instancia de Webots para conexión con otros procesos (por defecto: 1234).
- **world** (str): Ruta al archivo .wbt del mundo a lanzar. Puede ser cadena literal o `launch.Substitution`.
- **output** (str): (Por defecto 'screen') determina el destino de la salida del proceso.

- `ros2_supervisor` (bool): Si es True, lanza el nodo Ros2Supervisor, que permite:
 - Publicar el tiempo de simulación en `/clock`.
 - Spawn de robots desde URDF o PROTOs.
 - Control avanzado sobre la simulación desde ROS 2.

E.8.2. WebotsController

Desde la versión 2023.1.0 del paquete `webots_ros2`, el nodo `WebotsController` (heredado de `ExecuteProcess`) se encarga de lanzar los controladores ROS 2 que interactúan con robots simulados en Webots.

Parámetros principales:

- `robot_name` (str): Nombre del robot simulado con el que conectar.
- `port` (int): Puerto al que conectarse. Por defecto: 1234.
- `namespace` (str): Namespace ROS para este controlador.
- `parameters` (list de diccionarios): Lista con parámetros a pasar:
 - `use_sim_time` (bool): Utiliza `/clock` del simulador como fuente de tiempo.
 - `robot_description` (str): Ruta a un archivo URDF. Este se interpreta para configurar el nodo ROS y activar sensores, plugins, etc.
 - `set_robot_state_publisher` (bool): Si es True, el URDF generado desde Webots (PROTO $\rightarrow$ URDF) se usará en `robot_state_publisher`.
 - `components_remapping` (str): Ruta a YAML con equivalencias de nombres de componentes para mapear nombres del URDF generado a nombres compatibles con ROS estándar (útil para `sim2real`).

También se pueden incluir archivos YAML completos para configuraciones como `ros2_control`.

E.8.3. WaitForControllerConnection

Este nodo permite controlar la sincronización de otros nodos dependiendo de la conexión del controlador principal del robot. Útil para evitar mensajes caóticos al inicio.

Parámetros:

- **target_driver** (str): Nombre del nodo driver de referencia.
- **nodes_to_start** (list): Lista de nodos que se iniciarán automáticamente una vez que el **target_driver** esté conectado correctamente al simulador.